CSE 489 · ASSIGNMENT 1

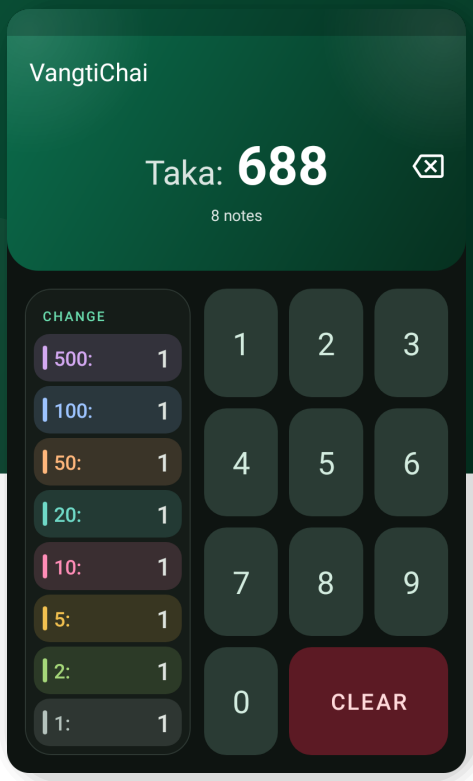
Mobile Application Development, BRAC University

VangtiChai

ভাংতি চাই · *"I need change"*

The Complete Project Guide

A from-scratch, step-by-step explanation of the VangtiChai Android app: what every file does, what every line of code means, why each decision was made, and what could be done differently.



WHAT'S INSIDE

- Every part of PROJECT_GUIDE.md: the assignment, Android and Kotlin basics, a step-by-step build, design decisions, testing, improvements, history and a glossary
- Every file in the repository: all 47 text files printed in full with line numbers, and the 27 binary files (icons, screenshots and the wrapper JAR) listed and shown
- Added explanations, worked examples and screenshots of the app

Kotlin · XML Views · ConstraintLayout · Material 3 · minSdk 21 · targetSdk 37

Describes the project as of 15 October 2026
github.com/ahsanauddry027/VhangtiChai_AndroidApp_CSE489

PDF edition of PROJECT_GUIDE.md
15 October 2026

About this guide

Describes the project as of 2026-10-15.

> **What this file is.** A from-scratch, step-by-step explanation of the VangtiChai Android app: what every file does, what every line of code means, why each decision was made, and what could be done differently. It assumes **no prior Android or Kotlin knowledge**.
>
> **Who it is for.**
>
> - **Beginners** who want to rebuild this app themselves and understand every part of it.
> - **AI coding assistants** (such as Claude Code) that need full context before changing the project. If you are an assistant, read Part 0 first. It is a dense summary of the facts, rules and decisions you must respect.
>
> **How it is organised.** Part 0 is a summary for assistants. Part 1 is the assignment. Part 2 teaches Android and Kotlin from the basics. Part 3 builds the project step by step. Parts 4 to 7 cover decisions, testing, better alternatives and history. The appendix has the complete source code of every file, copied directly from the project.

About this PDF edition

This PDF contains all of `PROJECT_GUIDE.md`: every part, table and code example, and the complete source code from its appendix. The guide's text is unchanged; only its list of contents has been replaced by the page-numbered contents that follows. A few things were added:

- **In detail** boxes explain further: worked examples with real numbers, how Android chooses between resource folders, a rotation step by step, the row-fade colours in numbers, and the documentation commits the project history table leaves out.
- **About this file** boxes in the appendix say what each source file is for and point to its important lines.
- **A.9 The rest of the repository** adds what the guide's appendix leaves out: `README.md` and the two Gradle wrapper scripts in full, and all 27 binary files. With it, this PDF covers every one of the 75 files in the repository.
- **Figures** show the app itself, using the screenshots from `docs/images/`: next to the steps that build those screens, on the emulators it was tested on (Part 5.2), and the launcher icons.
- Source listings have **line numbers**. A line too long for the page continues on the next line, marked with ↪.

The green boxes are the guide's own notes and the amber ones its warnings; the blue boxes are the additions. The contents pages, the cross-references and the PDF bookmarks are all clickable.

Contents

Part 0: Quick context for AI assistants . . . 7

What the app is . . . 7

Facts . . . 7

File map (only the files that matter) . . . 8

Hard rules you must keep . . . 8

Behaviour summary . . . 9

How to build and test . . . 10

Conventions . . . 10

Part 1: The assignment and how each requirement is met . . . 11

What the brief asks for . . . 11

Requirement-by-requirement . . . 11

Extras beyond the brief . . . 12

Part 2: Android and Kotlin from the basics . . . 13

2.1 How an Android app works: the big picture . . . 13

2.2 The tools you need . . . 13

2.3 The project folder structure . . . 14

2.4 Gradle: the build system, file by file . . . 14

2.5 SDK versions explained . . . 16

2.6 The manifest . . . 16

2.7 Resources, the `R` class, qualifiers and units . . . 17

2.8 Views, ViewGroups and XML layouts . . . 19

2.9 ConstraintLayout in depth . . . 19

2.10 Activities, the lifecycle and saving state . . . 21

2.11 View Binding . . . 21

2.12 Themes, styles and Material 3 . . . 22

2.13 Kotlin crash course, with examples from this project . . . 22

Part 3: Building VangtiChai from scratch, step by step . . . 26

Step 1: Install Android Studio and create the project . . . 26

Step 2: Configure Gradle . . . 27

Step 3: The manifest . . . 27

Step 4: Colours (`res/values/colors.xml`) . . . 27

Step 5: Text (`res/values/strings.xml`) . . . 29

Step 6: Sizes (`res/values*/sizes.xml`) . . . 29

Step 7: Integers and the theme . . . 30

Step 8: Widget styles (`res/values/styles.xml`) . . . 31
Step 9: Drawables . . . 31
Step 10: The key-press animation (`res/animator/key_press.xml`) . . . 32
Step 11: The logic: `ChangeCalculator.kt` . . . 33
Step 12: One change-table row (`res/layout/view_note_row.xml`) . . . 35
Step 13: The phone portrait layout (`res/layout/activity_main.xml`) . . . 35
Step 14: The phone landscape layout (`res/layout-land/activity_main.xml`) . . . 39
Step 15: The tablet layouts (`layout-sw600dp/` and `layout-sw600dp-land/`) . . . 39
Step 16: The screen's behaviour, `MainActivity.kt`, section by section . . . 40
16.1 The core idea: state, then render . . . 40
16.2 Fields (the class's variables) . . . 41
16.3 `onCreate`: setting everything up . . . 42
16.4 `onSaveInstanceState`: surviving rotation . . . 44
16.5 `onKeyDown`: hardware keyboards . . . 44
16.6 Input handlers . . . 44
16.7 `rejectDigit`: telling the user "no" . . . 45
16.8 `render`: drawing the state . . . 46
16.9 `renderBackspace`: showing the key only when useful . . . 46
16.10 Rows: `renderRow`, `lightRow` and `setRowLevel` . . . 47
16.11 `applySystemBarInsets`: edge-to-edge done right . . . 48
16.12 `playEntrance`: the opening animation . . . 49
16.13 Two small helpers (extension functions) . . . 49
16.14 The constants . . . 50
Step 17: Tests . . . 50
17.1 Unit tests: `app/src/test/java/com/example/vangtichai/ChangeCalculatorTest.kt` . . . 50
17.2 UI tests: `app/src/androidTest/java/com/example/vangtichai/MainActivityTest.kt` . . . 51
Step 18: Run it . . . 52
Step 19: README.txt, Git and GitHub . . . 52

Part 4: Every design decision and why . . . 53

Part 5: How the app was tested and verified . . . 56
5.1 Unit tests and Espresso tests . . . 56
5.2 Emulator testing: the required devices, more screen sizes and Android 5.0 . . . 56
5.3 The Robolectric "render harness" (how the README screenshots were made) . . . 59
5.4 Devices covered . . . 60

Part 6: Better ways and possible improvements . . . 61
Architecture . . . 61
Layout . . . 61
Polish and features . . . 61

Quality and tooling . . . 62
Things that would be *worse* (avoid) . . . 62

Part 7: Project history . . . 63

Part 8: Glossary . . . 64

Appendix: Complete source code . . . 66

A.1 Build configuration . . . 66
`settings.gradle.kts` . . . 66
`build.gradle.kts` . . . 67
`app/build.gradle.kts` . . . 67
`gradle/libs.versions.toml` . . . 68
`gradle.properties` . . . 70
`gradle/wrapper/gradle-wrapper.properties` . . . 70
`gradle/gradle-daemon-jvm.properties` . . . 71
`.gitignore` . . . 72
`.gitattributes` . . . 72
`app/.gitignore` . . . 73
`app/src/main/keepRules/rules.keep` . . . 73
A.2 Manifest . . . 74
`app/src/main/AndroidManifest.xml` . . . 74
A.3 Kotlin source . . . 75
`app/src/main/java/com/example/vangtichai/ChangeCalculator.kt` . . . 75
`app/src/main/java/com/example/vangtichai/MainActivity.kt` . . . 76
A.4 Tests . . . 83
`app/src/test/java/com/example/vangtichai/ChangeCalculatorTest.kt` . . . 83
`app/src/androidTest/java/com/example/vangtichai/MainActivityTest.kt` . . . 85
A.5 Values . . . 87
`app/src/main/res/values/colors.xml` . . . 87
`app/src/main/res/values/strings.xml` . . . 89
`app/src/main/res/values/sizes.xml` . . . 90
`app/src/main/res/values-land/sizes.xml` . . . 93
`app/src/main/res/values-sw600dp/sizes.xml` . . . 94
`app/src/main/res/values-sw600dp-land/sizes.xml` . . . 95
`app/src/main/res/values/integers.xml` . . . 96
`app/src/main/res/values/themes.xml` . . . 96
`app/src/main/res/values/styles.xml` . . . 97
A.6 Layouts . . . 100
`app/src/main/res/layout/view_note_row.xml` . . . 100
`app/src/main/res/layout/activity_main.xml` . . . 101
`app/src/main/res/layout-land/activity_main.xml` . . . 107

app/src/main/res/layout-sw600dp/activity_main.xml . . . 113
app/src/main/res/layout-sw600dp-land/activity_main.xml . . . 119
A.7 Drawables, icons and animators . . . 125
app/src/main/res/drawable/bg_header.xml . . . 125
app/src/main/res/drawable-v23/bg_header.xml . . . 125
app/src/main/res/drawable/bg_note_row.xml . . . 127
app/src/main/res/drawable/bg_note_accent.xml . . . 127
app/src/main/res/drawable/ic_backspace.xml . . . 127
app/src/main/res/drawable/ic_launcher_background.xml . . . 128
app/src/main/res/drawable/ic_launcher_foreground.xml . . . 128
app/src/main/res/drawable/ic_launcher_monochrome.xml . . . 129
app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml . . . 130
app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml . . . 130
app/src/main/res/animator/key_press.xml . . . 131
app/src/main/res/xml/backup_rules.xml . . . 132
app/src/main/res/xml/data_extraction_rules.xml . . . 132
A.8 README.txt (assignment deliverable) . . . 133
README.txt . . . 133
A.9 The rest of the repository . . . 136
README.md . . . 136
gradlew . . . 140
gradlew.bat . . . 144
Binary files . . . 146

PART 0

Quick context for AI assistants

What the app is

VangtiChai (Bangla ভাংতি চাই, "I need change") is a single-screen Android app for **CSE 489: Mobile Application Development, Assignment 1** at BRAC University. The user types an amount in Taka on an on-screen keypad. The app shows how many of each Taka note (500, 100, 50, 20, 10, 5, 2, 1) make up that amount, always using the fewest notes.

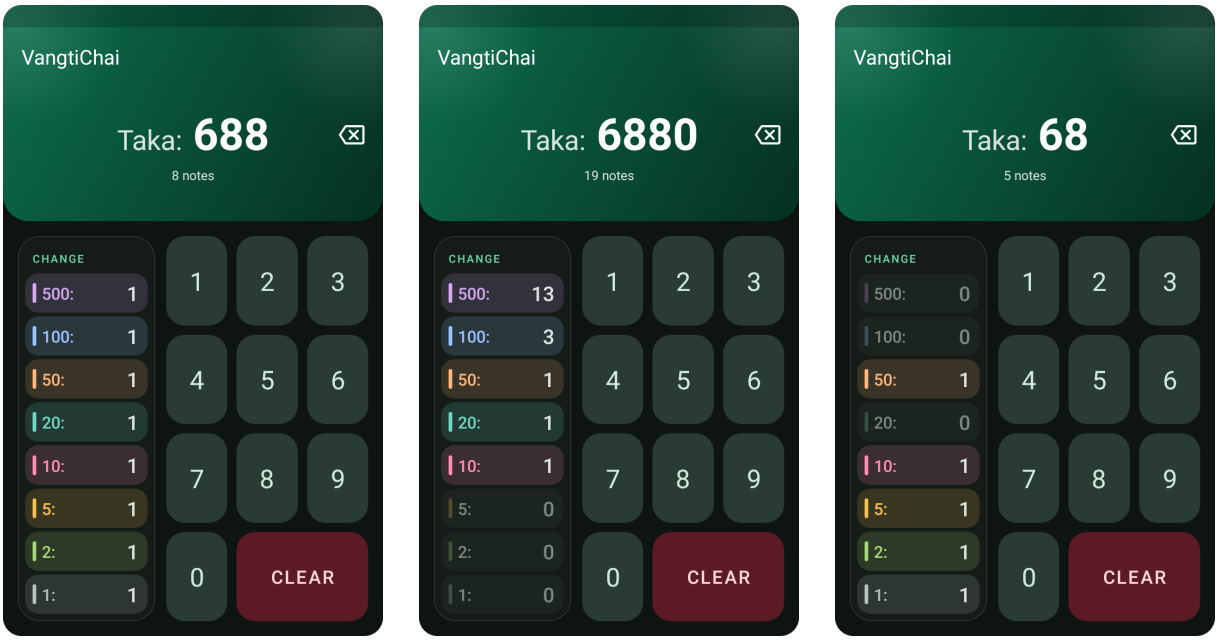


Figure 1. *VangtiChai on a phone in portrait. Left: 688 Taka takes one of each note, 8 notes in all. Middle: 6880 Taka takes 19 notes (13 × 500, 3 × 100 and one each of 50, 20 and 10); the 5, 2 and 1 rows are not needed, so they fade back. Right: 68 Taka takes 5 notes.*

Facts

Item	Value
Language	Kotlin (no Java sources)
UI toolkit	Classic Android Views with XML layouts (**not** Jetpack Compose)
Main layout	`ConstraintLayout` everywhere (the assignment focuses on it)
Architecture	One `Activity` (`MainActivity`) plus one pure-logic `object` (`ChangeCalculator`). No ViewModel, no fragments, no database.
Package / applicationId	`com.example.vangtichai`
minSdk / targetSdk / compileSdk	21 / 37 / 37 (21 is the minimum the assignment suggests)

Item	Value
Build	Gradle 9.6.0 wrapper, Android Gradle Plugin (AGP) 9.4.1, Kotlin DSL (`*.gradle.kts`), version catalog `gradle/libs.versions.toml`. AGP 9 compiles Kotlin itself, so there is no separate Kotlin plugin.
Dependencies	`core-ktx`, `appcompat`, `activity-ktx`, `constraintlayout`, `material`. Tests: `junit`, `androidx.test.ext:junit`, `espresso-core`. The versions are the newest that still support minSdk 21 (see rule 9).
View access	View Binding (`buildFeatures { viewBinding = true }`)
Theme	**Always dark.** `Theme.Material3.Dark.NoActionBar`. There is no `values-night` folder and no light theme.
Repository	`https://github.com/ahsanauddry027/VhangtiChai_AndroidApp_CSE489` (branch `main`)

File map (only the files that matter)

```
app/src/main/java/com/example/vangtichai/
    MainActivity.kt        All UI behaviour: keypad, backspace, rendering, animations, state saving, insets
    ChangeCalculator.kt    Pure logic (no Android imports): note breakdown, digit append/remove, limits
app/src/main/res/
    layout/activity_main.xml                Phone portrait   (designed for Pixel XL, 411 x 731 dp)
    layout-land/activity_main.xml           Phone landscape  (731 x 411 dp)
    layout-sw600dp/activity_main.xml        Tablet portrait  (Nexus 10, 800 x 1280 dp)
    layout-sw600dp-land/activity_main.xml   Tablet landscape (1280 x 800 dp)
    layout/view_note_row.xml                One change-table row, <include>d 8 times per layout
    values/sizes.xml (+ -land, -sw600dp, -sw600dp-land)   ALL dimensions and proportions
    values/strings.xml     All user-visible text, plurals, accessibility descriptions
    values/colors.xml      All colours (dark palette only)
    values/styles.xml      Widget styles (keys, backspace, amount, caption, card, rows)
    values/themes.xml      App theme (Material3 Dark) and colour roles
    values/integers.xml    Small integer constants used by XML (max lines, key press duration)
    animator/key_press.xml Keys shrink to 93% while pressed
    drawable/              Header background (plain), row/tab shapes, backspace icon, launcher icon layers
    drawable-v23/          Header background with its two glows, used from API 23 on
app/src/test/.../ChangeCalculatorTest.kt       JVM unit tests (11 tests)
app/src/androidTest/.../MainActivityTest.kt    Espresso UI tests (7 tests, need device/emulator)
README.txt   Required by the assignment: describes the app and lists tested devices
README.md    GitHub landing page with screenshots (images in docs/images/)
```

Hard rules you must keep

1. **No hardcoded values in layout XML.** The assignment says sizes, paddings and margins must come from `sizes.xml`. Every number in a layout is a `@dimen/...`, `@integer/...` or `@string/...` reference, even ConstraintLayout's `0dp`, which is written `@dimen/match_constraint`. Only `tools:` preview text, which never reaches the app, contains literal numbers. Add new values to `values/sizes.xml`, and override them in `values-land`, `values-sw600dp` or `values-sw600dp-land` only where they differ. Android Studio's design view writes a literal `0dp` when you pick "match constraints" there; replace it with `@dimen/match_constraint`.
2. **Four layouts, all `ConstraintLayout`.** Any view you add must be added to **all four** `activity_main.xml` files with the **same id**. View Binding generates one class from all of them. An id missing from some layouts becomes a nullable field.
3. **Keypad configuration is fixed by the assignment brief.**
   - Portrait: rows `1 2 3`, `4 5 6`, `7 8 9`, then `0` + a wide `CLEAR`.
   - Landscape: rows `1 2 3 4`, `5 6 7 8`, then `9 0` + a wide `CLEAR`.

   Do **not** add keys to the grid. The backspace key lives in the header for this reason.
4. **Digits are added from the right** (typing 2, 3, 4 shows 2, 23, 234). The amount is stored as a `String` of digits.

5. **Rotation must keep the amount and the change list.** This is done with `onSaveInstanceState()` / `onCreate(savedInstanceState)`. **Do not** add `android:configChanges` to the manifest. The activity must be recreated so Android loads the landscape or portrait layout.
6. **Must work on Pixel XL (411x731 dp) and Nexus 10 (800x1280 dp), both orientations.** It is tested on Pixel XL and Nexus 10 emulators, on the Nexus 5 (360x640), Pixel 7-sized phone (411x914), Nexus 7 (600x960) and Pixel C (900x1280) screen sizes emulated on them, and on an Android 5.0 (API 21) emulator.
7. `README.txt` is a required deliverable. It must list the other devices and screen types tested. Keep it updated when behaviour changes.
8. The root layout sets `android:layoutDirection="ltr"`, so "start" always means left and "end" always means right, even in right-to-left languages.
9. **The minimum SDK is 21 (Android 5.0), as the assignment suggests.** To keep it working:
   - Don't upgrade `core-ktx`, `appcompat`, `activity-ktx` or `material` past the versions in `libs.versions.toml`. Their newer releases require API 23, and the build then fails at the manifest merge.
   - Don't colour shape (`GradientDrawable`) backgrounds with `backgroundTint`. Android 5.0 ignores it and the shape stays white. `MainActivity` colours the change table's rows with `GradientDrawable.setColor()` instead.
   - Layer-list items only support `width`, `height` and `gravity` from API 23. That is why the header with glows is in `drawable-v23/`, and `drawable/bg_header.xml` is a plain gradient for API 21 and 22.
   - Any framework call newer than API 21 needs an `if (Build.VERSION.SDK_INT >= ...)` check. `./gradlew lintDebug` reports missing ones (`NewApi`).

Behaviour summary

- **Input limit:** `ChangeCalculator.MAX_DIGITS = 9` (up to 999,999,999 Taka). A 10th digit is rejected: the amount shakes, the phone gives reject haptics, and the caption shows "Up to 9 digits" for 1.6 s.
- **Leading zero:** typing `0` then `5` gives `5`, not `05`.
- **Change calculation:** greedy algorithm (take the biggest note that fits, repeat). It is optimal because the Taka note set is canonical. A unit test proves this against dynamic programming for 0 to 2000.
- **Backspace key:** an icon button at the right end of the amount row, like the one in a phone dialer.
  - A tap deletes one digit.
  - Holding it deletes one digit at the long-press point, then one every 70 ms until the finger lifts.
  - It is `INVISIBLE` (not `GONE`) when the amount is empty, and fades in and out over 150 ms.
  - A hardware keyboard's Backspace key routes through it.
- **Hardware keyboard:** digits and numpad digits type; Backspace deletes; Esc, Clear or C clears.
- **Rendering:**
  - The amount shows as `Taka: 688`, with "Taka:" at 62% size in a softer colour and the number in bold. The text auto-sizes to fit.
  - The caption shows "Enter an amount" or "N notes".
  - Each row lights up in its note's colour when its count is above 0. Background, colour tab, label and count fade together over 220 ms. The lit colours are pre-composited (opaque) so the fade never flashes brighter.
  - Counts "pop" (scale 1.35, then spring back) when they change, and the amount pops at 1.06.
- **Entrance animation:** on a fresh start only (not after rotation), the change table and then the keypad fade in while rising 24 dp (40 dp on tablets).

- **Edge-to-edge:** the app draws behind the system bars. The header extends under the status bar. The table and keypad are pushed clear of the navigation bar and display cutouts with a `WindowInsets` listener. Both bars use light icons.
- **Accessibility:**
  - The amount is a polite live region, so it is announced when it changes.
  - Each row has one spoken description, such as "1 note of 500 Taka".
  - The backspace key has the description "Delete the last digit".

How to build and test

```
./gradlew assembleDebug            # build the APK (app/build/outputs/apk/debug/)
./gradlew test                     # 11 JVM unit tests (ChangeCalculatorTest)
./gradlew connectedAndroidTest     # 7 Espresso tests, needs a running emulator or device
```

On Windows use `gradlew.bat`. The Gradle daemon runs on JDK 25, which the foojay plugin downloads automatically.

Conventions

- Code comments explain *why*, briefly, in plain English. Every non-trivial function has a one-line KDoc (`/** ... */`).
- Animation timings and magic numbers live in `MainActivity`'s `private companion object` as `const val`s with clear names.
- Pure logic goes in `ChangeCalculator` so it can be unit tested without Android.
- Commit messages use a short imperative title ("Add ...", "Make ..."), then a body explaining what and why.

PART 1

The assignment and how each requirement is met

What the brief asks for

The assignment PDF ("CSE 489 Assigment L1 v2.pdf", kept out of git) says:

> **Objective:** The assignment focuses on layouts, especially the ConstraintLayout, general Android programming concepts, such as the activity lifecycle and designing for different screen sizes and orientations.
>
> Create an Android app (VangtiChai) that lets you enter a money amount via a numeric keypad implemented from scratch (to practice layouts!), and calculates change for that amount in terms of Taka notes (500, 100, 50, 20, 10, 5, 2, 1).

It then lists the requirements, which are mapped to the implementation below. It also includes reference screenshots:

- Portrait: "Taka: 688" at the top, a single-column change table on the left, and a 3-column keypad on the right.
- Landscape: the change table in two columns (500/100/50/20 and 10/5/2/1), and a 4-column keypad.

Requirement-by-requirement

#	Requirement (from the brief)	How VangtiChai meets it	Where
1	Portrait: a numeric keyboard with buttons for the 10 digits and a clear button, "in the configuration shown".	3-column keypad: `1 2 3 / 4 5 6 / 7 8 9 / 0 CLEAR`. CLEAR spans two columns.	`layout/activity_main.xml`, `layout-sw600dp/activity_main.xml`
1	To the left of it, a table listing the notes that make up the change.	A `MaterialCardView` on the left holds 8 rows (`view_note_row.xml`), from 500 down to 1.	same files
1	In the top part of the screen, the entered amount next to a "Taka:" label.	The header shows "Taka: 688" (the label and number are one styled text).	`amount_text` in every layout
2	Each tap adds the digit to the right of the amount (2, 3, 4 shows 2, then 23, then 234).	`ChangeCalculator.appendDigit()` appends to a `String`.	`ChangeCalculator.kt`
3	On rotation, keep the entered amount and change list, and show a "slightly altered layout".	`onSaveInstanceState()` saves both, and `onCreate()` restores them. The landscape layouts have a 2-column table and a 4-column keypad.	`MainActivity.kt`, `layout-land/`, `layout-sw600dp-land/`

#	Requirement (from the brief)	How VangtiChai meets it	Where
4	Layouts work gracefully on phones and tablets. Test with the emulator on at least a Pixel XL and a Nexus 10.	Four layouts chosen by resource qualifiers (`-land`, `-sw600dp`). Tested on Pixel XL and Nexus 10 emulators in both orientations, plus four more screen sizes and Android 5.0.	Part 5
Hint	"4 different layouts" are enough for full credit.	Exactly four `activity_main.xml` variants.	`res/layout*/`
Hint	List the other devices and screen types you tested in `README.txt`.	`README.txt` lists the two required devices, four more screen sizes (in px, dpi and dp) and Android 5.0.	`README.txt`
Hint	Do not hardcode values in layout XML. Put sizes in `sizes.xml` and reference them.	Every size is a `@dimen` from one of the four `sizes.xml` files, even ConstraintLayout's `0dp` (`@dimen/match_constraint`). Text is in `strings.xml` and colours in `colors.xml`.	`res/values*/sizes.xml`
Hint	Kotlin, Java or Flutter, and a minimum API level of 21 ("or lower if need be").	Kotlin, and min SDK 21 (Android 5.0). The libraries are pinned to their last releases that support API 21, and the app was run on an Android 5.0 emulator.	`app/build.gradle.kts`, `gradle/libs.versions.toml`

Every requirement is met. Each one was checked on the final build with the Android Emulator: on the Pixel XL and the Nexus 10 that the brief names, in portrait and landscape, and on Android 5.0, the minimum version (Part 5.2).

Extras beyond the brief

All of these were added on top of the requirements, without breaking any of them:

- Material 3 dark design, with a gradient header and colour-coded notes
- Smooth row fades, count pops and an entrance animation
- Haptic feedback on every key
- An on-screen backspace key (tap to delete one digit, hold to repeat)
- Hardware keyboard support
- A 9-digit limit with a shake and a message
- Edge-to-edge drawing that handles display cutouts
- Screen reader support
- A custom adaptive launcher icon with a themed (monochrome) version
- Unit tests and Espresso tests

PART 2

Android and Kotlin from the basics

This part explains the ideas you need before reading the code. Each concept is followed by where it appears in VangtiChai.

2.1 How an Android app works: the big picture

An Android app is a package file (**APK**, or AAB for the Play Store) that contains three things:

1. **Compiled code.** Your Kotlin is compiled to JVM bytecode, then converted ("dexed") to Android's **DEX** format, which the Android Runtime (ART) runs.
2. **Resources.** Layouts (screens described in XML), text, colours, sizes, images and icons.
3. **The manifest** (`AndroidManifest.xml`). It tells Android what the app is called, what its icon is, and which screen to open first.

When you tap the app icon:

1. Android reads the manifest, finds the **launcher activity**, and starts it. An *Activity* is one screen. VangtiChai has exactly one, `MainActivity`.
2. Android calls the activity's `onCreate()` method. Your code there **inflates** a layout (turns XML into real on-screen objects called *Views*), and sets up click listeners.
3. From then on the app is **event-driven**. Nothing happens until the user does something, such as tapping a key. Then Android calls your listener, your code updates some state, and you update the Views.

Everything happens on one thread, the **main thread** (or UI thread). It draws the screen about 60 to 120 times a second, so your code must never block it with slow work. VangtiChai's work is tiny (a few divisions), so everything runs on the main thread.

2.2 The tools you need

Tool	What it is	In this project
Android Studio	The official IDE (code editor + layout editor + emulator manager + debugger).	Used to create and run the project.
Android SDK	The libraries and tools for building against Android versions (installed by Android Studio).	`compileSdk 37`
Emulator (AVD)	A virtual phone or tablet on your computer. *AVD* = Android Virtual Device.	Pixel XL and Nexus 10 profiles for the assignment.
JDK	Java Development Kit. Gradle and the Kotlin compiler run on it.	Android Studio's bundled JBR. The Gradle daemon uses JDK 25, auto-downloaded.
Gradle	The build system. It compiles code, processes resources, runs tests and packages the APK.	Gradle 9.6.0 through the **wrapper** (`gradlew`).
AGP	Android Gradle Plugin: teaches Gradle how to build Android apps.	9.4.1
Git	Version control.	Repo on GitHub.

2.3 The project folder structure

```
CSE489_App_VhangtiChai/                the project root
├── settings.gradle.kts                which modules exist, where to download libraries from
├── build.gradle.kts                   root build file (declares plugins, does nothing else)
├── gradle.properties                  Gradle settings (memory, configuration cache, code style)
├── gradle/
│   ├── libs.versions.toml             the "version catalog": every library and version in one place
│   ├── gradle-daemon-jvm.properties   which JDK version the Gradle daemon uses (25)
│   └── wrapper/                       the Gradle wrapper (downloads the right Gradle automatically)
├── gradlew, gradlew.bat               run Gradle: ./gradlew (macOS/Linux), gradlew.bat (Windows)
├── local.properties                   path to your Android SDK (machine-specific, NOT committed)
├── README.txt / README.md             documentation
├── docs/images/                       screenshots used by README.md and the guide's PDF
└── app/                               the one and only module: the app itself
    ├── build.gradle.kts               app build settings: SDK levels, dependencies, view binding
    └── src/
        ├── main/                      the real app
        │   ├── AndroidManifest.xml
        │   ├── java/com/example/vangtichai/   Kotlin source (the folder is called "java" by convention)
        │   ├── keepRules/rules.keep           R8 keep rules (for release shrinking; template default)
        │   └── res/                            resources (see 2.7)
        ├── test/                      JVM unit tests (run on your computer, fast)
        └── androidTest/               instrumented tests (run on a device or emulator)
```

A **module** is a buildable unit. Big apps have many (`:app`, `:core`, `:feature-x`). This project has one, `:app`.

2.4 Gradle: the build system, file by file

Gradle files written in Kotlin end in `.gradle.kts` (Kotlin DSL). They are Kotlin code, so you can use Kotlin syntax inside them.

`settings.gradle.kts` runs first. It says:

- `pluginManagement { repositories { google(); mavenCentral(); gradlePluginPortal() } }`: where to download *plugins* (like AGP) from. The `google { content { includeGroupByRegex(...) } }` filter makes Gradle only ask Google's repository for `com.android.*`, `com.google.*` and `androidx.*` packages. This is faster and safer.
- `plugins { id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0" }`: lets Gradle auto-download the right JDK ("toolchain") if you don't have it.
- `dependencyResolutionManagement { repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS); repositories { google(); mavenCentral() } }`: where to download *libraries*. The FAIL mode forbids modules from declaring their own repositories, so there is one source of truth.
- `rootProject.name = "VangtiChai"` and `include(":app")`: the project name and its single module.

`gradle/libs.versions.toml` is the **version catalog**. Instead of writing `"androidx.core:core-ktx:1.17.0"` in every build file, you name it once:

```
[versions]
coreKtx = "1.17.0"                      # a version number, referenced below

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
```

In build files you then write `libs.androidx.core.ktx`, where dashes become dots. To upgrade, change one line.

`build.gradle.kts` **(root)**:

```
plugins {
    alias(libs.plugins.android.application) apply false   // make AGP available to modules, don't apply it
      ↪ here
}
```

`app/build.gradle.kts` is the important one:

```
plugins {
    alias(libs.plugins.android.application)     // this module is an Android *application* (not a library)
}

android {
    namespace = "com.example.vangtichai"        // package of the generated R and binding classes
    compileSdk { version = release(37) }        // compile against Android API 37 (newer AGP syntax)

    defaultConfig {
        applicationId = "com.example.vangtichai" // the app's unique ID on a device / the Play Store
        minSdk = 21                              // oldest Android version that can install it (5.0)
        targetSdk = 37                           // the version whose behaviour the app is tested against
        versionCode = 1                          // integer version, must increase for each release
        versionName = "1.0"                      // human-readable version
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"  // runs androidTest/
    }

    buildTypes {
        release {
            optimization {                       // newer AGP DSL from the project template: turns on R8
                enable = true                    //   shrinking/optimisation for release builds
                packageScope = setOf("androidx.**", "kotlin.**", "kotlinx.**")
            }
        }
    }
    compileOptions {                             // Java language level of the compiled bytecode
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    buildFeatures {
        viewBinding = true                       // generate *Binding classes for layouts (see 2.11)
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)               // Kotlin extensions for Android
      ↪ (buildSpannedString, isVisible...)
    implementation(libs.androidx.appcompat)              // AppCompatActivity: modern features on old
      ↪ Androids
    implementation(libs.androidx.activity.ktx)           // enableEdgeToEdge()
    implementation(libs.androidx.constraintlayout)       // ConstraintLayout
    implementation(libs.material)                        // Material 3 components: MaterialButton, cards,
      ↪ toolbar
    testImplementation(libs.junit)                       // JUnit 4 for unit tests
    androidTestImplementation(libs.androidx.junit)       // AndroidJUnit4 runner, ActivityScenarioRule
    androidTestImplementation(libs.androidx.espresso.core) // Espresso UI testing
}
```

- `implementation` means the library ships inside the app.
- `testImplementation` means it is only on the classpath of `src/test`.
- `androidTestImplementation` means it is only used for `src/androidTest`.

> **Note: AGP 9 and Kotlin.** Older projects also applied the `org.jetbrains.kotlin.android` plugin. AGP 9 has Kotlin support built in, so this project doesn't need it.

`gradle.properties`:

- `org.gradle.jvmargs=-Xmx2048m` gives Gradle 2 GB of memory.
- `org.gradle.configuration-cache=true` caches the build configuration for speed.
- `kotlin.code.style=official` sets the Kotlin formatting style.

The wrapper. `gradle/wrapper/gradle-wrapper.properties` pins Gradle **9.6.0**, including a SHA-256 checksum for safety. `./gradlew` downloads exactly that version the first time. Everyone gets the same Gradle without installing it.

2.5 SDK versions explained

Setting	Meaning	Value
`minSdk`	Phones older than this cannot install the app.	21 (Android 5.0 Lollipop, 2014), the minimum the assignment suggests
`targetSdk`	Tells Android "I've been tested with this version's behaviour changes".	37
`compileSdk`	Which Android API you *compile* against, so you can use its newest functions.	37

Because `minSdk` is 21, code that uses newer APIs must check the version at runtime. VangtiChai does this once:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) HapticFeedbackConstants.REJECT   // API 30+
else HapticFeedbackConstants.LONG_PRESS                                              // older phones
```

Resources can depend on the Android version too. A folder with a `-vNN` qualifier is only used from API NN on. The header's two soft glows are sized and placed with layer-list features that need API 23, so `drawable-v23/bg_header.xml` has them, and Android 5.0 and 5.1 use the plain gradient in `drawable/bg_header.xml`.

Libraries have a minimum SDK as well. The newest releases of `core-ktx`, `appcompat`, `activity-ktx` and `material` need API 23, so the version catalog pins the last releases that support API 21.

2.6 The manifest

`app/src/main/AndroidManifest.xml` declares the app to Android:

```
<application
    android:allowBackup="true"                                  <!-- allow Android's auto-backup -->
    android:dataExtractionRules="@xml/data_extraction_rules"    <!-- backup rules, Android 12+ (template) -->
    android:fullBackupContent="@xml/backup_rules"               <!-- backup rules, older Androids (template)
      ↪ -->
    android:icon="@mipmap/ic_launcher"                          <!-- the launcher icon -->
    android:label="@string/app_name"                            <!-- the name under the icon: "VangtiChai"
      ↪ -->
    android:roundIcon="@mipmap/ic_launcher_round"               <!-- round icon for launchers that want one
      ↪ -->
    android:supportsRtl="true"                                  <!-- support right-to-left languages -->
    android:theme="@style/Theme.VangtiChai">                    <!-- the app's theme (themes.xml) -->
    <activity
        android:name=".MainActivity"                            <!-- the class
          ↪ com.example.vangtichai.MainActivity -->
        android:exported="true">                                <!-- other apps (the launcher) may start it
          ↪ -->
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />          <!-- an entry point... -->
            <category android:name="android.intent.category.LAUNCHER" />  <!-- ...shown in the app drawer
              ↪ -->
        </intent-filter>
    </activity>
</application>
```

> **Careful when copying.** In this guide, `<!-- ... -->` notes sometimes appear *inside* a tag, next to the attribute they explain. That is for teaching only: real XML does not allow comments inside a tag. When you type the real files, leave those notes out, or copy the files from the Appendix.

The comment in the real file, `<!-- No configChanges: ... -->`, records an important decision. It is explained in 2.10.

2.7 Resources, the `R` class, qualifiers and units

Resources are everything in `res/` that isn't code. Each kind has its own folder:

Folder	Contains	Referenced in XML as	In Kotlin as
`layout/`	Screen layouts	`@layout/view_note_row`	`R.layout.view_note_row`
`values/strings.xml`	Text	`@string/app_name`	`R.string.app_name`
`values/colors.xml`	Colours	`@color/note_500`	`R.color.note_500`
`values/sizes.xml`	Dimensions (`<dimen>`)	`@dimen/key_gap`	`R.dimen.key_gap`
`values/integers.xml`	Integers	`@integer/single_line`	`R.integer.single_line`
`values/styles.xml`, `themes.xml`	Styles and themes	`@style/Widget.VangtiChai.Key`	`R.style...`
`drawable/`	Shapes, vector images	`@drawable/bg_header`	`R.drawable.bg_header`
`mipmap-*/`	Launcher icons	`@mipmap/ic_launcher`	`R.mipmap.ic_launcher`
`animator/`	Property animations	`@animator/key_press`	`R.animator.key_press`
`xml/`	Misc XML (backup rules)	`@xml/backup_rules`	`R.xml.backup_rules`

The build generates a class called `R` with an integer ID for every resource. Your code refers to resources through these IDs, for example `getString(R.string.label_taka)`.

Inside `values/` the **file names don't matter**, only the tags inside. `sizes.xml` is a name the assignment asked for. Android Studio's default would be `dimens.xml`, and both work identically.

Qualifiers are the heart of "designing for different screens". A folder name can have suffixes that say when its files apply. When the app needs `activity_main.xml`, Android looks at the current device and picks the **best matching** folder:

Folder	Applies when	Used for
`layout/`	always (the default/fallback)	phone portrait
`layout-land/`	orientation is landscape	phone landscape
`layout-sw600dp/`	the screen's **s**mallest **w**idth is at least 600 dp	tablet portrait
`layout-sw600dp-land/`	smallest width at least 600 dp **and** landscape	tablet landscape

"Smallest width" is the shorter side of the screen in dp. It doesn't change when you rotate, so a tablet is a tablet in both orientations. A Pixel XL is 411 dp wide, so it uses `layout/`, or `layout-land/` when rotated. A Nexus 10 is 800 dp, so it uses `layout-sw600dp/` or `layout-sw600dp-land/`.

The same qualifiers work for `values/`. For example, `values-land/sizes.xml` overrides only the sizes that differ in landscape. Android merges files: a `<dimen>` missing from `values-land/` falls back to `values/`. A tablet in landscape looks in `values-sw600dp-land/`, then `values-sw600dp/`, then `values/`.

> There used to be a `values-night/` folder too: the `night` qualifier means "the system dark mode is on". VangtiChai is now dark-only, so it was deleted (see Part 4).

IN DETAIL **How Android picks between folders**

When the app asks for `activity_main.xml` or a `<dimen>`, Android first throws out every folder whose qualifiers contradict the device: a phone in portrait rules out `-land` and `-sw600dp`. If several folders are left, it goes through the qualifier types in a fixed order of importance and keeps only the folders that match the most important one. **Smallest width comes before orientation** in that order.

Screen	Layout file used	Size files consulted, most specific first
Phone, portrait (Pixel XL, 411 × 731 dp)	`layout/`	`values/`
Phone, landscape (731 × 411 dp)	`layout-land/`	`values-land/`, then `values/`
Tablet, portrait (Nexus 10, 800 × 1280 dp)	`layout-sw600dp/`	`values-sw600dp/`, then `values/`
Tablet, landscape (1280 × 800 dp)	`layout-sw600dp-land/`	`values-sw600dp-land/`, then `values-sw600dp/`, then `values/`

Strictly, `values-land/` is also a candidate for a tablet in landscape, ranked just above `values/`. It never wins in this project, because every value in `values-land/sizes.xml` is also defined in `values-sw600dp/sizes.xml`.

The rule is "smallest width of *at least* 600 dp", so a Nexus 7 (600 dp) counts as a tablet, while a Nexus 5 (360 dp) and a Pixel 7-sized phone (411 dp) are phones.

Units:

- **px**: real screen pixels. Never use these for layout; phones have very different pixel densities.
- **dp** (density-independent pixel): about 1/160 of an inch on any screen. A Pixel XL's screen density is 560 dpi, so on it 1 dp = 3.5 px (560 ÷ 160), and its 1440 px wide screen is about 411 dp wide. Use dp for sizes, margins and paddings.
- **sp** (scale-independent pixel): like dp, but also scales with the user's font-size setting. Use sp for text sizes.

> IN DETAIL **Why the phone screenshots are 822 px wide**
>
> Most screenshots in this PDF come from the renders described in Part 5.3, which drew the Pixel XL's 411 × 731 dp screen at xhdpi density: 2 px per dp, so 822 px across. The real phone draws 3.5 px per dp and would show the same screen 1440 px across. (The tablet renders used 1 px per dp, which is why the Nexus 10 screenshots are 800 px wide.) The screenshots in Part 5.2 come from the emulators instead: the real 1440 px Pixel XL screen and 1600 px Nexus 10 screen, shrunk to half size.
>
> The layout is the same at every density. It is written in dp, and only the number of pixels behind each dp changes. That is the whole point of dp.

`sizes.xml` uses `dp` for spacing and `sp` for text. Some entries are plain numbers, like `<item name="half" format="float" type="dimen">0.5</item>`. These are proportions, used for percentages and letter spacing.

2.8 Views, ViewGroups and XML layouts

A **View** is anything on screen: `TextView` (text), `Button`, `View` (a plain box). A **ViewGroup** is a View that contains and positions other Views: `ConstraintLayout`, `LinearLayout`, `FrameLayout`, `MaterialCardView`. A screen is a **tree** of views.

A layout XML file describes that tree:

```
<TextView                                        <!-- the class to create -->
    android:id="@+id/amount_text"                <!-- "@+id/" CREATES an id named amount_text -->
    style="@style/Widget.VangtiChai.Amount"      <!-- apply a bundle of attributes from styles.xml -->
    android:layout_width="@dimen/match_constraint" <!-- 0dp: "decided by constraints" (see 2.9) -->
    android:layout_height="@dimen/amount_height"  <!-- height from sizes.xml -->
    android:text="@string/label_taka"            <!-- initial text from strings.xml -->
    app:layout_constraintTop_toBottomOf="@id/toolbar"  <!-- "@id/" REFERS to an existing id -->
    tools:text="Taka: 688" />                    <!-- shown ONLY in Android Studio's preview -->
```

- **Namespaces.** `android:` is for attributes built into Android. `app:` is for attributes defined by libraries (ConstraintLayout, Material). `tools:` is for design-time hints that never reach the app, such as preview text, `tools:visibility` and `tools:context`. They are declared once on the root element (`xmlns:android=...`, `xmlns:app=...`, `xmlns:tools=...`).
- `layout_width` / `layout_height` are required on every view. The values are a size (`@dimen/...`), `wrap_content` (as big as the content), `match_parent` (as big as the parent), or `0dp`, which inside a ConstraintLayout means "match constraints" (see 2.9). VangtiChai's layouts write that `0dp` as `@dimen/match_constraint`.
- **References.** `@string/...`, `@dimen/...` and `@color/...` point to resources. `?attr/colorPrimary` points to a **theme attribute**: "whatever the current theme says the primary colour is".
- `<include layout="@layout/view_note_row" android:id="@+id/note_500" .../>` inserts another layout file as a child. VangtiChai includes the same row layout 8 times, giving each copy a different id.

2.9 ConstraintLayout in depth

`ConstraintLayout` positions each child with **constraints**: rules that attach one of the child's edges to an edge of the parent or of a sibling. It lets you build complex, flat (not deeply nested) layouts that adapt to any screen size, which is why the assignment focuses on it.

Constraints. Each child needs at least one horizontal and one vertical constraint:

```
app:layout_constraintStart_toStartOf="parent"     <!-- my left edge sticks to the parent's left edge -->
app:layout_constraintEnd_toStartOf="@id/split"    <!-- my right edge sticks to the guideline "split" -->
app:layout_constraintTop_toBottomOf="@id/header"  <!-- my top sticks to the header's bottom -->
app:layout_constraintBottom_toBottomOf="parent"   <!-- my bottom sticks to the parent's bottom -->
```

"Start/End" mean left/right in left-to-right languages. VangtiChai forces left-to-right with `android:layoutDirection="ltr"` on the root, so start is always left.

`0dp` **= match constraints.** With constraints on both sides and a size of `0dp`, the view **stretches to fill the space between its constraints**. The change card uses it for both width and height, so it fills from the screen's left edge to the `split` guideline, and from below the header to the screen's bottom.

`@dimen/match_constraint`**.** VangtiChai never writes `0dp` in a layout. `values/sizes.xml` defines `<dimen name="match_constraint">0dp</dimen>`, and the layouts use `@dimen/match_constraint` instead, so even this zero is not hardcoded, as the brief asks. ConstraintLayout only sees the final value, 0, so it behaves exactly as a written `0dp` would. This was checked on the emulators: every view has the same position and size as with a written `0dp` (Part 5.2).

Margins (`android:layout_marginTop="@dimen/section_gap"`) add space between the edge and what it is attached to.

Guidelines are invisible lines you can attach to:

```
<androidx.constraintlayout.widget.Guideline
    android:id="@+id/split"
    android:orientation="vertical"                                    <!-- a vertical line... -->
    app:layout_constraintGuide_percent="@dimen/notes_width_percent" /> <!-- ...at 40% of the width -->
```

The change table ends at `split` and the keypad starts there. Changing one number in `sizes.xml` re-proportions the whole screen.

Chains are views linked to each other in both directions:

```
parent.top ← key_1 ⇄ key_4 ⇄ key_7 ⇄ key_0 → parent.bottom
```

Each key's top is constrained to the previous key's bottom, and its bottom to the next key's top. When all are `0dp` in that direction, the default **spread** chain shares the space **equally**. That's how the keypad gets four equal rows on any screen. A **packed** chain (`app:layout_constraintVertical_chainStyle="packed"`) instead groups the views together in the middle. The header uses one to keep the amount and caption together and centred.

IN DETAIL **The numbers on a Pixel XL in portrait**

Here is what the guideline, margins and chains work out to on a screen 411 dp wide:

What	Worked out as	Result
`split` guideline	0.4 × 411	164.4 dp from the left edge
Change card width	164.4 − 16 (`screen_margin`)	148.4 dp
Keypad's left edge	164.4 + 12 (`column_gap`)	176.4 dp
Keypad width	411 − 16 (`screen_margin`) − 176.4	218.6 dp
One key column	(218.6 − 2 × 10 (`key_gap`)) ÷ 3	about 66.2 dp

The rows are shared out the same way: each of the four key rows is (keypad height − 3 × 10 dp) ÷ 4. You can check this on Figure 1: its screenshots use 2 px per dp, and each key there is about 132 px (66 dp) wide.

Percent sizes and limits:

- `app:layout_constraintHeight_default="percent"` together with `app:layout_constraintHeight_percent="@dimen/header_height_percent"` makes the header 32% of the screen height.
- `app:layout_constraintHeight_min="wrap"` means "but never smaller than your content".

- `app:layout_constraintHeight_max="@dimen/content_max_height"` caps how tall the card may grow on very tall screens.
- **Bias** (`app:layout_constraintVertical_bias="0.5"`) decides where a capped view sits in the leftover space: 0 = top, 0.5 = middle, 1 = bottom.

Baseline (`app:layout_constraintBaseline_toBaselineOf="@id/notes_title"`) lines up the text baselines of two TextViews. The tablet's "COUNT" heading uses it.

2.10 Activities, the lifecycle and saving state

An **Activity** is one screen. Android calls its **lifecycle methods** as things happen:

```
onCreate → onStart → onResume      (now visible and interactive)
              ... user uses the app ...
onPause → onStop → onDestroy       (leaving / being destroyed)
```

Configuration changes. When the device **rotates** (or changes language, dark mode, window size, and so on), Android by default **destroys and recreates** the activity. That is exactly how it loads `layout-land/activity_main.xml` after rotation: the new activity inflates the layout that matches the new configuration. But it also means every field in the old activity object is lost.

Saving state. Before destroying, Android calls `onSaveInstanceState(outState: Bundle)`. A `Bundle` is a key-value box, and you put your important state in it. The new activity receives the same Bundle as `savedInstanceState` in `onCreate`:

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putString(KEY_AMOUNT, amount)        // "688"
    outState.putLongArray(KEY_CHANGE, change)     // [1, 1, 1, 1, 1, 1, 1, 1]
}

override fun onCreate(savedInstanceState: Bundle?) {
    ...
    if (savedInstanceState != null) {             // null on a fresh start, non-null after rotation
        amount = savedInstanceState.getString(KEY_AMOUNT).orEmpty()
        change = savedInstanceState.getLongArray(KEY_CHANGE) ?: ChangeCalculator.breakdown(...)
    }
}
```

The saved Bundle also survives if Android kills the app in the background to free memory, and the user comes back later ("process death").

Why not `android:configChanges="orientation|screenSize"`**?** That manifest setting tells Android *not* to recreate the activity on rotation. Android would then **not** load the landscape layout. You would have to swap layouts by hand, and that defeats the assignment's lesson. The manifest has a comment saying so.

2.11 View Binding

To change a view from code, you need a reference to it. The old way was `findViewById<TextView>(R.id.amount_text)`. It is verbose and unsafe: the wrong type or a missing id crashes at runtime.

With `viewBinding = true`, the build generates a **binding class per layout file**:

Layout file	Generated class	A view with id `key_backspace` becomes
`activity_main.xml`	`ActivityMainBinding`	`binding.keyBackspace` (snake_case becomes camelCase)
`view_note_row.xml`	`ViewNoteRowBinding`	`row.noteCount`

```
binding = ActivityMainBinding.inflate(layoutInflater)   // create all views from the XML
setContentView(binding.root)                            // show them; root = the top ConstraintLayout
binding.keyClear.setOnClickListener { ... }             // typed, non-null access to the CLEAR key
```

- An `<include>` with an id becomes a field of the included layout's binding type: `binding.note500` is a `ViewNoteRowBinding`, so you can write `binding.note500.noteCount`.
- With four variants of `activity_main.xml`, View Binding generates **one** class covering all of them. An id that exists in only some variants becomes nullable. Examples are `notes_count_heading` (tablet portrait only) and `notes_columns` (landscape only). The code never touches those, so it doesn't matter.

2.12 Themes, styles and Material 3

A **style** is a named bundle of view attributes. Instead of repeating 12 attributes on all 11 keys, each key says `style="@style/Widget.VangtiChai.Key"`. Styles can **inherit**: `Widget.VangtiChai.Key.Clear` (the dot means "child of `Widget.VangtiChai.Key`") overrides only the text size and colours.

A **theme** is a style applied to the whole app (`android:theme` in the manifest). It sets **theme attributes**, which views look up with `?attr/...`. For example, `colorPrimary` is the main brand colour, `colorOnSurface` the text colour on surfaces, and `colorSecondaryContainer` the key background.

Material 3 (also called Material You) is Google's design system. The `com.google.android.material` library provides components that follow it (`MaterialButton`, `MaterialCardView`, `MaterialToolbar`) and themes (`Theme.Material3.*`). Material 3 colours come in **roles**: `primary` / `onPrimary` / `primaryContainer` / `onPrimaryContainer`, and the same for secondary, tertiary, error, surface and others. "on" means "content drawn on top of". `themes.xml` maps every role to a colour from `colors.xml`.

VangtiChai's theme extends `Theme.Material3.Dark.NoActionBar`:

- **Dark** means it is always dark.
- **NoActionBar** means "don't add a default title bar". The app draws its own header with a `MaterialToolbar` inside it.

2.13 Kotlin crash course, with examples from this project

Kotlin is a modern, statically typed language that runs on the JVM and Android. Here is every piece of syntax used in VangtiChai.

Files, packages and imports

```
package com.example.vangtichai              // this file's namespace; matches the folder path
import android.os.Bundle                    // use the class Bundle from package android.os
import androidx.core.text.bold              // you can import single functions too
```

Variables: `val` vs `var`

```
val notes = 5           // val = read-only (can't be reassigned). Prefer val.
var amount = ""         // var = can change. The type (String) is inferred from "".
private var change = LongArray(8)   // private = visible only inside this class
```

Basic types

Type	Example	Notes
`Int`	`9`, `0x29`	32-bit whole number. `0x29` is hexadecimal (= 41).
`Long`	`70L`, `999_999_999`	64-bit whole number. The `L` suffix makes a literal a Long. `_` separates digits for readability.

Type	Example	Notes
`Float`	`0.62f`, `1f`	Decimal number. The `f` suffix marks a Float.
`Boolean`	`true`	
`String`	`"688"`	Text.
`Char`	`' '`	One character, in single quotes.

Kotlin never silently converts between number types. `count.toInt()` converts a Long to an Int, and `width / 2f` divides by a Float to get a Float.

Null safety. A type with `?` *may* be `null`, and the compiler forces you to handle it:

```
fun appendDigit(amount: String, digit: Int): String?   // may return null ("no, limit reached")

val updated = ChangeCalculator.appendDigit(amount, digit)
if (updated == null) { rejectDigit(); return }         // after this check, Kotlin knows updated is not null
setAmount(updated)

savedInstanceState.getString(KEY_AMOUNT).orEmpty()     // null becomes ""
savedInstanceState.getLongArray(KEY_CHANGE) ?: fallback  // ?: "Elvis": use the right side if the left is
  ↪ null
rowAnimators[index]?.cancel()                          // ?. call cancel() only if it's not null
two!!                                                  // !! "I promise it's not null" (crashes if it is;
  ↪ tests only)
```

Functions

```
private fun onDigit(digit: Int) { ... }                // parameter "digit" of type Int, returns nothing
  ↪ (Unit)

fun valueOf(amount: String): Long = amount.toLongOrNull() ?: 0L   // "= expression" body; returns Long

private fun onClear() = setAmount("")                  // one-liner function

private fun View.pop(scale: Float, pivotAtEnd: Boolean = false)   // default argument value
row.noteCount.pop(COUNT_POP_SCALE, pivotAtEnd = true)  // named argument when calling
enableEdgeToEdge(statusBarStyle = ..., navigationBarStyle = ...)  // named args make calls readable
```

Classes, inheritance and `override`

```
class MainActivity : AppCompatActivity() {   // MainActivity *extends* AppCompatActivity; () calls its
  ↪ constructor
    override fun onCreate(savedInstanceState: Bundle?) {   // replace the parent's version of a method...
        super.onCreate(savedInstanceState)                 // ...but still run the parent's code
    }
}
```

`object`, `companion object` and `const val`

```
object ChangeCalculator {            // a singleton: exactly one instance, created automatically
    val NOTES = intArrayOf(500, 100, 50, 20, 10, 5, 2, 1)
    const val MAX_DIGITS = 9         // compile-time constant (only for primitives and String)
    fun breakdown(amount: Long): LongArray { ... }
}
ChangeCalculator.breakdown(688)      // call it without creating anything

class MainActivity ... {
    private companion object {       // "static" members of the class, shared by all instances
        const val KEY_AMOUNT = "amount"
        const val ROW_FADE_DURATION = 220L
    }
}
```

`lateinit` and `by lazy`: initialise later

```
private lateinit var binding: ActivityMainBinding   // "I'll assign this before using it" (in onCreate)
                                                    // using it earlier throws an error

private val noteColors by lazy { NOTE_COLORS.map { ContextCompat.getColor(this, it) } }
// computed the FIRST time it's used, then remembered. Useful because resources and themes
// aren't ready when the Activity object is constructed, only from onCreate on.
```

Lambdas (function values) and trailing lambdas

```
key.setOnClickListener { view -> onDigit(digit) }     // a lambda: { parameters -> body }
key.setOnClickListener { it.performHapticFeedback(...) }  // single parameter? it's called "it"
noteRows.forEachIndexed { i, row -> ... }             // two parameters
ViewCompat.setOnApplyWindowInsetsListener(binding.root) { _, insets -> ... }  // _ = unused parameter
```

When a function's last parameter is a function, you can write the lambda *after* the parentheses (a "trailing lambda"). If it is the only parameter, the parentheses can go: `setOnClickListener { ... }`. The last expression in a lambda is its result. That's why the long-click listener ends with `true` ("I handled it").

SAM conversion and anonymous objects. Java interfaces with a **S**ingle **A**bstract **M**ethod (like `View.OnClickListener` or `Runnable`) accept a Kotlin lambda directly:

```
private val restoreCaption = Runnable { binding.captionText.text = caption() }   // lambda becomes a
  ↪ Runnable

private val repeatBackspace = object : Runnable {   // an anonymous object implementing Runnable...
    override fun run() {
        ...
        key.postDelayed(this, BACKSPACE_REPEAT_INTERVAL)   // ...used because it needs "this" to re-post
          ↪ itself
    }
}
```

Extension functions add functions to existing classes without changing them:

```
private fun View.flashPressed() {     // now every View has .flashPressed() (inside MainActivity)
    isPressed = true                  // "this" is the View it's called on
    postDelayed({ isPressed = false }, KEY_FLASH_DURATION)
}
key.flashPressed()
```

Android KTX provides many of these: `view.isVisible`, `view.isInvisible`, `updatePadding(...)` and `updateLayoutParams { }`.

`when`: a powerful switch

```
val key = when (keyCode) {
    in KeyEvent.KEYCODE_0..KeyEvent.KEYCODE_9 -> digitKeys[keyCode - KeyEvent.KEYCODE_0]  // a range check
    KeyEvent.KEYCODE_ESCAPE, KeyEvent.KEYCODE_CLEAR, KeyEvent.KEYCODE_C -> binding.keyClear // several
      ↪ values
    KeyEvent.KEYCODE_DEL -> binding.keyBackspace
    else -> return super.onKeyDown(keyCode, event)   // any other key: let Android handle it
}
```

`when` is an **expression**: its result is assigned to `key`. `a..b` is an inclusive range, and `x in a..b` checks membership.

`if` as an expression

```
val alpha = if (show) 1f else 0f
```

String templates. `"amount $amount"` inserts a variable, and `"${a.b}"` inserts an expression.

Collections and arrays

```
listOf(key0, key1, key2)                        // read-only List
NOTE_COLORS.map { getColor(it) }                // transform each element into a new List
noteRows.forEachIndexed { i, row -> ... }       // loop with index
change.sum()                                    // add all elements
intArrayOf(500, 100)                            // primitive Int array
LongArray(8) { i -> remaining / NOTES[i] }      // array of 8 Longs, each computed by the lambda
FloatArray(8)                                   // 8 Floats, all 0f
arrayOfNulls<ValueAnimator>(8)                  // Array<ValueAnimator?> filled with null
"1".repeat(9)                                   // "111111111"
amount.dropLast(1)                              // "688" becomes "68"
```

Scope functions run a block with an object in scope:

```
with(binding) { listOf(note500, note100, ...) }  // inside, "binding." is implied
ValueAnimator.ofFloat(0f, 1f).apply {            // configure the object, then return it
    duration = ROW_FADE_DURATION                 // = this.duration = ...
    addUpdateListener { ... }
    start()
}
```

Properties instead of getters and setters. Kotlin turns Java's `view.getAlpha()` / `view.setAlpha(0f)` into `view.alpha` / `view.alpha = 0f`, and `textView.setText(x)` into `textView.text = x`.

`require` checks an argument and throws `IllegalArgumentException` with your message if it is false:

```
require(amount >= 0) { "amount must not be negative: $amount" }
```

Comments and KDoc. `// line comment` and `/* block */`. `/** ... */` is **KDoc** documentation. `[name]` inside KDoc links to a symbol, as in "/** Removes the rightmost digit of the typed [amount] */".

Annotations in tests:

- `@Test` marks a test method.
- `@RunWith(AndroidJUnit4::class)` picks the test runner (`::class` gets a class reference).
- `@get:Rule` attaches the annotation to the property's *getter*, which JUnit requires.
- `@Test(expected = IllegalArgumentException::class)` passes if that exception is thrown.

PART 3

Building VangtiChai from scratch, step by step

Follow these steps in order. Each step says **what** to create, **why**, and explains the code. The complete final version of every file is in the Appendix. When a step shows only the important part of a file, copy the full file from there.

Step 1: Install Android Studio and create the project

1. Install **Android Studio** from developer.android.com/studio. On first launch it downloads the Android SDK.
2. **File, then New, then New Project.** Choose **Empty Views Activity**.
   - Do **not** choose "Empty Activity". That one uses Jetpack Compose, and this project uses XML layouts (Views).
3. Fill in:
   - **Name:** `VangtiChai`
   - **Package name:** `com.example.vangtichai`
   - **Language:** Kotlin
   - **Minimum SDK:** API 21 ("Android 5.0 (Lollipop)"), the level the assignment suggests
   - **Build configuration language:** Kotlin DSL (`build.gradle.kts`)
4. Click **Finish** and wait for the **Gradle sync** (the progress bar at the bottom) to finish.

The template gives you:

- `MainActivity.kt` with a `setContentView(R.layout.activity_main)`
- a default `activity_main.xml`
- `themes.xml`, `colors.xml` and `strings.xml`
- launcher icons
- an example unit test and an example instrumented test
- all the Gradle files described in Part 2

You will replace most of these. Delete the example tests, and any `values-night/themes.xml` the template created, because this app is dark-only.

5. Create two emulators: **Tools, then Device Manager, then Create Virtual Device**. Pick the **Pixel XL** hardware profile, then the **Nexus 10** (under "Tablet"). Any recent system image works. These are the two devices the assignment requires. To check the minimum version too, make a third with the Pixel XL profile and an **API 21** system image.

Step 2: Configure Gradle

1. In `gradle/libs.versions.toml`, make sure these libraries exist. Add `activity-ktx` and `material` if the template didn't, and set `coreKtx` and `appcompat` to the versions below:

```
[versions]
coreKtx = "1.17.0"
appcompat = "1.7.1"
activityKtx = "1.11.0"
material = "1.13.0"
constraintlayout = "2.2.2"
# (plus the template's agp, junit, junitVersion, espressoCore)

[libraries]
androidx-activity-ktx = { group = "androidx.activity", name = "activity-ktx", version.ref =
  ↪ "activityKtx" }
material = { group = "com.google.android.material", name = "material", version.ref = "material" }
androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout",
  ↪ version.ref = "constraintlayout" }
```

2. In `app/build.gradle.kts`, add `buildFeatures { viewBinding = true }` inside `android { }`, and the `implementation(...)` lines shown in Part 2.4.
3. Click **Sync Now**.

Why these versions? They are the newest releases that still support API 21. The next releases of all four need API 23, and with `minSdk = 21` the build would stop with an error like "minSdkVersion 21 cannot be smaller than version 23 declared in library". Android Studio will suggest the newer versions anyway. Don't take them.

Why these libraries?

- `material` provides `MaterialButton`, `MaterialCardView`, `MaterialToolbar` and the Material 3 themes.
- `constraintlayout` is the layout the assignment is about.
- `activity-ktx` provides `enableEdgeToEdge()`.
- `core-ktx` provides Kotlin helpers like `buildSpannedString` and `isVisible`.
- `appcompat` provides `AppCompatActivity`, which is needed for the Material themes.

Step 3: The manifest

Use the manifest from Part 2.6. The template's version is almost identical. Add one comment above `<activity>` so nobody "fixes" rotation later:

```
<!-- No configChanges: rotating recreates the activity so the landscape layout is loaded. -->
```

Step 4: Colours (`res/values/colors.xml`)

A colour is written `#RRGGBB` (opaque) or `#AARRGGBB`, where **AA** is the alpha (opacity: `00` = invisible, `FF` = solid). For example, `#3DFFFFFF` is white at about 24% opacity.

The file has five groups:

1. **Material 3 colour roles** (`md_primary`, `md_on_surface`, `md_secondary_container` and so on): a full dark palette built around a Taka green. For example, `md_primary` = `#6ADBAD` (mint green) and `md_background` = `#0E1411` (near-black green).
2. **Header colours:**
   - `header_gradient_start` `#0B6E4D` and `header_gradient_end` `#06301F`, the header's diagonal gradient
   - `header_glow`, a faint white glow

  - `header_content` (white text) and `header_content_secondary` (85% white, for "Taka:" and the caption)
  - `header_ripple` (the backspace key's touch ripple)
3. **Change table:** `card_surface` `#151C18` (the card) and `note_row_idle` `#1C2420` (rows whose count is 0).
4. **One colour per note:** `note_500` purple, `note_100` blue, `note_50` orange, `note_20` teal, `note_10` pink, `note_5` amber, `note_2` green, `note_1` grey. They are pastel shades so they stay readable on the dark card.
5. **Launcher icon colours** (`icon_*`).

> IN DETAIL **The palette at a glance**
>
> The table below shows the colours that matter most, as they look in the app. The hex values come straight from `colors.xml`; the last column says where each one shows up. Appendix A.5 maps out the whole file.

Colour	Hex	Where it shows up
`note_500`	#D4A9F2	500 Taka row: label, colour tab and lit background
`note_100`	#9DC4FF	100 Taka row
`note_50`	#FFB77C	50 Taka row
`note_20`	#6FD9C7	20 Taka row
`note_10`	#FF8DB7	10 Taka row
`note_5`	#F2C150	5 Taka row
`note_2`	#A6D77A	2 Taka row
`note_1`	#B5C3BD	1 Taka row
`header_gradient_start`	#0B6E4D	Header gradient, top-left corner
`header_gradient_end`	#06301F	Header gradient, bottom-right corner
`md_background`	#0E1411	Screen background
`card_surface`	#151C18	Change table card
`note_row_idle`	#1C2420	Rows whose count is 0
`md_secondary_container`	#2A3B34	Digit keys (through `colorSecondaryContainer`)
`md_on_secondary_container`	#D3ECDF	The digits on the keys
`md_tertiary_container`	#5C1A24	CLEAR key (through `colorTertiaryContainer`)
`md_on_tertiary_container`	#FFDADB	The CLEAR text
`md_primary`	#6ADBAD	"CHANGE" title and the keys' ripple
`md_outline_variant`	#2E3833	The card's 1 dp border
`md_on_surface`	#DEE4DF	Counts in lit rows (`strongTextColor`)
`md_on_surface_variant`	#BFC9C2	Faded labels and counts, drawn at 60% (`mutedTextColor`)

Why put colours in a resource file at all? One place to change the look, and the assignment forbids hardcoded values in layouts.

Step 5: Text (`res/values/strings.xml`)

```
<string name="app_name">VangtiChai</string>
<string name="label_taka">Taka:</string>
<string name="hint_enter_amount">Enter an amount</string>
<string name="note_label">%1$d:</string>                    <!-- "%1$d" = 1st argument, as an integer -->
<string name="key_1" translatable="false">1</string>       <!-- digits never need translating -->
<string name="key_clear">CLEAR</string>
<string name="cd_amount">%1$s Taka</string>                <!-- "%1$s" = 1st argument, as a string -->
<string name="cd_key_backspace">Delete the last digit</string>

<plurals name="caption_note_count">                        <!-- grammar-correct counts -->
    <item quantity="one">%1$s note</item>                  <!-- "1 note" -->
    <item quantity="other">%1$s notes</item>               <!-- "8 notes" -->
</plurals>
<plurals name="cd_note_row">
    <item quantity="one">%2$s note of %1$d Taka</item>     <!-- arguments can be reordered: %2$ then %1$ -->
    <item quantity="other">%2$s notes of %1$d Taka</item>
</plurals>
```

- `getString(R.string.note_label, 500)` gives `"500:"`.
- `resources.getQuantityString(R.plurals.caption_note_count, 8, "8")` gives `"8 notes"`. The second argument picks the plural form, and the rest fill the placeholders.
- Strings prefixed `cd_` are **content descriptions**: what a screen reader (TalkBack) says.

Why? All text in one file means the app can be translated (for example to Bangla) by adding a `values-bn/strings.xml`, without touching code.

Step 6: Sizes (`res/values*/sizes.xml`)

Create `res/values/sizes.xml` (phone portrait, the defaults) and three override files: `values-land/`, `values-sw600dp/` and `values-sw600dp-land/`.

```
<dimen name="screen_margin">16dp</dimen>          <!-- a size -->
<dimen name="key_text">28sp</dimen>               <!-- a text size -->
<item name="notes_width_percent" format="float" type="dimen">0.4</item>   <!-- a plain number -->
```

The base file is grouped by area:

- **Match constraints:** `match_constraint` (`0dp`), which the layouts use for every width or height that stretches between constraints (Part 2.9)
- **Spacing:** `screen_margin`, `section_gap`, `column_gap`
- **Header:** corner radius, paddings, glow geometry, toolbar height and text sizes, `amount_height`, the amount's auto-size range `amount_text_min`/`amount_text_max`, `amount_side_margin`, caption size, `shake_distance`, `entrance_rise`, and the backspace key's `backspace_size`, `backspace_icon_size` and `backspace_margin_end`
- **Change table:** card radius, stroke, paddings, row gap, row corner radius, colour tab size, label text size, the count's auto-size range
- **Keypad:** `key_gap`, `key_corner_radius`, `key_text`, `key_clear_text`
- **Proportions:** `half`, `notes_width_percent`, `header_height_percent`, `content_vertical_bias`, `note_accent_height_percent`, letter spacings

The override files list **only what changes**:

Value	Phone portrait (`values`)	Phone landscape (`values-land`)	Tablet portrait (`values-sw600dp`)	Tablet landscape (`values-sw600dp-land`)
`notes_width_percent`	0.4	0.42	0.4	0.47

Value	Phone portrait (values)	Phone landscape (values-land)	Tablet portrait (values-sw600dp)	Tablet landscape (values-sw600dp-land)
amount_height	64dp	44dp	112dp	88dp
amount_text_max	48sp	36sp	88sp	68sp
key_text / key_clear_text	28sp / 20sp	24sp / 18sp	46sp / 30sp	40sp / 28sp
note_label_text	18sp	18sp	30sp	28sp
note_count_text_max	22sp	20sp	32sp	30sp
key_gap	10dp	8dp	16dp	(16dp)
backspace_size / icon	48dp / 28dp	(48 / 28)	72dp / 44dp	(72 / 44)
amount_side_margin	60dp	(60dp)	96dp	(96dp)
entrance_rise	24dp	(24dp)	40dp	(40dp)
content_max_height	500dp	n/a	780dp	n/a

Values in brackets are inherited from a more general file.

> IN DETAIL **All 56 sizes**
>
> Appendix A.5 has the full table: every dimension in `values/sizes.xml`, as each of the four configurations sees it, generated from the four files. It also shows one value the table above leaves out: tablets set `header_height_percent` to 0.3, against 0.32 on phones. Only the portrait layouts use it; in landscape the header wraps its content.

How the numbers were chosen: by rendering the app at each target size, measuring, and adjusting (see Part 5). Two examples:

- `notes_width_percent` went from 0.36 to 0.4 in portrait so that a 9-digit amount's biggest count (1,999,999) still fits on a 360 dp phone.
- `amount_side_margin` = `backspace_margin_end` + `backspace_size` + a little gap. That leaves room for the backspace key, and the same margin on the left keeps the amount centred.

Step 7: Integers and the theme

`res/values/integers.xml`:

```
<integer name="single_line">1</integer>          <!-- used for maxLines="1" in styles -->
<integer name="key_press_duration">90</integer>  <!-- ms; how fast keys shrink when pressed -->
```

`res/values/themes.xml`:

```
<!-- Always dark, whatever the system setting, so there is no light theme to switch to -->
<style name="Base.Theme.VangtiChai" parent="Theme.Material3.Dark.NoActionBar">
    <item name="colorPrimary">@color/md_primary</item>
    <item name="colorOnPrimary">@color/md_on_primary</item>
    <!-- ... one <item> per Material 3 colour role (about 30 in total) ... -->
    <item name="android:colorBackground">@color/md_background</item>
</style>
<style name="Theme.VangtiChai" parent="Base.Theme.VangtiChai" />
```

Why two styles (`Base.Theme...` and `Theme...`)? It's the template's convention. You can later add version-specific tweaks to `Theme.VangtiChai` (for example in `values-v31/themes.xml`) without copying the whole base.

Step 8: Widget styles (`res/values/styles.xml`)

Each style is a reusable set of attributes:

Style	Applied to	Key attributes and why
`Widget.VangtiChai.Toolbar`	`MaterialToolbar` in the header	Transparent background (the header gradient shows through); white title; title starts at the header padding.
`Widget.VangtiChai.Amount`	`amount_text`	Centred, single line. `autoSizeTextType=uniform` with min/max sizes, so long amounts shrink to fit instead of being cut off. `fontFeatureSettings="tnum"` gives *tabular numbers* (every digit the same width, so the number doesn't wobble while you type). `accessibilityLiveRegion="polite"` makes TalkBack announce changes.
`Widget.VangtiChai.Backspace`	backspace key	Extends `Widget.Material3.Button.IconButton`. Icon only, no text. Insets and min sizes set to 0 so the button is exactly `backspace_size`. White icon tint, white ripple, a circle shape, and the same press animation as the keys.
`Widget.VangtiChai.Caption`	`caption_text`	Smaller, 85% white, single line with "…" if too long.
`Widget.VangtiChai.Card`	the change table card	An outlined Material card with `card_surface` background, rounded corners, and a 1 dp `colorOutlineVariant` border.
`Widget.VangtiChai.NotesTitle`	"CHANGE" / "COUNT"	Primary colour, all caps, wide letter spacing.
`TextAppearance.VangtiChai.NoteLabel`	"500:" labels	Medium weight, tabular numbers.
`Widget.VangtiChai.NoteCount`	the counts	Right-aligned, auto-sizing between `note_count_text_min` and `note_count_text_max`, so even "1999999" fits.
`Widget.VangtiChai.Key`	digit keys	Extends `Widget.Material3.Button.TonalButton`. Insets/min sizes/padding set to 0 so keys fill their constraint box exactly. `colorSecondaryContainer` background, rounded corners, `stateListAnimator=@animator/key_press`.
`Widget.VangtiChai.Key.Clear`	CLEAR	Inherits `Key`, then uses the *tertiary* (red) colours and wider letters.
`ShapeAppearance.VangtiChai.Key` / `.Circle`	key corners / backspace	`cornerSize` = `key_corner_radius` / `50%` (a circle).

> **Why do MaterialButtons need `insetTop=0`, `minHeight=0` and so on?** By default a MaterialButton adds invisible space above and below itself (for touch-target size) and has a minimum size. In a grid where ConstraintLayout decides the exact size, these defaults would make keys look smaller than their slots and break the alignment.

Step 9: Drawables

`drawable-v23/bg_header.xml`: a `layer-list` (several shapes stacked):

1. A rectangle with a **linear gradient** at `angle="315"` (from top-left to bottom-right), from `header_gradient_start` to `header_gradient_end`, with rounded *bottom* corners only.
2. Two **radial-gradient ovals** (soft glows) at the top-left and top-right corners. Each is `header_glow_size` across (the items' `android:width` and `android:height`) and pinned to its corner with `android:gravity`. Negative offsets (`header_glow_offset` = -90 dp) push them partly off-screen so only a soft edge shows.

Layer-list items only support `width`, `height` and `gravity` from API 23 on. On Android 5.0 the glows ignored them and spread into a pale blob across the middle of the header. So this file is in `drawable-v23/`, which Android only

uses from API 23 on, and `drawable/bg_header.xml` is the fallback for Android 5.0 and 5.1: just the gradient rectangle, without glows.

`drawable/bg_note_row.xml` and `bg_note_accent.xml`: plain rounded rectangles. Their white fill is only a placeholder. Code gives each row its colour with `GradientDrawable.setColor()` (16.10), so one drawable file serves all 8 rows and every colour.

`drawable/ic_backspace.xml`: a **vector drawable**, an image described by drawing commands, so it is sharp at any size. The `pathData` is Google's Material "backspace" (outlined) icon:

- `M`/`L`/`H`/`V` = move to, line to, horizontal line, vertical line
- `C` = curve
- `z` = close the shape

`android:autoMirrored="true"` would flip it in right-to-left languages. The key's `iconTint` colours it white.

Launcher icon (adaptive icon). On Android 8+ (API 26+), `mipmap-anydpi-v26/ic_launcher.xml` combines three layers:

- `ic_launcher_background.xml`: a green diagonal gradient filling the 108 × 108 dp canvas.
- `ic_launcher_foreground.xml`: two banknotes and a gold coin. Everything sits inside the central **66 dp "safe zone"**, because launchers crop icons into circles, squircles or teardrops.
- `ic_launcher_monochrome.xml`: a single-colour note. Android 13+ uses it for **themed icons**, recoloured to match the wallpaper.

Android 5.0 to 7.1 (API 21 to 25) don't support adaptive icons, so they use the `mipmap-*dpi/ic_launcher.webp` bitmaps instead. Those bitmaps are still the project template's default icon (the green Android robot), so on those versions the launcher shows the robot, not the VangtiChai design.



Figure 2. *The adaptive launcher icon, cropped to a circle and to a rounded square as different launchers do, and (right) the monochrome version that Android 13+ recolours to match the wallpaper.*

Step 10: The key-press animation (`res/animator/key_press.xml`)

A **StateListAnimator** runs animations when a view's state changes:

```
<selector>
    <item android:state_pressed="true">        <!-- while pressed: shrink to 93% -->
        <set>
            <objectAnimator android:propertyName="scaleX" android:valueTo="0.93"
                android:duration="@integer/key_press_duration" android:valueType="floatType" />
            <objectAnimator android:propertyName="scaleY" android:valueTo="0.93" ... />
        </set>
    </item>
    <item>                                      <!-- otherwise: spring back to 100% -->
        <set>
            <objectAnimator android:propertyName="scaleX" android:valueTo="1"
                android:interpolator="@android:anim/overshoot_interpolator" ... />
            ...
        </set>
    </item>
</selector>
```

The **overshoot interpolator** goes slightly past 100% and settles back, which feels springy. Every key (and the backspace key) uses it through its style.

Step 11: The logic: `ChangeCalculator.kt`

Create `app/src/main/java/com/example/vangtichai/ChangeCalculator.kt`. It has **no Android imports**, so it can be unit tested on your computer in milliseconds.

```
package com.example.vangtichai

/**
 * Pure (Android-free) logic behind VangtiChai, kept separate from the UI so it can be unit tested.
 */
object ChangeCalculator {

    /** Taka notes the change is given in, largest first. */
    val NOTES = intArrayOf(500, 100, 50, 20, 10, 5, 2, 1)

    /**
     * Longest amount (in digits) the keypad accepts: up to 999,999,999 Taka. The largest count
     * (1,999,999 five-hundreds) still shows in full on small phones; the text shrinks to fit.
     */
    const val MAX_DIGITS = 9

    /**
     * Returns how many of each note in [NOTES] make up [amount], using as few notes as possible.
     * The Taka note system is canonical, so taking the largest note that fits each time is optimal.
     */
    fun breakdown(amount: Long): LongArray {
        require(amount >= 0) { "amount must not be negative: $amount" }
        var remaining = amount
        return LongArray(NOTES.size) { i ->
            val count = remaining / NOTES[i]     // how many of this note fit (integer division)
            remaining %= NOTES[i]                // what's left over (remainder)
            count                                // the lambda's result becomes element i
        }
    }

    /**
     * Adds [digit] to the right of the typed [amount] ("23" + 4 -> "234").
     * A lone leading zero is replaced ("0" + 5 -> "5"). Returns null when [MAX_DIGITS] is reached.
     */
    fun appendDigit(amount: String, digit: Int): String? {
        require(digit in 0..9) { "not a digit: $digit" }
        return when {
            amount == "0" -> digit.toString()
            amount.length >= MAX_DIGITS -> null
            else -> amount + digit
```

```
        }
    }

    /** Removes the rightmost digit of the typed [amount] ("234" -> "23"). Nothing typed stays empty. */
    fun removeLastDigit(amount: String): String = amount.dropLast(1)

    /** Numeric value of the typed digits; nothing typed counts as zero. */
    fun valueOf(amount: String): Long = amount.toLongOrNull() ?: 0L
}
```

Worked example: `breakdown(688)`

i	Note	`remaining / note` (count)	`remaining % note` (left over)
0	500	688 / 500 = **1**	188
1	100	188 / 100 = **1**	88
2	50	88 / 50 = **1**	38
3	20	38 / 20 = **1**	18
4	10	18 / 10 = **1**	8
5	5	8 / 5 = **1**	3
6	2	3 / 2 = **1**	1
7	1	1 / 1 = **1**	0

The result is `[1, 1, 1, 1, 1, 1, 1, 1]`: 8 notes.

IN DETAIL **Two more worked examples**

6880 Taka, the fourth screenshot in the brief (and the test on line 16 of `ChangeCalculatorTest.kt`):

Note	500	100	50	20	10	5	2	1
Count	13	3	1	1	1	0	0	0
Left over	380	80	30	10	0	0	0	0

13 + 3 + 1 + 1 + 1 = **19 notes**.

999,999,999 Taka, the largest amount the keypad accepts (the test on line 26):

Note	500	100	50	20	10	5	2	1
Count	1,999,999	4	1	2	0	1	2	0
Left over	499	99	49	9	9	4	0	0

That is 2,000,009 notes, nearly all of them 500s. The count 1,999,999 is the longest number the change table ever has to show, which is why the count text auto-sizes (Step 8).

Why is "take the biggest note first" (greedy) always optimal here? For some note systems it isn't. With notes {1, 3, 4}, greedy pays 6 as 4+1+1 (3 notes) instead of 3+3 (2 notes). Systems where greedy always works are called **canonical**. The Taka set {500, 100, 50, 20, 10, 5, 2, 1} is canonical, like most real currencies. The unit test `usesTheFewestNotesPossible` *proves* it for every amount from 0 to 2000 by comparing against a slow but always-correct **dynamic programming** solution.

Design notes:

- The amount is stored as a `String` of typed digits, not a number. Appending is then trivial (`amount + digit`), an empty amount ("nothing typed") is different from "0", and there is no overflow while typing.

- The value and the counts are `Long` (64-bit). 999,999,999 would still fit in an `Int` (whose maximum is about 2.1 billion), but `Long` keeps the logic correct if the digit limit is ever raised past 9.
- `appendDigit` returns `null` to say "rejected". The caller must handle that, and Kotlin's null safety enforces it.
- The leading-zero rule makes typing `0, 5` show `5`, like a calculator.

Step 12: One change-table row (`res/layout/view_note_row.xml`)

Create a new layout file. It is one row: a colour tab, the note label ("500:"), and the count.

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@drawable/bg_note_row"          <!-- rounded rect, coloured from code -->
    android:importantForAccessibility="yes"              <!-- TalkBack reads the ROW as one item -->
    android:paddingStart="@dimen/note_row_padding_start"
    android:paddingEnd="@dimen/note_row_padding_end">

    <View android:id="@+id/note_accent"                   <!-- the small colour tab -->
        android:layout_width="@dimen/note_accent_width"
        android:layout_height="@dimen/match_constraint"  <!-- 0dp: set by the percent below -->
        android:background="@drawable/bg_note_accent"
        app:layout_constraintHeight_percent="@dimen/note_accent_height_percent"  <!-- 50% of the row -->
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent" />   <!-- top+bottom = vertically centred -->

    <TextView android:id="@+id/note_label"                <!-- "500:" -->
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="@dimen/note_label_margin_start"
        android:importantForAccessibility="no"            <!-- the row's description covers it -->
        android:textAppearance="@style/TextAppearance.VangtiChai.NoteLabel"
        app:layout_constraintStart_toEndOf="@id/note_accent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent" />

    <TextView android:id="@+id/note_count"                <!-- "1" -->
        style="@style/Widget.VangtiChai.NoteCount"
        android:layout_width="@dimen/match_constraint"   <!-- fills from the label to the row's end -->
        android:layout_height="@dimen/match_constraint"  <!-- and the full row height: needed for auto-size
          ↪ -->
        android:layout_marginStart="@dimen/note_count_margin_start"
        android:importantForAccessibility="no"
        app:layout_constraintStart_toEndOf="@id/note_label"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

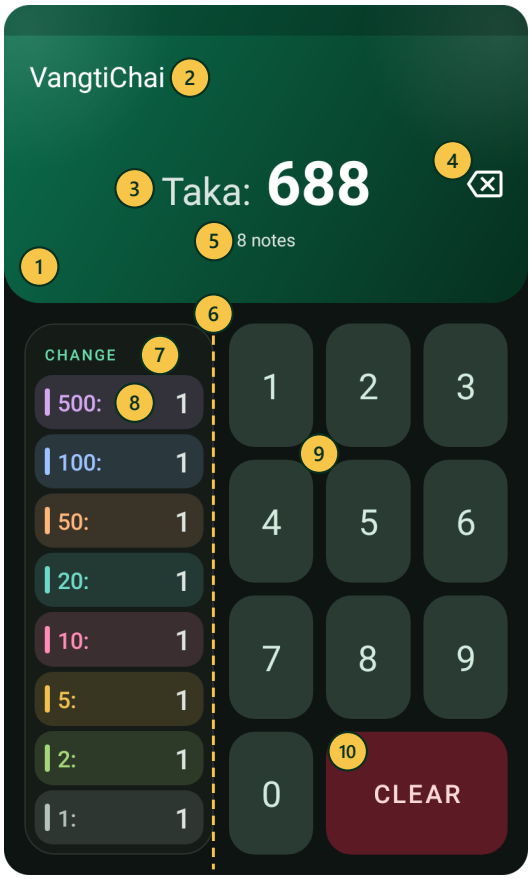
Why match constraints in both directions for the count? **Auto-sizing text needs a fixed box.** With `wrap_content` the box would just grow with the text, and the text would never shrink.

Why one reusable row file? Eight identical rows written out by hand would be 8 copies to keep in sync. `<include>` keeps one source of truth.

Step 13: The phone portrait layout (`res/layout/activity_main.xml`)

This is the main screen. Its structure (all inside one root `ConstraintLayout` with `android:id="@+id/root"` and `android:layoutDirection="ltr"`):

```
root (ConstraintLayout)
├── header (ConstraintLayout, gradient background)        ← top 32% of the screen
│   ├── toolbar (MaterialToolbar, title "VangtiChai")
│   ├── amount_text (TextView "Taka: 688")    ┐ packed vertical chain, centred
│   ├── caption_text (TextView "8 notes")     ┘ in the space under the toolbar
│   └── key_backspace (MaterialButton, icon)   ← right end of the amount row
├── split (Guideline, vertical, at 40%)
├── notes_card (MaterialCardView)                         ← left of split, under the header
│   └── ConstraintLayout
│       ├── notes_title ("CHANGE")
│       └── note_500 ... note_1 (8 × <include view_note_row>) ← vertical chain, equal heights
└── keypad (ConstraintLayout)                             ← right of split, same top/bottom as the card
    ├── key_1 key_2 key_3
    ├── key_4 key_5 key_6
    ├── key_7 key_8 key_9
    └── key_0 key_clear (CLEAR spans two columns)
```



1. `header`: a `ConstraintLayout` with the gradient background, 32% of the screen height.
2. `toolbar`: the `MaterialToolbar` showing the app name.
3. `amount_text`: "Taka: 688", one `TextView` with styled spans that auto-sizes.
4. `key_backspace`: invisible until a digit is typed.
5. `caption_text`: "8 notes", or "Enter an amount".
6. `split`: the vertical guideline at 40% of the width (dashed).
7. `notes_card`: the `MaterialCardView` with the "CHANGE" title.
8. `note_500` to `note_1`: eight included rows in a vertical chain.
9. `keypad`: a `ConstraintLayout` holding the 11 chained keys.
10. `key_clear`: CLEAR, spanning the second and third columns.

Figure 3. *The phone portrait layout on a Pixel XL, with the view ids from* `layout/activity_main.xml`*. The dashed line is the* `split` *guideline.*

The header:

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:id="@+id/header"
    android:layout_width="@dimen/match_constraint"
    android:layout_height="@dimen/match_constraint"
    android:background="@drawable/bg_header"
    android:paddingBottom="@dimen/header_padding_bottom"
    app:layout_constraintHeight_default="percent"                        <!-- height is a percentage... -->
    app:layout_constraintHeight_percent="@dimen/header_height_percent"   <!-- ...32% of the screen... -->
    app:layout_constraintHeight_min="wrap"                               <!-- ...but at least its content -->
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent">
```

Inside it:

- **The toolbar** sits at the top.
- `amount_text` has `layout_width="@dimen/match_constraint"` (match constraints) with equal start and end margins (`amount_side_margin`). That gives a centred box, with room on the right for the backspace key. Its fixed height (`amount_height`) is what lets its text auto-size.
- **The amount and caption** form a **packed vertical chain** (`app:layout_constraintVertical_chainStyle="packed"` on the first one): `toolbar.bottom ← amount ⇄ caption → header.bottom`. Packed means "keep them together, centred in the leftover space".
- **The backspace key** is `backspace_size` square. Its top and bottom are constrained to the amount's top and bottom, which centres it on the amount row. Its end is constrained to the parent's end with `backspace_margin_end`. It starts `android:visibility="invisible"`. `tools:visibility="visible"` shows it in Android Studio's preview only.

The split guideline and the card:

```
<androidx.constraintlayout.widget.Guideline android:id="@+id/split"
    android:orientation="vertical"
    app:layout_constraintGuide_percent="@dimen/notes_width_percent" />

<com.google.android.material.card.MaterialCardView android:id="@+id/notes_card"
    style="@style/Widget.VangtiChai.Card"
    android:layout_width="@dimen/match_constraint"
    android:layout_height="@dimen/match_constraint"
    android:layout_marginStart="@dimen/screen_margin"
    android:layout_marginTop="@dimen/section_gap"
    android:layout_marginBottom="@dimen/screen_margin"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toStartOf="@id/split"
    app:layout_constraintTop_toBottomOf="@id/header"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintHeight_max="@dimen/content_max_height"          <!-- don't get absurdly tall -->
    app:layout_constraintVertical_bias="@dimen/content_vertical_bias">   <!-- if capped, centre it -->
```

Inside the card, a `ConstraintLayout` holds the title and the 8 rows. The rows form a vertical chain from the title down to the bottom, all with `layout_height="@dimen/match_constraint"`, so they share the height **equally**:

```
<include android:id="@+id/note_500" layout="@layout/view_note_row"
    android:layout_width="@dimen/match_constraint"
    android:layout_height="@dimen/match_constraint"
    android:layout_marginTop="@dimen/notes_title_margin_vertical"
    app:layout_constraintTop_toBottomOf="@id/notes_title"
    app:layout_constraintBottom_toTopOf="@id/note_100"
    app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent" />
<include android:id="@+id/note_100" layout="@layout/view_note_row"
    android:layout_width="@dimen/match_constraint"
    android:layout_height="@dimen/match_constraint"
    android:layout_marginTop="@dimen/note_row_gap"
    app:layout_constraintTop_toBottomOf="@id/note_500"
    app:layout_constraintBottom_toTopOf="@id/note_50" ... />
<!-- ... note_50, note_20, note_10, note_5, note_2 the same way ... -->
<include android:id="@+id/note_1" ...
    app:layout_constraintTop_toBottomOf="@id/note_2"
    app:layout_constraintBottom_toBottomOf="parent" ... />
```

> **About `<include>` attributes.** To override layout attributes on an `<include>` (like `layout_width` and constraints), you must give **both** `layout_width` and `layout_height`. Otherwise the overrides are ignored.

The keypad. A `ConstraintLayout` placed right of `split`. Its top and bottom are constrained to the card's top and bottom, so it is **exactly as tall as the change table**.

```
<androidx.constraintlayout.widget.ConstraintLayout android:id="@+id/keypad"
    android:layout_width="@dimen/match_constraint"
    android:layout_height="@dimen/match_constraint"
    android:layout_marginStart="@dimen/column_gap"
    android:layout_marginEnd="@dimen/screen_margin"
    app:layout_constraintStart_toEndOf="@id/split"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="@id/notes_card"
    app:layout_constraintBottom_toBottomOf="@id/notes_card">
```

The 11 keys are arranged with chains:

- **Each of rows 1 to 3 is a horizontal chain:** `parent.start ← key_1 ⇄ key_2 ⇄ key_3 → parent.end`. All have width `@dimen/match_constraint`, so the three columns are equal. `key_2` and `key_3` have `layout_marginStart="@dimen/key_gap"` for the gaps.
- **The first column is a vertical chain:** `parent.top ← key_1 ⇄ key_4 ⇄ key_7 ⇄ key_0 → parent.bottom`, all height `@dimen/match_constraint`, giving four equal rows with `key_gap` between them.
- **Other keys copy their row's height** by constraining top and bottom to the first key of their row. For example, `key_5` has `top_toTopOf="@id/key_4"` and `bottom_toBottomOf="@id/key_4"`.
- `key_0` takes its start and end from `key_7`, so it is one column wide.
- **CLEAR spans two columns** by taking its start from `key_8` and its end from `key_9`, with top and bottom from `key_0`.

```
<com.google.android.material.button.MaterialButton android:id="@+id/key_clear"
    style="@style/Widget.VangtiChai.Key.Clear"
    android:layout_width="@dimen/match_constraint"
    android:layout_height="@dimen/match_constraint"
    android:contentDescription="@string/cd_key_clear"
    android:text="@string/key_clear"
    app:layout_constraintStart_toStartOf="@id/key_8"
    app:layout_constraintEnd_toEndOf="@id/key_9"
    app:layout_constraintTop_toTopOf="@id/key_0"
    app:layout_constraintBottom_toBottomOf="@id/key_0" />
```

Why not `GridLayout` *or a* `TableLayout`*?* The assignment is about ConstraintLayout. Chains also let the keys stretch to whatever height the change table has, so the keypad always lines up with it.

Step 14: The phone landscape layout (`res/layout-land/activity_main.xml`)

Create `res/layout-land/` and a copy of `activity_main.xml` in it, then change it. Height is scarce in landscape (411 dp on a Pixel XL), so:

1. **The header is** `wrap_content`, not a percentage. The **caption moves up into the toolbar row**, aligned to the right (`gravity="end"`, top and bottom constrained to the toolbar's). The amount sits on its own line under the toolbar.
2. **The change table has two columns.** A second guideline, `notes_columns`, at `@dimen/half` (50%) splits the card:
   - The left column is a vertical chain `note_500` ⇄ `note_100` ⇄ `note_50` ⇄ `note_20`, with `end_toStartOf="@id/notes_columns"`.
   - The right column is `note_10` ⇄ `note_5` ⇄ `note_2` ⇄ `note_1`, with `start_toEndOf="@id/notes_columns"`.
   - `note_column_half_gap` margins on each side of the guideline make the gap between the columns.
3. **The keypad has four columns:**

   ```
   1 2 3 4
   5 6 7 8
   9 0 CLEAR      (0 lines up with the 2nd column, CLEAR spans the 3rd and 4th)
   ```

   Rows 1 and 2 are horizontal chains of four keys. The first column is a vertical chain `key_1` ⇄ `key_5` ⇄ `key_9`. `key_9` and `key_0` take their columns from `key_5` and `key_6`. CLEAR goes from `key_7`'s start to `key_8`'s end.
4. `notes_width_percent` is 0.42 here (from `values-land/sizes.xml`), and there is no `content_max_height` cap.

The **ids are the same** as in portrait (`amount_text`, `key_1`, `note_500` and so on), so the same Kotlin code drives both layouts.

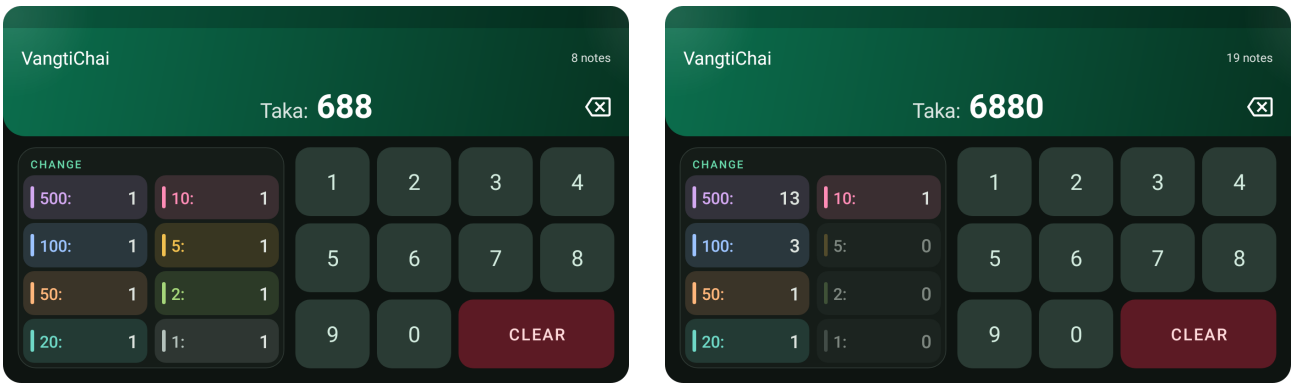


Figure 4. *Phone landscape (`layout-land`). The caption moves into the toolbar row, the change table splits into two columns (500 to 20, then 10 to 1) and the keypad has four columns, with 9, 0 and a wide CLEAR at the bottom. Right: 6880 Taka, with the 5, 2 and 1 rows faded.*

Step 15: The tablet layouts (`layout-sw600dp/` and `layout-sw600dp-land/`)

Tablet portrait (`layout-sw600dp/activity_main.xml`) is the phone portrait layout with three additions:

- The toolbar uses `android:layout_height="@dimen/toolbar_height"` (76 dp) and shows a subtitle, `app:subtitle="@string/app_tagline"` ("Taka change calculator").

- The card has a **"COUNT" heading** (`notes_count_heading`) on the right of the title. It is lined up with the title using `app:layout_constraintBaseline_toBaselineOf="@id/notes_title"`.
- All sizes come from `values-sw600dp/sizes.xml`, which is scaled up (key text 46 sp, amount up to 88 sp, and so on).

Tablet landscape (`layout-sw600dp-land/activity_main.xml`):

- Below the header it is identical to phone landscape: a two-column table and a 4-column keypad.
- The header is like tablet portrait: a toolbar with a subtitle, and the caption *under* the amount instead of in the toolbar row, because a tablet has enough height.

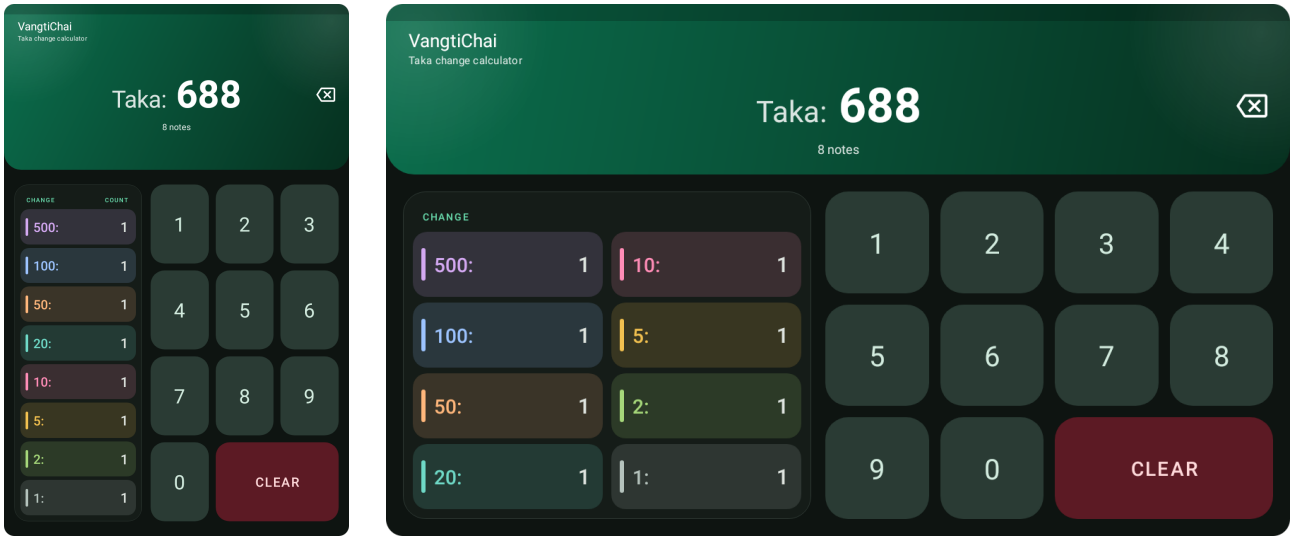


Figure 5. *The tablet layouts at Nexus 10 size. Portrait (left) adds the subtitle "Taka change calculator" and a COUNT heading. Landscape (right) combines the two-column table and four-column keypad with the caption under the amount.*

> **Tip: previewing layouts.** In Android Studio's layout editor, use the device dropdown to choose "Pixel XL" or "Nexus 10". The orientation button rotates the preview. The `tools:text` and `tools:visibility` attributes make the preview look realistic.

Step 16: The screen's behaviour, `MainActivity.kt`, section by section

`MainActivity` connects the layouts to the logic. The whole file is in the Appendix. Below, it is explained from top to bottom.

16.1 The core idea: state, then render

The activity holds **two pieces of state**:

```
private var amount = ""                                    // the digits typed so far, e.g. "688"
private var change = LongArray(ChangeCalculator.NOTES.size)    // the count of each note, e.g.
  ↪ [1,1,1,1,1,1,1,1]
```

Every input goes through **one function** that updates both and redraws:

```
private fun setAmount(newAmount: String) {
    amount = newAmount
    change = ChangeCalculator.breakdown(ChangeCalculator.valueOf(amount))
    render(animate = true)
}
```

`render()` makes the screen match the state: the amount text, the caption, the 8 rows and the backspace key. This "**state goes in, the UI comes out**" pattern (often called *unidirectional data flow*) means the UI can never get out of sync. There's no code that updates "just the 50 row" and forgets something else.

16.2 Fields (the class's variables)

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding             // all views of activity_main.xml
    private lateinit var noteRows: List<ViewNoteRowBinding>       // the 8 rows, in NOTES order
    private lateinit var rowShapes: List<GradientDrawable>        // each row's background shape
    private lateinit var accentShapes: List<GradientDrawable>     // each row's colour tab shape
    private lateinit var digitKeys: List<MaterialButton>          // digitKeys[3] is the "3" key

    private var amount = ""
    private var change = LongArray(ChangeCalculator.NOTES.size)
```

`lateinit` is needed because views only exist after `onCreate` inflates the layout.

Colours, resolved once and cached with `by lazy`:

```
private val noteColors by lazy { NOTE_COLORS.map { ContextCompat.getColor(this, it) } }   // 8 note colours
private val idleRowColor by lazy { ContextCompat.getColor(this, R.color.note_row_idle) }
private val litRowColors by lazy {
    val card = ContextCompat.getColor(this, R.color.card_surface)
    noteColors.map { ColorUtils.compositeColors(ColorUtils.setAlphaComponent(it, ACTIVE_ROW_ALPHA), card) }
}
private val labelColor by lazy { ContextCompat.getColor(this, R.color.header_content_secondary) }
private val strongTextColor by lazy {
    MaterialColors.getColor(binding.root, com.google.android.material.R.attr.colorOnSurface)
}
private val mutedTextColor by lazy {
    val color = MaterialColors.getColor(binding.root, com.google.android.material.R.attr.
      ↪ colorOnSurfaceVariant)
    ColorUtils.setAlphaComponent(color, MUTED_TEXT_ALPHA)
}
```

- `ContextCompat.getColor(context, R.color.x)` turns a colour *resource ID* into an actual colour value (an `Int` like `0xFF1C2420`).
- `MaterialColors.getColor(view, attr)` reads a **theme attribute** (`?attr/colorOnSurface`). It needs a view, which is why it reads `binding.root` and must be lazy.
- `ColorUtils.setAlphaComponent(color, 0x99)` returns the same colour at about 60% opacity. `0x99` = 153 out of 255.
- `litRowColors` is the faint note colour (`ACTIVE_ROW_ALPHA` = `0x29`, about 16%) *already blended over the card colour* (`compositeColors`). The result is opaque. Why that matters is explained in 16.10.

Animation bookkeeping:

```
private val rowLevels = FloatArray(ChangeCalculator.NOTES.size)                // 0 = faded, 1 = lit, per
  ↪ row
private val rowAnimators = arrayOfNulls<ValueAnimator>(ChangeCalculator.NOTES.size)  // the running fade,
  ↪ per row
```

Two `Runnable`s. A `Runnable` is "a piece of code to run later":

```
private val restoreCaption = Runnable { binding.captionText.text = caption() }   // undo "Up to 9 digits"

private val repeatBackspace = object : Runnable {
    override fun run() {
        if (amount.isEmpty()) return                          // nothing left: stop repeating
        val key = binding.keyBackspace
        key.performHapticFeedback(HapticFeedbackConstants.CLOCK_TICK)   // a light tick per digit
        onBackspace()                                          // delete one digit
        key.postDelayed(this, BACKSPACE_REPEAT_INTERVAL)       // schedule myself again in 70 ms
    }
}
```

`view.postDelayed(runnable, ms)` puts the runnable on the main thread's queue to run after `ms` milliseconds. `view.removeCallbacks(runnable)` cancels it if it hasn't run yet. `repeatBackspace` is a **self-rescheduling** runnable: each run deletes a digit and books the next run. It must be an `object : Runnable` (not a lambda) because it refers to itself as `this`.

16.3 `onCreate`: setting everything up

```
override fun onCreate(savedInstanceState: Bundle?) {
    enableEdgeToEdge(
        statusBarStyle = SystemBarStyle.dark(Color.TRANSPARENT),
        navigationBarStyle = SystemBarStyle.dark(Color.TRANSPARENT),
    )
    super.onCreate(savedInstanceState)
```

Edge-to-edge means the app draws *behind* the status bar (top) and the navigation bar (bottom), so the green header reaches the very top of the screen. `SystemBarStyle.dark(...)` means "the background behind the bar is dark, so draw **light** icons". Both bars are set to dark because the app is always dark. Without the navigation-bar setting, a phone in *light* mode would draw dark navigation icons on the app's dark background, where they would be invisible. `enableEdgeToEdge` has to run before the content view is set. Calling it first, before `super.onCreate`, is the usual place.

```
    binding = ActivityMainBinding.inflate(layoutInflater)   // build the view tree from XML
    setContentView(binding.root)                            // make it the screen's content
    applySystemBarInsets()                                  // keep content clear of the bars (16.11)
```

Android picks which `activity_main.xml` to inflate (portrait, landscape or tablet) based on the current configuration.

```
    noteRows = with(binding) { listOf(note500, note100, note50, note20, note10, note5, note2, note1) }
    noteRows.forEachIndexed { i, row ->
        row.noteLabel.text = getString(R.string.note_label, ChangeCalculator.NOTES[i])   // "500:",
          ↪ "100:"...
    }
    rowShapes = noteRows.map { it.root.background.mutate() as GradientDrawable }        // coloured in 16.10
    accentShapes = noteRows.map { it.noteAccent.background.mutate() as GradientDrawable }
```

The 8 included rows are collected **in the same order as** `ChangeCalculator.NOTES`, so `noteRows[i]` always shows `change[i]`. Each label is filled from code, so the row layout stays generic.

The last two lines keep each row's two shapes, its background and its colour tab, so 16.10 can colour them. All eight rows are inflated from the same drawable files, and Android shares one copy of a drawable's state between everything that uses it, so recolouring one row would recolour all of them. `mutate()` gives each row its own copy first.

```
    digitKeys = with(binding) { listOf(key0, key1, key2, key3, key4, key5, key6, key7, key8, key9) }
    digitKeys.forEachIndexed { digit, key ->
        key.setOnClickListener {
            it.performHapticFeedback(HapticFeedbackConstants.KEYBOARD_TAP)   // a small vibration
            onDigit(digit)
        }
    }
```

The list is ordered so that **the index equals the digit**: `digitKeys[7]` is the "7" key. Each listener **captures** its own `digit` value (a *closure*), so one loop wires up all 10 keys.

```
    binding.keyClear.setOnClickListener {
        it.performHapticFeedback(HapticFeedbackConstants.KEYBOARD_TAP)
        onClear()
    }

    // Backspace works like a phone's: a tap deletes one digit, holding it keeps deleting.
    binding.keyBackspace.setOnClickListener {
        it.performHapticFeedback(HapticFeedbackConstants.KEYBOARD_TAP)
        onBackspace()
    }
    binding.keyBackspace.setOnLongClickListener {
        it.removeCallbacks(repeatBackspace)     // never run two repeat loops at once
        repeatBackspace.run()                   // delete one now, and schedule the rest
        true                                    // "handled": also stops a click firing on release
    }
    binding.keyBackspace.setOnTouchListener { key, event ->
        if (event.actionMasked == MotionEvent.ACTION_UP || event.actionMasked == MotionEvent.ACTION_CANCEL)
          ↪ {
            key.removeCallbacks(repeatBackspace)   // finger lifted (or gesture cancelled): stop repeating
        }
        false // The button still handles the touch itself: press state, click and long click
    }
```

How tap versus hold works, as a timeline:

Time	Quick tap	Hold
0 ms	Finger down: the key shows pressed and shrinks	Finger down
~100 ms	Finger up: the **click** listener deletes 1 digit	(still holding)
~400 ms (long-press timeout)		The **long-click** listener deletes 1 digit and schedules a repeat
+70, +140, ... ms		The repeat deletes 1 digit each time
finger up		The **touch** listener cancels the pending repeat. No click fires, because the long click returned `true`.

The touch listener returns `false`, which means "I only peeked; carry on with normal handling". The button's own pressed state, ripple, click and long-click still work.

```
    if (savedInstanceState != null) {                       // we're being recreated (e.g. rotation)
        amount = savedInstanceState.getString(KEY_AMOUNT).orEmpty()
        change = savedInstanceState.getLongArray(KEY_CHANGE)
            ?: ChangeCalculator.breakdown(ChangeCalculator.valueOf(amount))   // fallback: recompute
    }
    render(animate = false)                                 // draw the current state, no animations
    if (savedInstanceState == null) playEntrance()          // fresh start only: the content rises in
}
```

16.4 `onSaveInstanceState`: surviving rotation

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putString(KEY_AMOUNT, amount)
    outState.putLongArray(KEY_CHANGE, change)
}
```

Both the amount *and* the change list are saved, as the brief says to "keep the currently entered amount and change list". The change could be recomputed from the amount, so `onCreate` falls back to recomputing if it is missing. `KEY_AMOUNT` and `KEY_CHANGE` are constants, so a typo can't make save and restore use different keys.

> IN DETAIL **A rotation, step by step**
>
> 1. The user turns the phone. The orientation and screen size change, and because the manifest does not ask to handle that itself, Android decides to recreate the activity.
> 2. The old activity is paused and stopped. Android calls `onSaveInstanceState()`, and the app puts "688" under `"amount"` and [1, 1, 1, 1, 1, 1, 1, 1] under `"change"` in the Bundle. (For apps targeting Android 9 or newer, this call comes after `onStop()`.)
> 3. The old activity is destroyed, along with every view and field in it.
> 4. A new `MainActivity` is created and `onCreate()` runs with the saved Bundle. This time `ActivityMainBinding.inflate()` picks `layout-land/activity_main.xml`, because the configuration now says landscape.
> 5. `onCreate()` reads "688" and the change list back, calls `render(animate = false)` so everything appears in place, and skips `playEntrance()` because the Bundle is not null.
> 6. `onStart()` and `onResume()` follow. The landscape screen shows the same amount and the same table.
>
> The `amountAndChangeSurviveRecreation` test drives exactly this path with `scenario.recreate()` (Step 17.2).

16.5 `onKeyDown`: hardware keyboards

```
override fun onKeyDown(keyCode: Int, event: KeyEvent): Boolean {
    val key = when (keyCode) {
        in KeyEvent.KEYCODE_0..KeyEvent.KEYCODE_9 -> digitKeys[keyCode - KeyEvent.KEYCODE_0]
        in KeyEvent.KEYCODE_NUMPAD_0..KeyEvent.KEYCODE_NUMPAD_9 -> digitKeys[keyCode -
          ↪ KeyEvent.KEYCODE_NUMPAD_0]
        KeyEvent.KEYCODE_ESCAPE, KeyEvent.KEYCODE_CLEAR, KeyEvent.KEYCODE_C -> binding.keyClear
        KeyEvent.KEYCODE_DEL -> binding.keyBackspace
        else -> return super.onKeyDown(keyCode, event)
    }
    key.flashPressed()     // show the on-screen key pressed for 90 ms
    key.performClick()     // run the same click listener a finger tap would
    return true            // "consumed"
}
```

Key codes for 0 to 9 are consecutive numbers, so `keyCode - KEYCODE_0` gives the digit. Instead of duplicating logic, a physical key press **clicks the on-screen button**. Haptics, rendering and everything else behave identically. Holding a physical key repeats automatically, because Android sends repeated `onKeyDown` events. This is handy on an emulator, where you can type with your computer's keyboard.

16.6 Input handlers

```
private fun onDigit(digit: Int) {
    val updated = ChangeCalculator.appendDigit(amount, digit)
    if (updated == null) {        // 9 digits already: refuse
        rejectDigit()
        return
```

```
    }
    setAmount(updated)
}

private fun onClear() = setAmount("")

private fun onBackspace() {
    if (amount.isNotEmpty()) setAmount(ChangeCalculator.removeLastDigit(amount))
}
```

All three end in `setAmount`, the single place where state changes.

16.7 `rejectDigit`: telling the user "no"

```
private fun rejectDigit() {
    val amountView = binding.amountText
    amountView.performHapticFeedback(
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) HapticFeedbackConstants.REJECT   // Android 11+
        else HapticFeedbackConstants.LONG_PRESS                                              // older
    )
    val d = resources.getDimension(R.dimen.shake_distance)          // 10dp, converted to pixels
    ObjectAnimator.ofFloat(amountView, View.TRANSLATION_X, 0f, d, -d, d / 2, -d / 2, 0f)
        .setDuration(SHAKE_DURATION)                                // 400 ms
        .start()

    val caption = binding.captionText
    caption.text = resources.getQuantityString(
        R.plurals.caption_max_digits, ChangeCalculator.MAX_DIGITS, ChangeCalculator.MAX_DIGITS
    )                                                               // "Up to 9 digits"
    caption.removeCallbacks(restoreCaption)                         // restart the timer if tapped again
    caption.postDelayed(restoreCaption, CAPTION_RESTORE_DELAY)      // put "N notes" back after 1.6 s
}
```

- An `ObjectAnimator` animates one property of one object through a list of values (keyframes). Here it moves the amount right, left, right a little, left a little, then back to centre: a "no" shake.
- `TRANSLATION_X` moves the view's drawing without changing the layout.
- `getDimension` converts dp to pixels for the current screen.
- The message comes from plurals with `MAX_DIGITS`, so changing the limit automatically updates the text.

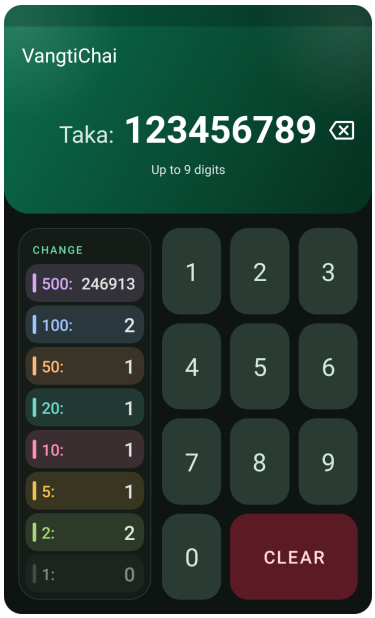


Figure 6. *A tenth digit is refused: the amount (123,456,789) shakes and the caption reads "Up to 9 digits" for 1.6 seconds. The 500 row needs 246,913 notes, and its count has shrunk to fit.*

16.8 `render`: drawing the state

```
private fun render(animate: Boolean) {
    binding.amountText.text = buildSpannedString {
        scale(LABEL_SCALE) { color(labelColor) { append(getString(R.string.label_taka)) } }  // smaller,
          ↪ softer "Taka:"
        if (amount.isNotEmpty()) {
            append(' ')
            bold { append(amount) }                                                          // bold digits
        }
    }
```

A **Spanned string** is text with styling on parts of it (spans). `buildSpannedString { }` (from core-ktx) builds one:

- `scale(0.62f) { ... }` makes the text inside 62% size.
- `color(c) { ... }` colours it.
- `bold { ... }` makes it bold.

So one `TextView` shows "Taka:" small and soft and "688" big and bold. Using one view (not two) keeps the pair centred together and auto-sizing together.

```
    binding.amountText.contentDescription =
        if (amount.isEmpty()) getString(R.string.cd_amount_empty) else getString(R.string.cd_amount, amount)
```

A screen reader would otherwise read "Taka colon six eight eight". The content description makes it say "688 Taka", or "No amount entered".

```
    binding.captionText.removeCallbacks(restoreCaption)   // a new input cancels a pending "restore"
    binding.captionText.text = caption()                  // "Enter an amount" or "8 notes"

    change.forEachIndexed { i, count -> renderRow(i, count, animate) }
    renderBackspace(animate)
    if (animate) binding.amountText.pop(AMOUNT_POP_SCALE)   // a subtle 6% pop on every change
}

private fun caption(): String {
    if (amount.isEmpty()) return getString(R.string.hint_enter_amount)
    val notes = change.sum()
    return resources.getQuantityString(R.plurals.caption_note_count, notes.toInt(), notes.toString())
}
```

Why the `animate` flag? On first draw and after rotation, things should just *appear* (`animate = false`). Animations play only in response to the user's input.

16.9 `renderBackspace`: showing the key only when useful

```
private fun renderBackspace(animate: Boolean) {
    val key = binding.keyBackspace
    val show = amount.isNotEmpty()
    val alpha = if (show) 1f else 0f
    if (key.isVisible == show && key.alpha == alpha) return   // already right: do nothing
    key.animate().cancel()                                    // stop any fade in progress
    if (!animate) {
        key.alpha = alpha
        key.isInvisible = !show
        return
    }
    key.isInvisible = false                                   // must be visible to fade
    key.animate()
        .alpha(alpha)
        .setDuration(BACKSPACE_FADE_DURATION)                 // 150 ms
        .withEndAction { key.isInvisible = !show }            // after fading out, hide it properly
        .start()
}
```

- `view.animate()` returns a **ViewPropertyAnimator**, the simplest way to animate a view's alpha, translation or scale.
- `INVISIBLE` **vs** `GONE`**:** `INVISIBLE` hides the view but keeps its space; `GONE` removes it from the layout. `INVISIBLE` is used so nothing shifts when the key appears. An invisible view also can't be tapped or focused by TalkBack, which is correct when there's nothing to delete.

16.10 Rows: `renderRow`, `lightRow` and `setRowLevel`

```
private fun renderRow(index: Int, count: Long, animate: Boolean) {
    val row = noteRows[index]
    val countText = count.toString()
    val changed = row.noteCount.text.toString() != countText
    row.noteCount.text = countText
    row.root.contentDescription = resources.getQuantityString(
        R.plurals.cd_note_row, count.toInt(), ChangeCalculator.NOTES[index], countText
    )                                                    // "1 note of 500 Taka"
    lightRow(index, if (count > 0) 1f else 0f, animate)  // lit if it's part of the change
    if (animate && changed) row.noteCount.pop(COUNT_POP_SCALE, pivotAtEnd = true)   // pop only if it
      ↪ changed
}
```

Each row has a **level** from 0 (faded back) to 1 (lit up). `lightRow` animates the level:

```
private fun lightRow(index: Int, target: Float, animate: Boolean) {
    rowAnimators[index]?.cancel()               // stop this row's previous fade, if any
    rowAnimators[index] = null
    val shown = rowLevels[index]                // where it is right now (maybe mid-fade)
    if (!animate || shown == target) {
        setRowLevel(index, target)
        return
    }
    rowAnimators[index] = ValueAnimator.ofFloat(shown, target).apply {   // animate a number shown→target
        duration = ROW_FADE_DURATION                                      // 220 ms
        addUpdateListener { setRowLevel(index, it.animatedValue as Float) }   // every frame: recolour
        start()
    }
}
```

A `ValueAnimator` just produces numbers over time, and you decide what to do with them in the update listener. Starting from `shown` (not 0 or 1) means that if you type quickly, a half-finished fade continues smoothly from where it is, with no jump.

```
private fun setRowLevel(index: Int, level: Float) {
    rowLevels[index] = level
    val row = noteRows[index]
    val noteColor = noteColors[index]
    row.noteLabel.setTextColor(ColorUtils.blendARGB(mutedTextColor, noteColor, level))
    row.noteCount.setTextColor(ColorUtils.blendARGB(mutedTextColor, strongTextColor, level))
    accentShapes[index].setColor(
        ColorUtils.blendARGB(ColorUtils.setAlphaComponent(noteColor, IDLE_ACCENT_ALPHA), noteColor, level)
    )
    rowShapes[index].setColor(ColorUtils.blendARGB(idleRowColor, litRowColors[index], level))
}
```

`ColorUtils.blendARGB(a, b, t)` mixes colour `a` and colour `b`: `t = 0` gives all `a`, `t = 1` all `b`, and `0.5` half and half. All four parts of the row (label, count, colour tab, background) use the **same level**, so they fade together.

`GradientDrawable.setColor()` replaces the fill of the row's `bg_note_row` and `bg_note_accent` shapes, which are white in the XML. A background *tint* (`backgroundTintList`) looks the same on newer Android, and an earlier version of this code used one. But Android 5.0 (API 21) can't tint shape drawables. On the Android 5.0 emulator every row stayed white, with the light text on top almost invisible. `setColor()` works on every version.

Why `litRowColors` is pre-blended (opaque). The lit background is the note colour at 16% opacity over the card. If you blend an *opaque* idle colour toward a *16%-transparent* colour, the halfway point is a half-transparent, half-strength colour. Over the dark card it actually looks **brighter** than both ends, so you'd see a flash mid-fade. Pre-blending the lit colour onto the card colour (`compositeColors`) makes both ends opaque. The fade then moves steadily from one to the other, and the final colour on screen is exactly the same as before. This was measured: the on-screen brightness goes 0.016, 0.024, 0.035 (idle, halfway, lit).

IN DETAIL **The flash, in numbers**

Take the 50 row. These are the colours its background goes through, worked out from `colors.xml` with the same arithmetic `ColorUtils` uses. Brightness is relative luminance, from 0 (black) to 1 (white).

Row background	On screen	Brightness
Idle (level 0)	#1C2420	0.016
Halfway, blending two opaque colours (what the app does)	#2B2C24	0.024
Lit (level 1)	#3A3428	0.035
Halfway, blending toward the see-through colour (the old way)	#5B4B37	0.075

Blended toward the see-through colour, the halfway point is a half-transparent mix of the idle grey and the full orange. Drawn over the dark card, it comes out more than twice as bright as the lit row it is heading for, which is the flash. Blending between two opaque colours keeps every frame between the two ends: 0.016, then 0.024, then 0.035, the values quoted above. Figure 1 shows both end states, lit rows in their note colour and faded rows in grey.

16.11 `applySystemBarInsets`: edge-to-edge done right

```
private fun applySystemBarInsets() {
    val header = binding.header
    val headerLeft = header.paddingLeft            // remember the ORIGINAL paddings and margins...
    val headerTop = header.paddingTop
    val headerRight = header.paddingRight
    val notesMargins = binding.notesCard.layoutParams as ViewGroup.MarginLayoutParams
    val notesStart = notesMargins.marginStart
    val notesBottom = notesMargins.bottomMargin
    val keypadEnd = (binding.keypad.layoutParams as ViewGroup.MarginLayoutParams).marginEnd

    ViewCompat.setOnApplyWindowInsetsListener(binding.root) { _, insets ->
        val bars = insets.getInsets(
            WindowInsetsCompat.Type.systemBars() or WindowInsetsCompat.Type.displayCutout()
        )                                          // sizes of status bar, nav bar and notch, in px
        header.updatePadding(                      // header: extends under the status bar,
            left = headerLeft + bars.left,         // but its CONTENT is pushed down/in
            top = headerTop + bars.top,
            right = headerRight + bars.right,
        )
        binding.notesCard.updateLayoutParams<ViewGroup.MarginLayoutParams> {
            marginStart = notesStart + bars.left       // clear of a left-side nav bar or cutout
            bottomMargin = notesBottom + bars.bottom   // clear of the bottom nav bar
        }
        binding.keypad.updateLayoutParams<ViewGroup.MarginLayoutParams> {
            marginEnd = keypadEnd + bars.right         // clear of a right-side nav bar (landscape)
        }
        WindowInsetsCompat.CONSUMED                    // children don't need to handle insets again
    }
}
```

- **Insets** are the sizes of the system UI that overlaps your app: the status bar at the top, the navigation bar at the bottom (or the side in landscape), and display cutouts (camera notches).

- The listener can be called **many times** (rotation, the nav bar appearing). That's why it adds the insets to the **original** values saved outside the lambda. Adding to the *current* values would grow the paddings every time.
- The keypad's top and bottom follow the card, so only its right side needs adjusting.
- `or` here is a **bitwise OR**, combining two inset types into one request.

16.12 `playEntrance`: the opening animation

```
private fun playEntrance() {
    val rise = resources.getDimension(R.dimen.entrance_rise)        // 24dp (40dp on tablets)
    listOf(binding.notesCard, binding.keypad).forEachIndexed { i, section ->
        section.alpha = 0f                                          // start invisible...
        section.translationY = rise                                 // ...and a bit lower
        section.animate()
            .alpha(1f)
            .translationY(0f)
            .setStartDelay(ENTRANCE_DELAY + i * ENTRANCE_STAGGER)    // card at 60 ms, keypad at 130 ms
            .setDuration(ENTRANCE_DURATION)                         // 380 ms
            .setInterpolator(DecelerateInterpolator(ENTRANCE_DECELERATION))  // fast start, gentle stop
            .start()
    }
}
```

- An **interpolator** shapes an animation's speed over time. *Decelerate* starts fast and eases into place, which feels natural for things arriving.
- The **stagger** (the keypad starts 70 ms after the card) makes the motion feel organised.
- It runs only when `savedInstanceState == null`, so rotation stays instant.
- If the user has turned animations off in Android's settings, the system's animator duration scale is 0 and everything appears instantly.

16.13 Two small helpers (extension functions)

```
/** Briefly enlarges the view and lets it spring back, drawing the eye to a changed value. */
private fun View.pop(scale: Float, pivotAtEnd: Boolean = false) {
    animate().cancel()
    pivotX = if (pivotAtEnd) width.toFloat() else width / 2f   // grow from the right edge (counts) or
      ↪ centre
    pivotY = height / 2f
    scaleX = scale                                             // jump to e.g. 135%...
    scaleY = scale
    animate()
        .scaleX(1f)
        .scaleY(1f)
        .setDuration(POP_DURATION)                             // ...and spring back over 280 ms
        .setInterpolator(OvershootInterpolator(OVERSHOOT_TENSION))
        .start()
}

/** Shows the pressed state for a moment, so hardware key presses are visible on screen. */
private fun View.flashPressed() {
    isPressed = true
    postDelayed({ isPressed = false }, KEY_FLASH_DURATION)
}
```

The **pivot** is the point a scale grows from. Counts are right-aligned, so they pop from their right edge and don't jump sideways.

16.14 The constants

```
private companion object {
    const val KEY_AMOUNT = "amount"            // Bundle keys
    const val KEY_CHANGE = "change"
    val NOTE_COLORS = intArrayOf(R.color.note_500, ..., R.color.note_1)   // same order as NOTES
    const val LABEL_SCALE = 0.62f              // "Taka:" size relative to the number
    const val ACTIVE_ROW_ALPHA = 0x29          // lit row: note colour at ~16%
    const val IDLE_ACCENT_ALPHA = 0x40         // faded colour tab: ~25%
    const val MUTED_TEXT_ALPHA = 0x99          // faded text: ~60%
    const val AMOUNT_POP_SCALE = 1.06f
    const val COUNT_POP_SCALE = 1.35f
    const val POP_DURATION = 280L
    const val OVERSHOOT_TENSION = 3f           // how far the pop overshoots
    const val ROW_FADE_DURATION = 220L
    const val SHAKE_DURATION = 400L
    const val CAPTION_RESTORE_DELAY = 1600L
    const val KEY_FLASH_DURATION = 90L
    const val BACKSPACE_REPEAT_INTERVAL = 70L
    const val BACKSPACE_FADE_DURATION = 150L
    const val ENTRANCE_DELAY = 60L
    const val ENTRANCE_STAGGER = 70L
    const val ENTRANCE_DURATION = 380L
    const val ENTRANCE_DECELERATION = 2f
}
```

All durations are in milliseconds (`Long`, hence the `L`). Named constants make the code self-explanatory and easy to tune. Why are these in code but sizes in XML? Sizes must vary per screen (resource qualifiers). Timings and opacities don't.

Step 17: Tests

17.1 Unit tests: `app/src/test/java/com/example/vangtichai/ChangeCalculatorTest.kt`

These run on your computer's JVM, without Android, in under a second: `./gradlew test`, or click the green arrow next to the class in Android Studio.

Test	What it proves
`breakdownMatchesTheAssignmentScreenshots`	6, 68, 688 and 6880 give exactly the counts in the brief's screenshots.
`zeroNeedsNoNotes`	0 gives all zeros.
`largestAcceptedAmount`	999,999,999 gives `[1999999, 4, 1, 2, 0, 1, 2, 0]`.
`notesAlwaysAddUpToTheAmount`	For every amount from 0 to 5000, the sum of count × note equals the amount.
`usesTheFewestNotesPossible`	For 0 to 2000, greedy's note count equals the true minimum found by dynamic programming.
`negativeAmountIsRejected`	`breakdown(-1)` throws `IllegalArgumentException`.
`digitsAreAddedFromTheRight`	"" + 2 gives "2", + 3 gives "23", + 4 gives "234".
`leadingZeroIsReplaced`	"0" + 5 gives "5", "0" + 0 gives "0", "5" + 0 gives "50".
`inputStopsAtMaxDigits`	A full amount rejects another digit (returns `null`), and one digit less accepts it.
`backspaceRemovesTheRightmostDigit`	"234" becomes "23", then "2", then "", and "" stays "".
`valueOfTypedDigits`	"" is 0 and "688" is 688.

The dynamic-programming check, explained:

```
val limit = 2_000
val fewest = IntArray(limit + 1) { Int.MAX_VALUE }   // fewest[a] = fewest notes that make amount a
fewest[0] = 0                                         // zero notes make 0
for (amount in 1..limit) {
    for (note in ChangeCalculator.NOTES) {
        // "use one `note`, then the best way to pay what's left"
        if (note <= amount) fewest[amount] = minOf(fewest[amount], fewest[amount - note] + 1)
    }
}
for (amount in 0..limit) {
    assertEquals("amount $amount", fewest[amount].toLong(),
      ↪ ChangeCalculator.breakdown(amount.toLong()).sum())
}
```

`assertEquals(message, expected, actual)` fails the test, printing the message, if they differ. `assertArrayEquals` compares arrays element by element.

17.2 UI tests: `app/src/androidTest/java/com/example/vangtichai/MainActivityTest.kt`

These run **on a device or emulator** (`./gradlew connectedAndroidTest`). They use **Espresso**, which finds views and performs real taps:

```
@RunWith(AndroidJUnit4::class)
class MainActivityTest {
    @get:Rule
    val activityRule = ActivityScenarioRule(MainActivity::class.java)   // launches the activity per test

    @Test
    fun digitsAreAddedFromTheRight() {
        tap(R.id.key_2); assertAmount("Taka: 2")
        tap(R.id.key_3); assertAmount("Taka: 23")
        tap(R.id.key_4); assertAmount("Taka: 234")
    }

    @Test
    fun amountAndChangeSurviveRecreation() {
        type("688")
        activityRule.scenario.recreate()            // the same destroy/recreate path as rotation
        assertAmount("Taka: 688")
        assertCounts("1", "1", "1", "1", "1", "1", "1", "1")
    }

    private fun tap(@IdRes key: Int) { onView(withId(key)).perform(click()) }
    private fun assertAmount(text: String) {
        onView(withId(R.id.amount_text)).check(matches(withText(text)))
    }
    // assertCounts finds each row's note_count INSIDE that row:
    //   onView(allOf(withId(R.id.note_count), isDescendantOfA(withId(row))))
    // because all 8 rows contain a view with the same id note_count.
}
```

The 7 tests:

- `digitsAreAddedFromTheRight`
- `changeTableListsTheNotes` (6880)
- `clearResetsAmountAndTable`
- `backspaceRemovesTheLastDigit`
- `backspaceShowsOnceSomethingIsTyped` (hidden at start, shown after typing)
- `holdingBackspaceKeepsDeleting` (Espresso's `longClick()` must delete at least two of seven digits)
- `amountAndChangeSurviveRecreation`

`withText("Taka: 688")` compares the *plain text* of the styled amount. Spans don't matter.

Step 18: Run it

1. Pick the **Pixel XL** emulator in the toolbar's device dropdown and press **Run** (the green triangle). Gradle builds the APK, installs it and launches it.
2. Type 6, 8, 8. The amount shows "Taka: 688", eight rows light up, and the caption says "8 notes".
3. **Rotate.** Use the emulator's rotate buttons (or Ctrl+Left/Right on Windows and Linux). The two-column landscape layout appears, still showing 688.
4. Try the **backspace** (tap and hold), **CLEAR**, typing a 10th digit (it shakes), and your computer's keyboard (digits, Backspace, Esc).
5. Repeat on the **Nexus 10** emulator, in both orientations.
6. Repeat steps 2 to 4 on the **API 21** Pixel XL emulator, to check the oldest Android version the app supports.
7. Optional: turn on **TalkBack** (Settings, then Accessibility) to hear the content descriptions.

Step 19: README.txt, Git and GitHub

`README.txt` is required by the assignment. It describes:

- what the app does
- the four layouts and their target devices
- where the sizes live
- the **list of tested devices and screen sizes**: the Pixel XL and Nexus 10 emulators, the Nexus 5, Pixel 7-sized phone, Nexus 7 and Pixel C screen sizes emulated on them, and Android 5.0
- how to run the tests

Git basics used in this project:

```
git init                                    # start tracking the folder
git add -A                                  # stage all changes
git commit -m "Short title" -m "Body"       # save a snapshot with a message
git remote add origin https://github.com/<user>/<repo>.git   # connect to GitHub
git push -u origin main                     # upload; -u remembers origin/main for later pushes
```

- `.gitignore` keeps generated and personal files out of the repository: `/build`, `.gradle`, `.kotlin`, `/.idea`, `*.iml`, `local.properties` (your SDK path), `/captures`, and the assignment PDF (course material).
- `.gitattributes` normalises line endings: `* text=auto`, `gradlew` always LF (Unix scripts break with CRLF), `*.bat` always CRLF, and `*.jar`/`*.png`/`*.webp` marked binary. That's why Git on Windows prints "LF will be replaced by CRLF" warnings. They are harmless.
- `README.md` is the GitHub landing page. It uses HTML inside Markdown (`<div align="center">`, `<img width="250">`) for a centred header and side-by-side screenshots, plus shields.io **badges** (`https://img.shields.io/badge/Kotlin-7F52FF?style=for-the-badge&logo=kotlin&logoColor=white`).

PART 4

Every design decision and why

#	Decision	Why	Alternatives considered
1	**XML Views + ConstraintLayout**, not Jetpack Compose	The assignment is explicitly about layouts and ConstraintLayout.	Compose is the modern toolkit (see Part 6), but it would skip the lesson.
2	**Four `activity_main.xml` files chosen by resource qualifiers** (`-land`, `-sw600dp`, `-sw600dp-land`)	Android picks the right one automatically, including after rotation. The brief says four layouts are enough.	One layout with code that moves views around: harder, and against the lesson.
3	**Every size in `sizes.xml`**, with per-configuration overrides	Required by the brief ("do not hardcode any values"). It also makes tuning a one-line change.	Hardcoding `16dp` in layouts: forbidden.
4	**Root forced to left-to-right** (`layoutDirection="ltr"`)	A calculator keypad and a number display read the same in every language. Forcing LTR also lets the inset code treat start as left.	Supporting RTL mirroring: unnecessary for a numeric UI.
5	**One Activity, no ViewModel**	The state is two small values. `onSaveInstanceState` covers rotation *and* process death. This also directly practises the "activity lifecycle" objective.	ViewModel + SavedStateHandle: better as apps grow (Part 6).
6	**Let rotation recreate the activity** (no `android:configChanges`)	Recreation is what loads the landscape layout.	`configChanges`: Android wouldn't switch layouts. Wrong for this assignment.
7	**Save the change list as well as the amount**	The brief says to keep "the currently entered amount and change list". A fallback recomputes the list if it is missing.	Saving only the amount: works, but doesn't follow the brief literally.
8	**Amount stored as a `String`**	Appending digits "from the right" is string concatenation. "Nothing typed" differs from "0". No overflow while typing.	Storing an `Int` and computing `amount * 10 + digit`: needs overflow checks and can't represent "empty".
9	**Greedy change algorithm**, proven by a test	Simple, fast, and optimal for canonical note sets like Taka. The DP test turns "I believe" into "I checked".	Dynamic programming in the app: always optimal but unnecessary here.
10	**Logic in a pure `object ChangeCalculator`**	It can be unit tested on the JVM in milliseconds, with no emulator.	Logic inside the Activity: untestable without a device.
11	**A single `setAmount` and a single `render`**	One path for every input means the UI can never be half-updated.	Updating views piecemeal in each listener: bug-prone.
12	**View Binding**	Type-safe and null-safe view access, generated by the build.	`findViewById`: verbose and crash-prone. Data Binding: heavier than needed.
13	**Material 3 components** (tonal keys, red CLEAR, outlined card, toolbar)	A modern, consistent look with little code. CLEAR uses the *tertiary* (red) role to signal "destructive".	Plain `Button`s: dated look and more styling work.
14	**Auto-sizing text plus tabular digits**	Long amounts and counts shrink instead of being cut off. Digits don't wobble while typing.	Fixed text sizes: 9-digit amounts would not fit.

#	Decision	Why	Alternatives considered
15	**9-digit limit** (originally 7)	The user asked for 9. Rendering the worst case (999,999,999, giving 1,999,999 five-hundreds) on six screen sizes showed it only failed on a 360 dp phone in portrait. Widening the change column from 36% to 40% fixed that without shrinking text too far.	No limit: counts would become unreadable. Lower minimum text size: unreadable.
16	**Backspace key in the header, beside the amount**	The brief fixes the keypad configuration ("10 digits and a clear button in the configuration shown"). Adding a grid key would break it. Phone dialers also put backspace beside the number.	A `0` `⌫` `CLEAR` bottom row: changes the required layout.
17	**Backspace: tap deletes one; hold repeats every 70 ms; hidden when empty**	This matches phone keyboards and dialers. A long click starts a self-rescheduling `Runnable`, and a touch listener cancels it on finger-up or cancel. Hidden as `INVISIBLE` so nothing shifts.	Hold to clear all (some dialers): CLEAR already does that. Accelerating repeat: possible (Part 6).
18	**Hardware keys "click" the on-screen buttons**	One code path, so haptics and visuals are identical, and keys flash so you can see the press.	Separate handling: duplicated logic.
19	**Dark-only theme**	The user preferred the dark look and asked for no theme switching. `values-night` was folded into `values`, the theme extends `Theme.Material3.Dark`, and both system bars are forced to light icons. Bonus: no white flash on launch.	DayNight (follow the system): the original behaviour, removed on request.
20	**Rows fade as one unit, with opaque "lit" colours**	Before, the background faded while the text snapped, which looked jumpy. Now all four parts share one level from 0 to 1. Opaque lit colours stop the mid-fade brightness flash.	`ValueAnimator.ofArgb` on the background only: the original approach.
21	**Entrance animation only on a fresh start**	It adds polish on launch, while rotation stays instant and the restored state appears in place.	Animate every time: rotation would feel slow.
22	**Haptics** (`KEYBOARD_TAP`, `CLOCK_TICK` for repeats, `REJECT` on Android 11+)	Physical feedback makes an on-screen keypad feel like real keys. `REJECT` isn't available before API 30, so `LONG_PRESS` is the fallback.	None: it would feel flat.
23	**Edge-to-edge with an insets listener that adds to the original paddings**	Modern Android draws behind bars (and Android 15+ requires it for apps targeting it). The header colour fills the status bar area. Adding to the original values avoids paddings growing on every insets callback.	Not edge-to-edge: a dated look, and enforced by newer targets anyway.
24	**Accessibility** (live region, row descriptions, plurals, child views hidden from TalkBack)	Screen-reader users hear "688 Taka" and "1 note of 500 Taka", not raw symbols.	Nothing: the app would be confusing with TalkBack.
25	`README.txt` ***and*** `README.md`	The `.txt` is the required deliverable. The `.md` is GitHub's landing page with screenshots.	Only one: either fails the brief or looks bare on GitHub.
26	**Minimum SDK 21, with libraries pinned to their last API 21 releases**	The brief names API 21 ("or lower if need be"), and a grader's emulator may run an old Android. The pinned versions have every feature the app uses.	`minSdk` 24 (the template default) with the newest libraries: the app can't be installed below Android 7.0.

#	Decision	Why	Alternatives considered
27	**Rows coloured with** `GradientDrawable.setColor()`	Works on every Android version. A background tint can't colour shapes on Android 5.0, where the rows stayed white.	`backgroundTintList`, used before: breaks on API 21. `DrawableCompat.wrap()` plus a tint also works there, but needs more code.
28	**Even** `0dp` **comes from** `sizes.xml` (`@dimen/match_constraint`)	The brief says not to hardcode *any* values in layouts, and a strict reading includes ConstraintLayout's `0dp`. ConstraintLayout gets the same 0 either way, so nothing changes on screen.	Writing `0dp`, as Android's docs and Android Studio do: shorter, but the one number left in the layouts.

PART 5

How the app was tested and verified

5.1 Unit tests and Espresso tests

- `./gradlew test`: **11 unit tests**, all passing.
- `./gradlew connectedAndroidTest`: **7 Espresso tests**. These need an emulator or device. All 7 pass on the Pixel XL and Nexus 10 emulators (Android 17, API 37) and on the Android 5.0 (API 21) Pixel XL emulator.

> **Android 5.0 and Gradle's test runner.** On the API 21 emulator the tests pass, and the JUnit report says so, but `connectedAndroidTest` then never finishes: Gradle's test platform keeps waiting after the run. Running the tests directly avoids that. Install both APKs (`./gradlew installDebug installDebugAndroidTest`), then run `adb shell am instrument -w com.example.vangtichai.test/androidx.test.runner.AndroidJUnitRunner`. It prints `OK (7 tests)`.

5.2 Emulator testing: the required devices, more screen sizes and Android 5.0

The assignment says to test with the emulator on at least a Pixel XL and a Nexus 10. Three emulators (AVDs) were used:

AVD	Screen	System image
Pixel XL	1440 × 2560 px at 560 dpi = 411 × 731 dp	Android 17 (API 37), Google Play
Nexus 10	2560 × 1600 px at 320 dpi = 1280 × 800 dp	Android 17 (API 37), Google Play
Pixel XL	the same	Android 5.0 (API 21), the oldest version the app supports

How each device was checked. A small script drove the emulator through `adb`, so every step used the real app:

1. Install the debug APK and launch `MainActivity`.
2. Tap the on-screen keys. `adb shell uiautomator dump` lists every view with its id and bounds, and `adb shell input tap x y` taps the middle of a key. So this is real touch input, not the hardware-keyboard path.
3. Read the screen back from the same dump: the amount text, the caption and all eight counts.
4. Rotate with `adb shell settings put system user_rotation 0` (or 1). Turning auto-rotate off first (`accelerometer_rotation 0`) makes the rotation stick. Each rotation recreates the activity, exactly as turning a real phone does.
5. Save a screenshot with `adb exec-out screencap -p > shot.png` and look at it.

What was checked in portrait and landscape on each device: 2, 3, 4 shows 2, then 23, then 234. CLEAR empties the amount and the table. 688 gives one of each note, and 6880 gives 13, 3, 1, 1, 1, 0, 0, 0, as in the brief's screenshots. Rotating portrait, landscape, portrait keeps the amount and the change list every time. Backspace deletes the last digit, and a 10th digit is refused. The phone and tablet layouts match the brief's screenshots. Half-size screenshots from these runs are in `docs/images/` (`emulator-pixel-xl-*.png` and `emulator-nexus-10-*.png`).

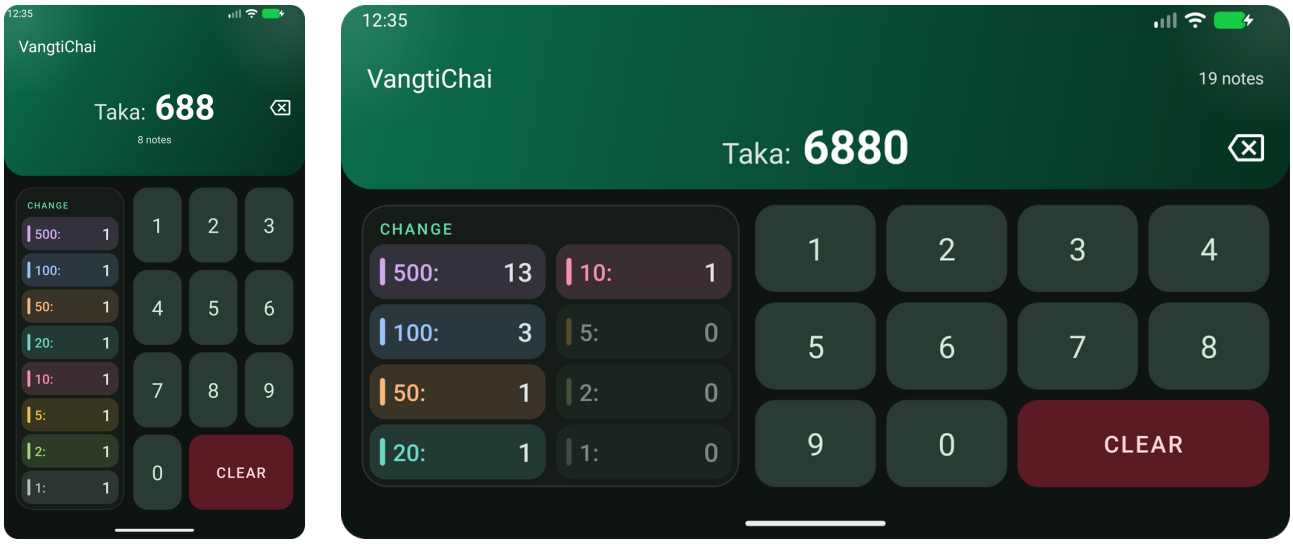


Figure 7. *The final build on the Pixel XL emulator (Android 17), one of the two devices the brief names: 688 Taka in portrait, then 6880 Taka after rotating to landscape, with the amount and the change list kept. Real emulator screenshots at half size, status bar included.*

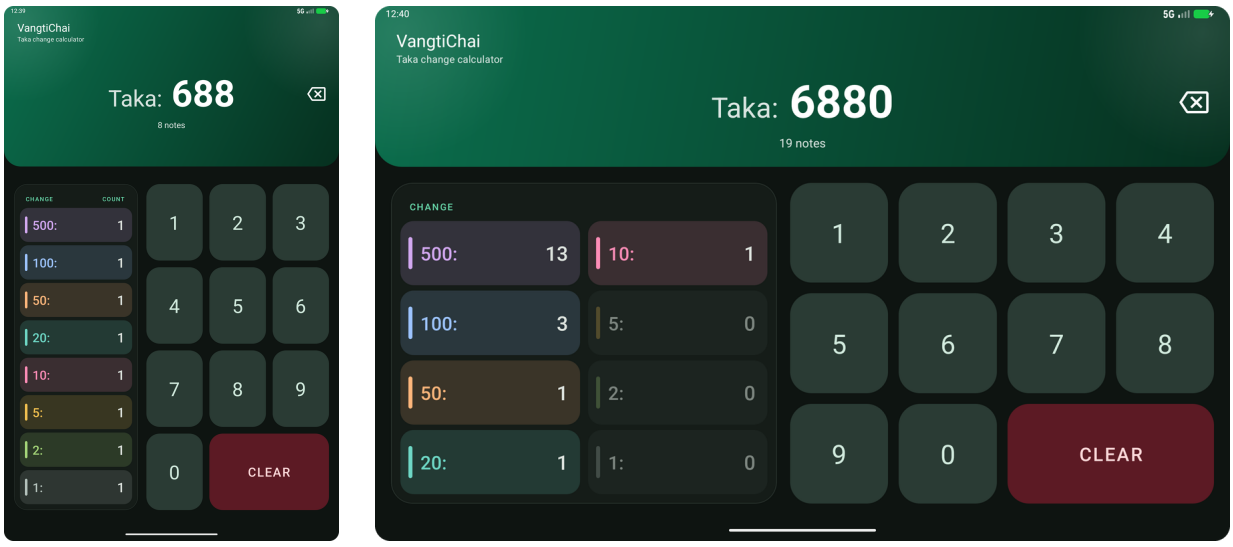


Figure 8. *The same on the Nexus 10 emulator, the other device the brief names: the tablet layouts, with the app bar's subtitle and, in portrait, the COUNT heading.*

Other screen sizes were emulated on the same two emulators. `adb shell wm size` and `wm density` change the screen Android reports, and the app picks its layouts from that, just as on a real device of that size:

Screen	Emulated on	`wm size`	`wm density`	dp
Nexus 5 (small phone)	Pixel XL	1080x1920	480	360 × 640
Medium phone (Pixel 7 size)	Pixel XL	1080x2400	420	411 × 914
Nexus 7 (7-inch tablet)	Nexus 10	1920x1200	320	960 × 600
Pixel C (large tablet)	Nexus 10	2560x1800	320	1280 × 900

On these the worst case was typed, 999999999 (1,999,999 five-hundreds), and checked in both orientations: everything stays on screen and readable. `wm size reset` and `wm density reset` put the emulator back.

What Android 5.0 found. The first run on API 21 showed two problems that the newer emulators can't show:

- **Every row of the change table was white**, so its light text was almost invisible. The rows were coloured with a background tint, and Android 5.0 can't tint shape drawables. They are now coloured with `GradientDrawable.setColor()` (Part 3, 16.10).
- **The header's two glows spread into a pale blob** in the middle of the header, because layer-list items' `width`, `height` and `gravity` only work from API 23. The glowing header moved to `drawable-v23/`, and Android 5.0 and 5.1 get a plain gradient (Part 3, Step 9).

After the fix, screenshots on the API 37 emulators were compared pixel by pixel with screenshots from before it: they are identical, so newer Android looks exactly as it did. The tablet layouts were also checked on Android 5.0, by emulating 800 × 1280 dp on the API 21 emulator. `docs/images/emulator-android-5-before.png` and `emulator-android-5-after.png` show the change table on Android 5.0 before and after the fix.

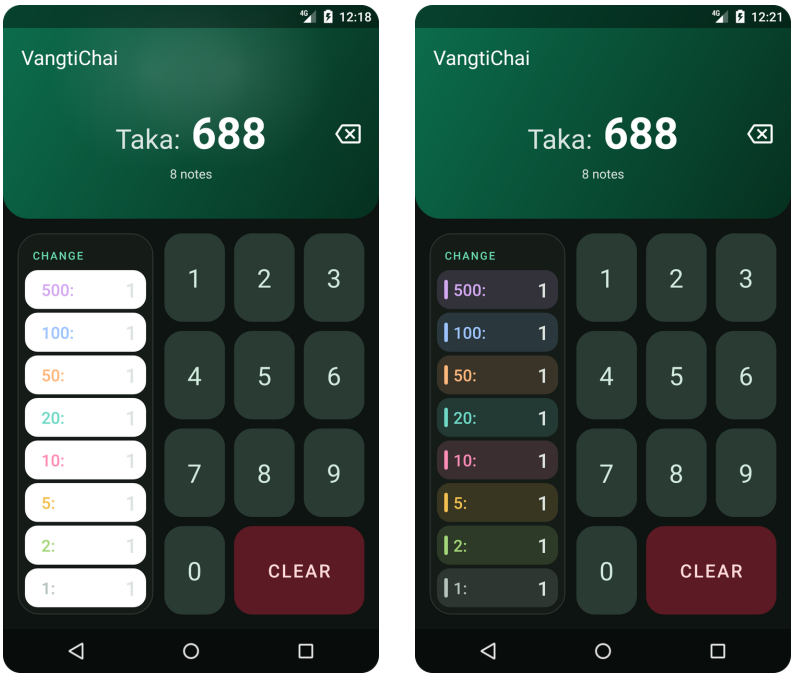


Figure 9. *Android 5.0 (API 21) on the Pixel XL emulator, 688 Taka. Left, before the fix: the rows stay white, so their light text is hard to read, and the header's glows make a pale patch near the top. Right, after it: the rows are coloured and the header is a plain gradient.*

Moving `0dp` into `sizes.xml`. When the layouts' `0dp` became `@dimen/match_constraint` (Part 2.9), the app before and after the change was installed on the same emulators, and every view's position and size was read from the accessibility tree once the animations had settled. That covered portrait and landscape on the Nexus 10 emulator, also at the Pixel XL and Nexus 5 sizes, and on Android 5.0 at the Pixel XL and Nexus 10 sizes. Every view was in exactly the same place, and the 7 Espresso tests passed on all three emulators. Screenshots alone can't prove this: a key caught at a slightly different moment of its press animation already changes a few pixels.

Pitfalls, if you script an emulator yourself:

- The Nexus 10's natural orientation is **landscape**, so `user_rotation 0` is landscape there and `1` is portrait. On the Pixel XL it is the other way round.
- On the Android 5.0 image `/sdcard` isn't writable, and `uiautomator dump /sdcard/ui.xml` silently writes nothing. Use `/data/local/tmp/ui.xml`.
- On Android 5.0, `screencap` saves landscape screens unrotated, so the picture comes out sideways. The layout itself is fine.
- Git Bash on Windows rewrites arguments that look like paths (`/sdcard/...` becomes `C:/Program Files/Git/sdcard/...`). Set `MSYS_NO_PATHCONV=1`, or call `adb` from a script.

5.3 The Robolectric "render harness" (how the README screenshots were made)

Before emulators were set up, the app was developed and checked with a **copy of the project in a scratch folder** that had extra test tooling. The README's screenshots still come from it. This harness is *not* in the repository. Recreating it is a recommended improvement (Part 6).

Robolectric runs Android code on your computer's JVM by simulating the Android framework. With `@GraphicsMode(GraphicsMode.Mode.NATIVE)` it really draws views with Android's graphics engine, so you can save pixel-accurate PNGs.

Setup, added to the copy's `app/build.gradle.kts`:

```
android {
    testOptions {
        unitTests {
            isIncludeAndroidResources = true          // let tests load layouts, styles and strings
            all {                                     // JVM flags Robolectric needed on JDK 25
                it.jvmArgs(
                    "--add-exports=java.base/jdk.internal.access=ALL-UNNAMED",
                    "--add-opens=java.base/jdk.internal.access=ALL-UNNAMED",
                    "--add-opens=java.base/java.lang=ALL-UNNAMED",
                    "--add-opens=java.base/java.io=ALL-UNNAMED",
                )
            }
        }
    }
}
dependencies {
    testImplementation("org.robolectric:robolectric:4.17")
    testImplementation("androidx.test:core-ktx:1.7.0")
    testImplementation("androidx.test.ext:junit-ktx:1.3.0")
    testImplementation("androidx.test.espresso:espresso-core:3.7.0")
}
```

How a render works:

1. Set the device with Robolectric qualifiers. For example, `RuntimeEnvironment.setQualifiers("sw411dp-w411dp-h731dp-port-notnight-xhdpi")` gives a Pixel XL-sized screen in portrait. The `xhdpi` part renders it at 2 px per dp (a real Pixel XL uses 3.5), which is why the phone screenshots are 822 px wide.
2. Launch the activity with `Robolectric.buildActivity(MainActivity::class.java).setup()`.
3. Type by calling `performClick()` on key views, then let time pass with `shadowOf(Looper.getMainLooper()).idleFor(Duration.ofSeconds(2))`.
4. Dispatch fake system-bar insets, measure and lay out the window's decor view at the device's pixel size, draw it into a `Bitmap`, and save a PNG.

What was checked this way:

- **All 6 screen sizes × 2 orientations**, with normal amounts (6, 68, 688, 6880) and the worst case (999999999). The **fit check** compared each text's needed width (`Layout.getDesiredWidth(text, paint)`) with its available width, and printed the final auto-sized text size. This is how the 9-digit problem on the 360 dp phone was found and fixed.
- **The Espresso tests** (`MainActivityTest`) also run unchanged under Robolectric. `AndroidJUnit4` picks Robolectric on the JVM.
- **Backspace timing:** real `MotionEvent`s (DOWN, then UP). A quick tap deletes exactly one digit. Holding deletes nothing for 300 ms, deletes one at the long press (+400 ms), and another 70 ms later. A finger lift queued at +520 ms with `Handler.postAtTime` stops it at exactly two extra deletions. Hardware DEL deletes

one. Holding until empty stops there and hides the key.

- **Animations:**
  - The entrance starts at alpha 0 and 24 dp low right after `create()`, and ends at alpha 1 and 0 offset.
  - It does not replay after `recreate()`.
  - For the row fade, the test drives the row's `ValueAnimator` to 110 ms with `currentPlayTime` and checks that all four colours are in between their idle and lit values, and that on-screen brightness rises steadily.

Pitfalls found (useful if you build a harness):

- Robolectric's fake clock can **jump ahead** while animation frames run. In one case, a 40 ms idle advanced the clock by over 700 ms. So don't check animation progress by sampling at a time. Instead check the start state, the end state, or drive the animator directly.
- `ViewConfiguration.getLongPressTimeout()` reported 500 ms, but views used 400 ms. So test the *sequence* of events, not exact timings.

README screenshots came from these renders. A tiny Java program (`javax.imageio`) cropped off the simulated navigation-bar strip, rounded the corners, and cut the round launcher icon out of an icon render to make `logo.png`.

5.4 Devices covered

Device	Size (dp)	Layout used	Tested on
Pixel XL (required)	411 × 731	`layout`, `layout-land`	Pixel XL emulator, Android 17 and Android 5.0
Nexus 10 (required)	800 × 1280	`layout-sw600dp`, `layout-sw600dp-land`	Nexus 10 emulator, Android 17; also emulated on Android 5.0
Nexus 5 (small phone)	360 × 640	`layout`, `layout-land`	emulated on the Pixel XL emulator
Medium phone (Pixel 7 size)	411 × 914	`layout`, `layout-land`	emulated on the Pixel XL emulator
Nexus 7 (7-inch tablet)	600 × 960	`layout-sw600dp`, `layout-sw600dp-land`	emulated on the Nexus 10 emulator
Pixel C (large tablet)	900 × 1280	`layout-sw600dp`, `layout-sw600dp-land`	emulated on the Nexus 10 emulator

PART 6

Better ways and possible improvements

These are honest trade-offs. The current choices fit the assignment, but here is what you would consider in a bigger or longer-lived app.

Architecture

- **ViewModel + `SavedStateHandle` + `StateFlow`.** Move `amount` and `change` into a `ViewModel`, which survives rotation without a Bundle round-trip. Expose the state as a `StateFlow<UiState>` and collect it in the Activity. It is better for testing and for larger apps. For this two-value screen it is more code than benefit, and the brief wants you to practise the lifecycle directly.
- **Jetpack Compose.** Android's modern declarative UI toolkit. The whole screen would be maybe 150 lines of Kotlin with no XML. State-driven rendering (what `render()` does by hand) is built in, and animations like `animateColorAsState` make the row fades one-liners. It is not used because the assignment is about XML layouts and ConstraintLayout.

Layout

- **ConstraintLayout's `Flow` helper** (`androidx.constraintlayout.helper.widget.Flow`) can lay out a grid of keys from a list of ids with much less XML. The two-column-wide CLEAR key makes it awkward, which is why explicit chains were used.
- **Window size classes** (`androidx.window`) describe the screen as compact, medium or expanded. They handle foldables and split-screen more precisely than `sw600dp`. Resource qualifiers are simpler and enough here.
- **MotionLayout** (a ConstraintLayout subclass) can animate between two constraint sets, for example a fancy portrait-to-landscape transition. It is overkill here.

Polish and features

- **Number formatting.** Show 999,999,999 as `99,99,99,999` (the Indian/Bangladeshi lakh-crore grouping) using `NumberFormat` with a `bn-BD` or `en-IN` locale. Optionally use Bangla digits (০১২৩...). Remember to re-check widths, because separators make text longer.
- **Bangla translation.** Add `values-bn/strings.xml`. All text is already in resources.
- **Spring animations** (`androidx.dynamicanimation`'s `SpringAnimation`) for the pops. They are physics-based and interrupt gracefully.
- **Accelerating backspace repeat**: start at around 100 ms, speed up to around 40 ms after a second. Or add "hold for 2 s to clear".
- **SplashScreen API** (`androidx.core:core-splashscreen`) for a branded launch screen on every Android version.
- **Bitmap launcher icons.** Regenerate the `mipmap-*dpi` bitmaps from the adaptive icon (Android Studio's Image Asset tool can do this), so Android 5.0 to 7.1 show the VangtiChai icon instead of the template's robot.
- **Material You dynamic colour** (`DynamicColors.applyToActivitiesIfAvailable`) would follow the user's wallpaper. It was deliberately not used, to keep the Taka-green brand.

Quality and tooling

- **Keep the Robolectric tests in the repository.** Then the UI tests run with `./gradlew test`, with no emulator needed.
- **Gradle Managed Devices** (`testOptions { managedDevices { ... } }`) let Gradle create the Pixel XL and Nexus 10 emulators itself and run the Espresso tests on both with one command. That would replace the hand-made AVDs and the adb script from 5.2.
- **Screenshot testing** with **Roborazzi** or **Paparazzi** records reference images and fails if the UI changes unexpectedly. This is what the render harness did manually.
- **CI** (GitHub Actions) can run `./gradlew test lint` on every push.
- **Static analysis**: Android Lint (built in: `./gradlew lint`), and **ktlint** or **detekt** for Kotlin style.
- **Large font sizes:** test with the system font scale at 1.5× or 2× (Settings, then Display). Auto-sizing helps, but the fixed-height amount box should be checked.

Things that would be *worse* (avoid)

- `android:configChanges="orientation|screenSize"`: stops the landscape layout loading.
- Hardcoded sizes in layouts: breaks the assignment rule and makes tuning painful.
- Storing the amount as `Int`: overflows past about 2.1 billion and can't represent "nothing typed".
- Nested `LinearLayout`s with weights: deeper view trees, slower measurement, and harder to keep the keypad aligned with the table.
- Adding the backspace key into the keypad grid: violates the brief's keypad configuration.

PART 7

Project history

Commit	Date	What changed	Why
`260cefb`	2026-10-13	**Initial app.** Keypad, change table, four layouts, sizes per configuration, state saving, Material 3 (light and dark), colours per note, animations, haptics, hardware keys, a 7-digit limit, accessibility, launcher icon, unit and Espresso tests, `README.txt`.	The assignment.
`7447373`	2026-10-14	**README.md with screenshots** (`docs/images/`).	An attractive GitHub page.
`10b7e55`	2026-10-14	**Backspace key** in the header. Tap deletes one digit, hold repeats, it is hidden when empty, and hardware DEL routes through it. New `ChangeCalculator.removeLastDigit` and tests.	Requested: "backspace like mobile phones". Placed in the header to keep the brief's keypad configuration.
`e9f35bc`	2026-10-14	README clone URL updated.	The GitHub repository was renamed to `VhangtiChai_AndroidApp_CSE489`.
`87347ba`	2026-10-14	**9-digit limit** (was 7). Change column widened from 36% to 40% in portrait.	Requested. The widening keeps the largest count readable on a 360 dp phone.
`48ad475`	2026-10-14	**Dark-only theme. Smoother motion:** rows fade as one unit with opaque lit colours, plus an entrance animation. Light bar icons forced.	Requested: "dark theme default, no theme switching, smooth UI".
`bcb849f`	2026-10-15	**Minimum SDK 21 and emulator testing.** `minSdk` 24 → 21, with core-ktx 1.17.0, appcompat 1.7.1, activity-ktx 1.11.0 and material 1.13.0, the last releases that support API 21. Rows are coloured with `GradientDrawable.setColor()` instead of a background tint, and the header's glows moved to `drawable-v23/`. Tested on Pixel XL, Nexus 10 and Android 5.0 emulators; `README.txt` lists the results.	A check of the brief: it suggests API 21 and asks for testing *with the emulator* on a Pixel XL and a Nexus 10. Android 5.0 showed white rows and a blurred header, which the code changes fix.
`4e51a2f`	2026-10-15	**No `0dp` in the layouts.** All 192 of ConstraintLayout's `0dp` widths and heights became `@dimen/match_constraint`, defined once in `values/sizes.xml`.	The brief says not to hardcode any values in layout files, and `0dp` was the last number in them. Every view's position and size was checked to be unchanged on the emulators.

IN DETAIL

After the guide was written

The table lists the changes to the app. Between `48ad475` and the row after it, these commits changed documentation only:

Commit	Date	What changed
`c890f16`	2026-10-14	Add a complete project guide
`d33b50c`	2026-10-14	Correct pixel density and launcher icon details in the guide
`9551e4e`	2026-10-14	Add a PDF edition of the project guide

They added this guide and its PDF edition. The rows after them changed the app again, and the guide and this PDF describe the app with those changes in place.

PART 8

Glossary

Term	Meaning
Activity	One screen of an Android app, with a lifecycle (`onCreate`, `onResume`, ...).
AGP	Android Gradle Plugin: teaches Gradle to build Android apps.
APK / AAB	The installable app package / the Play Store upload format.
ART	Android Runtime: runs the app's compiled code on the device.
Bundle	A key-value container Android saves and restores across recreation.
Chain	ConstraintLayout views linked to each other in both directions, sharing space.
Composite (colour)	The colour you actually see when a translucent colour is drawn over another.
Configuration change	A change such as rotation, language or dark mode. By default the Activity is recreated.
Constraint	A rule attaching a view's edge to another edge, in ConstraintLayout.
Content description	Text a screen reader speaks for a view.
dp / sp / px	Density-independent pixel / scale-independent pixel (text) / real pixel.
Drawable	Anything that can be drawn: a shape, gradient, vector or bitmap.
Edge-to-edge	The app draws behind the status and navigation bars.
Espresso	Android's UI testing library (find view, perform action, check result).
Gradle / wrapper	The build system / a script that downloads the exact Gradle version.
Guideline	An invisible line in ConstraintLayout, for example at 40% of the width.
Haptic feedback	A small vibration confirming a touch.
Inflate	Turn layout XML into real View objects.
Insets	The sizes of system UI (bars, cutouts) overlapping the app.
Interpolator	A curve that shapes an animation's speed (decelerate, overshoot, ...).
JVM	Java Virtual Machine: runs Kotlin/Java bytecode on a computer (unit tests, Gradle).
KDoc	Kotlin documentation comments, `/** ... */`.
Lambda	A function written inline: `{ x -> x * 2 }`.
Material 3	Google's current design system and component library.
mipmap	The resource folder type used for launcher icons.
Qualifier	A folder suffix (`-land`, `-sw600dp`) saying when its resources apply.
R class	The generated class holding an integer ID for every resource.
Robolectric	A library that runs Android code (even rendering) on the JVM for tests.

Term	Meaning
Runnable	A piece of code to run later (`post`, `postDelayed`).
Span	Styling applied to part of a text (size, colour, bold).
StateListAnimator	Runs animations when a view's state (pressed, ...) changes.
Theme attribute	A named value from the current theme, referenced as `?attr/name`.
ValueAnimator / ObjectAnimator / ViewPropertyAnimator	Animate a number / animate an object's property / the simple `view.animate()` API.
Version catalog	`libs.versions.toml`: one place for all library versions.
View / ViewGroup	A thing on screen / a container that positions other views.
View Binding	Generated classes giving type-safe access to a layout's views.

APPENDIX

Complete source code

Every text file of the project, copied verbatim from the project as it was on the date at the top of this guide.

These are **not** listed here:

- Binary files: the launcher icon bitmaps `app/src/main/res/mipmap-*dpi/*.webp`, `gradle/wrapper/gradle-wrapper.jar` and the screenshots in `docs/images/`.
- The `gradlew` / `gradlew.bat` scripts, which Gradle generates.
- `README.md`, the GitHub page.

A.1 Build configuration

`settings.gradle.kts`

26 lines · Kotlin script

ABOUT THIS FILE

The first file Gradle reads. It decides which modules belong to the build and where plugins and libraries are downloaded from.

- **Lines 1–13,** `pluginManagement`: repositories for Gradle *plugins* such as AGP. The `content { includeGroupByRegex(...) }` filter on lines 4–8 means Google's repository is only asked for groups matching `com.android.*`, `com.google.*` and `androidx.*`. Everything else is looked up on Maven Central or the Gradle Plugin Portal, which saves pointless network requests.
- **Lines 14–16**: the foojay toolchain resolver, which lets Gradle find and download JDKs ("toolchains") from the foojay Disco API. The download links in `gradle/gradle-daemon-jvm.properties` point at the same service.
- **Lines 17–23,** `dependencyResolutionManagement`: repositories for *libraries*. `FAIL_ON_PROJECT_REPOS` (line 18) makes the build fail if a module declares repositories of its own, so this block stays the only list.
- **Lines 25–26**: the project's name, and its only module, `:app` (the `app/` folder).

`settings.gradle.kts`

```
1  pluginManagement {
2      repositories {
3          google {
4              content {
5                  includeGroupByRegex("com\\.android.*")
6                  includeGroupByRegex("com\\.google.*")
7                  includeGroupByRegex("androidx.*")
8              }
9          }
10         mavenCentral()
11         gradlePluginPortal()
12     }
13 }
14 plugins {
15     id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
16 }
17 dependencyResolutionManagement {
18     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
19     repositories {
```

settings.gradle.kts (continued)

```
        google()
        mavenCentral()
    }
}

rootProject.name = "VangtiChai"
include(":app")
```

build.gradle.kts

4 lines · Kotlin script

ABOUT THIS FILE

The root build file. It declares the Android application plugin with `apply false`: the plugin and its version are resolved once for the whole build, but only applied by the modules that ask for it (`app/build.gradle.kts`, line 2). In a project with several modules, this keeps all of them on the same AGP version.

`libs.plugins.android.application` is an accessor Gradle generates from the `[plugins]` section of the version catalog.

build.gradle.kts

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
}
```

app/build.gradle.kts

47 lines · Kotlin script

ABOUT THIS FILE

The module's build file, explained line by line in Part 2.4. Where things are:

Lines	Block	What it controls
1–3	`plugins`	Applies AGP, making this module an Android app. AGP 9 compiles Kotlin itself, so no Kotlin plugin is listed.
6	`namespace`	Package of the generated `R` class and the View Binding classes, such as `com.example.vangtichai.databinding.ActivityMainBinding`.
7–9	`compileSdk`	Compile against API 37 (the newer DSL form of `compileSdk = 37`).
11–19	`defaultConfig`	App ID, `minSdk` 21 (the level the assignment suggests), `targetSdk` 37, version 1 ("1.0"), and the test runner for `androidTest`.
21–28	`buildTypes.release`	The template's `optimization` block, which turns R8 on for release builds. Debug builds are left alone.
29–32	`compileOptions`	Java 11 language level for the compiled bytecode.
33–35	`buildFeatures`	Turns on View Binding.
38–47	`dependencies`	Five libraries that ship in the app, one for unit tests and two for instrumented tests, all named through the version catalog.

`namespace` and `applicationId` hold the same text here but are separate settings. The first names generated classes; the second identifies the installed app. You could change the app ID (for example for a "free" and a "pro" edition) without renaming any code.

app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.example.vangtichai"
    compileSdk {
        version = release(37)
    }

    defaultConfig {
        applicationId = "com.example.vangtichai"
        minSdk = 21 // API 21 (Android 5.0), the minimum the assignment suggests
        targetSdk = 37
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            optimization {
                enable = true
                packageScope = setOf("androidx.**", "kotlin.**", "kotlinx.**")
            }
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    buildFeatures {
        viewBinding = true
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.androidx.activity.ktx)
    implementation(libs.androidx.constraintlayout)
    implementation(libs.material)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
}
```

gradle/libs.versions.toml

24 lines · TOML

ABOUT THIS FILE

The version catalog (Part 2.4), in three sections: `[versions]` (lines 1–11) names one version per library family, `[libraries]` (lines 13–21) gives each library's Maven coordinates and the version it uses, and `[plugins]` (lines 23–24) names AGP. Gradle turns each alias into an accessor, with dashes becoming dots: `androidx-core-ktx` is used as `libs.androidx.core.ktx`.

Alias	Maven coordinates	Version	Used for
`androidx-core-ktx`	androidx.core:core-ktx	1.17.0	Kotlin extensions such as `buildSpannedString`, `isVisible` and `updatePadding`; it also brings in `androidx.core` (`ContextCompat`, `ColorUtils`, `ViewCompat`)
`androidx-appcompat`	androidx.appcompat:appcompat	1.7.1	`AppCompatActivity`
`androidx-activity-ktx`	androidx.activity:activity-ktx	1.11.0	`enableEdgeToEdge()`
`androidx-constraintlayout`	androidx.constraintlayout:constraintlayout	2.2.2	`ConstraintLayout` and `Guideline`
`material`	com.google.android.material:material	1.13.0	Material 3 components and themes
`junit`	junit:junit	4.13.2	JVM unit tests
`androidx-junit`	androidx.test.ext:junit	1.3.0	`AndroidJUnit4`, `ActivityScenarioRule`
`androidx-espresso-core`	androidx.test.espresso:espresso-core	3.7.0	Espresso UI tests
`android-application` (plugin)	com.android.application	9.4.1	AGP

Line 3 explains the four versions under it. They are the last releases of `core-ktx`, `appcompat`, `activity-ktx` and `material` that support API 21, the app's `minSdk`. Their next releases need API 23, so updating them would break the build (Part 0, rule 9).

Mind the two JUnit names: `junit` is JUnit 4 itself, while `junitVersion` (line 10) is the version of AndroidX's JUnit *extensions*. Both names come from the project template.

gradle/libs.versions.toml

```
1  [versions]
2  agp = "9.4.1"
3  # Newest releases that still support minSdk 21 (later ones require API 23)
4  coreKtx = "1.17.0"
5  appcompat = "1.7.1"
6  activityKtx = "1.11.0"
7  material = "1.13.0"
8  constraintlayout = "2.2.2"
9  junit = "4.13.2"
10 junitVersion = "1.3.0"
11 espressoCore = "3.7.0"
12
13 [libraries]
14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
15 androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
16 androidx-activity-ktx = { group = "androidx.activity", name = "activity-ktx", version.ref =
   ↪     "activityKtx" }
17 androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout",
   ↪     version.ref = "constraintlayout" }
18 material = { group = "com.google.android.material", name = "material", version.ref = "material" }
19 junit = { group = "junit", name = "junit", version.ref = "junit" }
20 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
21 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
   ↪     "espressoCore" }
22
23 [plugins]
24 android-application = { id = "com.android.application", version.ref = "agp" }
```

gradle.properties

19 lines · Properties

ABOUT THIS FILE

Settings for Gradle itself, not for the app. Every line starting with `#` is a comment from the template; four lines do the work:

- **Line 9**: the Gradle daemon (the background process that runs builds) may use up to 2 GB of memory (`-Xmx2048m`), and reads and writes files as UTF-8 (`-Dfile.encoding=UTF-8`) whatever the operating system's default is.
- **Line 13**: parallel builds stay off (the line is commented out). With a single module there is nothing to run in parallel.
- **Line 17**: the configuration cache. Gradle saves the result of its configuration phase and reuses it while the build scripts are unchanged, so later builds start faster.
- **Line 19**: Android Studio's formatter follows the official Kotlin coding conventions.

gradle.properties

```
# Project-wide Gradle settings.
# IDE (e.g. Android Studio) users:
# Gradle settings configured through the IDE *will override*
# any settings specified in this file.
# For more details on how to configure your build environment visit
# http://www.gradle.org/docs/current/userguide/build_environment.html
# Specifies the JVM arguments used for the daemon process.
# The setting is particularly useful for tweaking memory settings.
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
# When configured, Gradle will run in incubating parallel mode.
# This option should only be used with decoupled projects. For more details, visit
# https://developer.android.com/r/tools/gradle-multi-project-decoupled-projects
# org.gradle.parallel=true
# When enabled, the Configuration Cache allows Gradle to skip the configuration
# phase entirely if nothing that affects the build configuration (such as build scripts)
# has changed. Additionally, Gradle applies performance optimizations to task execution.
org.gradle.configuration-cache=true
# Kotlin code style for this project: "official" or "obsolete":
kotlin.code.style=official
```

gradle/wrapper/gradle-wrapper.properties

9 lines · Properties

ABOUT THIS FILE

Pins the Gradle version for everyone who builds the project (Part 2.4, "The wrapper").

- **Line 5,** `distributionUrl`: Gradle 9.6.0, the `bin` (binaries only) distribution. In `.properties` files a colon is written `\:`, which is why the URL reads `https\://`.
- **Line 4,** `distributionSha256Sum`: the SHA-256 checksum of that zip. The wrapper checks the download against it and refuses a file that does not match.
- **Lines 2–3 and 8–9**: where the download and the unpacked Gradle are kept: under `GRADLE_USER_HOME` (normally `~/.gradle`), in `wrapper/dists`.
- **Lines 6–7**: a 10-second network timeout, and a check that the distribution URL is valid.
- **Line 1**: a timestamp written when the file was generated (13 October 2026, at GMT+6).

gradle/wrapper/gradle-wrapper.properties

```
#Tue Oct 13 20:48:45 GMT+06:00 2026
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionSha256Sum=bbaeb2fef8710818cf0e261201dab964c572f92b942812df0c3620d62a529a01
distributionUrl=https\://services.gradle.org/distributions/gradle-9.6.0-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

gradle/gradle-daemon-jvm.properties

12 lines · Properties

ABOUT THIS FILE

Says which JDK the **Gradle daemon** runs on. Gradle's `updateDaemonJvm` task generates it (line 1); it is not written by hand.

- **Line 12,** `toolchainVersion=25`: the daemon needs JDK 25.
- **Lines 2–11**: a download link for each operating system and processor: FreeBSD, Linux, macOS, other Unix systems and Windows, each on ARM64 ("AARCH64") and x86-64. They point at the foojay Disco API, so a machine without JDK 25 can fetch the right one.

This JDK only runs Gradle and the compilers. The app's bytecode targets Java 11 (`compileOptions` in `app/build.gradle.kts`), and on a phone it runs on Android's own runtime, ART.

gradle/gradle-daemon-jvm.properties

```
#This file is generated by updateDaemonJvm
toolchainUrl.FREE_BSD.AARCH64=https\://api.foojay.io/disco/v3.0/ids/cf726b4a1c84b50457225f9bba6d7650/
    redirect
toolchainUrl.FREE_BSD.X86_64=https\://api.foojay.io/disco/v3.0/ids/fa1e318c287360478e3c83a9a3ef1007/
    redirect
toolchainUrl.LINUX.AARCH64=https\://api.foojay.io/disco/v3.0/ids/cf726b4a1c84b50457225f9bba6d7650/
    redirect
toolchainUrl.LINUX.X86_64=https\://api.foojay.io/disco/v3.0/ids/fa1e318c287360478e3c83a9a3ef1007/
    redirect
toolchainUrl.MAC_OS.AARCH64=https\://api.foojay.io/disco/v3.0/ids/c2dd35c9d0aaf0ba6ad0791320f99dfc/
    redirect
toolchainUrl.MAC_OS.X86_64=https\://api.foojay.io/disco/v3.0/ids/e5810bd7fd1f8a586644409d395a7e55/
    redirect
toolchainUrl.UNIX.AARCH64=https\://api.foojay.io/disco/v3.0/ids/cf726b4a1c84b50457225f9bba6d7650/
    redirect
toolchainUrl.UNIX.X86_64=https\://api.foojay.io/disco/v3.0/ids/fa1e318c287360478e3c83a9a3ef1007/redirect
toolchainUrl.WINDOWS.AARCH64=https\://api.foojay.io/disco/v3.0/ids/7b3c4877c0749019e6805bb61e421497/
    redirect
toolchainUrl.WINDOWS.X86_64=https\://api.foojay.io/disco/v3.0/ids/d76df094a9cbbabd3b08251f9e61444a/
    redirect
toolchainVersion=25
```

.gitignore

14 lines · Git configuration

ABOUT THIS FILE

Files and folders Git should never commit (Part 3, Step 19):

Entry	What it is	Why it stays out
`*.iml`	IntelliJ / Android Studio module files	Generated by the IDE
`.gradle`	Gradle's caches for this project	Generated, and large
`.kotlin`	Kotlin compiler caches and session data	Generated
`/local.properties`, `local.properties`	The path to your Android SDK	Different on every computer
`/.idea`	Android Studio project settings	Personal and machine-specific
`.DS_Store`	macOS folder metadata	Not part of the project
`/build`	The root project's build output	Rebuilt on demand
`/captures`	Android Studio captures, such as profiler traces and heap dumps	Large and personal
`.externalNativeBuild`, `.cxx`	C/C++ (NDK) build folders	Template defaults; the app has no native code
`/CSE 489 Assigment L1 v2.pdf`	The assignment brief	Course material, kept out of a public repository

A pattern starting with `/` only matches at the top of the repository; without it, it matches at any depth. The module's own build folder is ignored by `app/.gitignore`.

.gitignore

```
*.iml
.gradle
.kotlin
/local.properties
/.idea
.DS_Store
/build
/captures
.externalNativeBuild
.cxx
local.properties

# Assignment brief (course material) stays out of the repository
/CSE 489 Assigment L1 v2.pdf
```

.gitattributes

6 lines · Git configuration

ABOUT THIS FILE

Tells Git how to handle line endings and binary files (Part 3, Step 19):

- **Line 1,** `* text=auto`: Git decides which files are text. Text files are stored with LF line endings in the repository, and on Windows they are checked out with CRLF. The harmless "LF will be replaced by CRLF" warnings come from this conversion.
- **Line 2**: `gradlew` is a Unix shell script and must keep LF endings, or `./gradlew` fails on macOS and Linux.
- **Line 3**: `*.bat` files are Windows batch files and always get CRLF.
- **Lines 4–6**: `.jar`, `.png` and `.webp` files are binary: never converted, never shown as text diffs.

.gitattributes

```
* text=auto
gradlew text eol=lf
*.bat text eol=crlf
*.jar binary
*.png binary
*.webp binary
```

app/.gitignore

1 lines · Git configuration

ABOUT THIS FILE

One line: `/build`, the module's build folder (`app/build/`). Everything the build generates lands there: the APK, compiled classes, the generated `R` and View Binding classes, and intermediate files. All of it is recreated by `./gradlew assembleDebug`.

app/.gitignore

```
/build
```

app/src/main/keepRules/rules.keep

12 lines · R8 keep rules

ABOUT THIS FILE

Keep rules for **R8**, the tool that shrinks and optimises release builds (switched on by the `optimization` block in `app/build.gradle.kts`). A keep rule protects code that R8 cannot see being used, such as classes that are only reached through reflection. As the comment on line 2 says, AGP passes every keep-rule file in `src/main/keepRules` to R8.

Every line here is a comment from the project template. VangtiChai needs no rules of its own: it uses no reflection, no WebView JavaScript interface and no serialisation library.

app/src/main/keepRules/rules.keep

```
# Add project specific R8 rules here.
# AGP will combine all keep rule files in src/main/keepRules to pass to R8
#
# For more details, see
#   https://d.android.com/r/tools/r8/keep-rules

# If your project uses WebView with JS, uncomment the following
# and specify the fully qualified class name to the JavaScript interface
# class:
#-keepclassmembers class fqcn.of.javascript.interface.for.webview {
#   public *;
#}
```

A.2 Manifest

app/src/main/**AndroidManifest.xml** 26 lines · XML

ABOUT THIS FILE

Explained attribute by attribute in Part 2.6. Where things are:

- **Lines 2–3**: the root `<manifest>` and its XML namespaces. (The template declares `tools` but the file does not use it.)
- **Lines 5–13,** `<application>`: the backup rules, launcher icons, app name, right-to-left support and the theme `Theme.VangtiChai`.
- **Line 14**: the comment recording why there is no `android:configChanges` (Part 2.10).
- **Lines 15–23,** `<activity>`: the app's one screen. The `MAIN` action and the `LAUNCHER` category (lines 19 and 21) put it in the app drawer, and `exported="true"` lets the launcher, which is another app, start it.

There are no `<uses-permission>` lines: the app uses no internet, storage, camera or location. The key-press vibrations go through `View.performHapticFeedback()`, which needs no permission. Driving the `Vibrator` directly would need `VIBRATE`.

app/src/main/AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.VangtiChai">
        <!-- No configChanges: rotating recreates the activity so the landscape layout is loaded. -->
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

A.3 Kotlin source

app/src/main/java/com/example/vangtichai/**ChangeCalculator.kt** 49 lines · Kotlin

ABOUT THIS FILE

The app's logic, explained in Part 3, Step 11. It has no Android imports, so the unit tests in A.4 run on any computer's JVM.

Lines	Member	Notes
6	`object ChangeCalculator`	A singleton. No instance is ever created; calls look like `ChangeCalculator.breakdown(688)`.
9	`NOTES`	The eight notes, largest first. The table rows, the colours and the counts all share this index order.
15	`MAX_DIGITS = 9`	`const` makes it a compile-time constant: the value 9 is written straight into the code that uses it.
21–29	`breakdown`	`require` rejects negative amounts. `LongArray(size) { i -> ... }` calls the lambda once per index, in order from 0 to 7, which is what lets `remaining` shrink from one note to the next.
35–42	`appendDigit`	A `when` without a subject tries its conditions from top to bottom. The lone-zero rule comes first, so "0" then 5 gives "5".
45	`removeLastDigit`	`dropLast(1)` of an empty string is still empty, so a backspace with nothing typed does no harm.
48	`valueOf`	`toLongOrNull()` returns null for "", and the Elvis operator `?:` turns that into 0.

`breakdown` always does exactly eight divisions, however large the amount. Even 999,999,999 is tiny next to the largest `Long`, 9,223,372,036,854,775,807.

app/src/main/java/com/example/vangtichai/ChangeCalculator.kt

```kotlin
package com.example.vangtichai

/**
 * Pure (Android-free) logic behind VangtiChai, kept separate from the UI so it can be unit tested.
 */
object ChangeCalculator {

    /** Taka notes the change is given in, largest first. */
    val NOTES = intArrayOf(500, 100, 50, 20, 10, 5, 2, 1)

    /**
     * Longest amount (in digits) the keypad accepts: up to 999,999,999 Taka. The largest count
     * (1,999,999 five-hundreds) still shows in full on small phones; the text shrinks to fit.
     */
    const val MAX_DIGITS = 9

    /**
     * Returns how many of each note in [NOTES] make up [amount], using as few notes as possible.
     * The Taka note system is canonical, so taking the largest note that fits each time is optimal.
     */
    fun breakdown(amount: Long): LongArray {
        require(amount >= 0) { "amount must not be negative: $amount" }
        var remaining = amount
        return LongArray(NOTES.size) { i ->
            val count = remaining / NOTES[i]
            remaining %= NOTES[i]
            count
        }
    }

```

app/src/main/java/com/example/vangtichai/ChangeCalculator.kt (continued)

```
31      /**
32       * Adds [digit] to the right of the typed [amount] ("23" + 4 -> "234").
33       * A lone leading zero is replaced ("0" + 5 -> "5"). Returns null when [MAX_DIGITS] is reached.
34       */
35      fun appendDigit(amount: String, digit: Int): String? {
36          require(digit in 0..9) { "not a digit: $digit" }
37          return when {
38              amount == "0" -> digit.toString()
39              amount.length >= MAX_DIGITS -> null
40              else -> amount + digit
41          }
42      }
43
44      /** Removes the rightmost digit of the typed [amount] ("234" -> "23"). Nothing typed stays empty. */
45      fun removeLastDigit(amount: String): String = amount.dropLast(1)
46
47      /** Numeric value of the typed digits; nothing typed counts as zero. */
48      fun valueOf(amount: String): Long = amount.toLongOrNull() ?: 0L
49  }
```

app/src/main/java/com/example/vangtichai/**MainActivity.kt** 414 lines · Kotlin

ABOUT THIS FILE

All of the screen's behaviour, explained section by section in Part 3, Step 16. Use this map to find each part in the listing:

Lines	What is there	Guide section
3–34	Imports	
36–43	Class KDoc and declaration	16.1
45–61	Views and state: `binding`, `noteRows`, `rowShapes`, `accentShapes`, `digitKeys`, `amount`, `change`	16.1, 16.2
63–81	Colours, resolved once with `by lazy`	16.2
83–98	Row fade bookkeeping, `restoreCaption` and `repeatBackspace`	16.2
100–158	`onCreate`	16.3
160–164	`onSaveInstanceState`	16.4
166–178	`onKeyDown`	16.5
180–199	`onDigit`, `onClear`, `onBackspace`, `setAmount`	16.6, 16.1
201–219	`rejectDigit`	16.7
221–238	`render`	16.8
240–258	`renderBackspace`	16.9
260–307	`renderRow`, `caption`, `lightRow`, `setRowLevel`	16.8, 16.10
309–343	`applySystemBarInsets`	16.11
345–359	`playEntrance`	16.12
361–380	`pop` and `flashPressed`	16.13
382–413	`companion object` constants	16.14

One key press, followed through the code. Say "68" is on screen and the user taps 6:

1. While the finger is down the key shrinks to 93% (`key_press.xml`, A.7), and it springs back on release. Android then calls the click listener set on lines 123–126.

2. The listener plays `KEYBOARD_TAP` haptics and calls `onDigit(6)` (line 180).
3. `ChangeCalculator.appendDigit("68", 6)` returns "686". That is not null, because the amount is far below 9 digits, so `setAmount("686")` runs (line 195).
4. `setAmount` stores the amount, works out `breakdown(686)` = [1, 1, 1, 1, 1, 1, 0, 1] and calls `render(animate = true)`.
5. `render` (line 221) writes "Taka: 686" with its spans, sets the spoken description "686 Taka" and the caption "7 notes", and calls `renderRow` for all eight rows. The backspace key is already showing, so `renderBackspace` returns straight away (line 245). Finally the amount pops to 106%.
6. Rows 500, 100 and 20 go from 0 notes to 1: they light up over 220 ms and their counts pop. Row 2 goes from 1 to 0: it fades back and its count pops. Rows 50, 10, 5 and 1 still hold 1 note, so nothing about them changes.
7. While the fades run, each row's `ValueAnimator` calls `setRowLevel` on every frame, which blends the row's four colours for the current level.

All of it runs on the main thread (Part 2.1).

app/src/main/java/com/example/vangtichai/MainActivity.kt

```kotlin
package com.example.vangtichai

import android.animation.ObjectAnimator
import android.animation.ValueAnimator
import android.graphics.Color
import android.graphics.drawable.GradientDrawable
import android.os.Build
import android.os.Bundle
import android.view.HapticFeedbackConstants
import android.view.KeyEvent
import android.view.MotionEvent
import android.view.View
import android.view.ViewGroup
import android.view.animation.DecelerateInterpolator
import android.view.animation.OvershootInterpolator
import androidx.activity.SystemBarStyle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import androidx.core.graphics.ColorUtils
import androidx.core.text.bold
import androidx.core.text.buildSpannedString
import androidx.core.text.color
import androidx.core.text.scale
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import androidx.core.view.isInvisible
import androidx.core.view.isVisible
import androidx.core.view.updateLayoutParams
import androidx.core.view.updatePadding
import com.example.vangtichai.databinding.ActivityMainBinding
import com.example.vangtichai.databinding.ViewNoteRowBinding
import com.google.android.material.button.MaterialButton
import com.google.android.material.color.MaterialColors

/**
 * Lets the user type an amount on a keypad and shows which Taka notes make up the change.
 *
 * Android picks one of four layouts (phone/tablet x portrait/landscape) from the resource
 * qualifiers. Rotating recreates the activity with the other layout, so the typed amount and
 * the change list are saved in [onSaveInstanceState] and restored in [onCreate].
 */
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

```

app/src/main/java/com/example/vangtichai/MainActivity.kt (continued)

```
    /** Change table rows, in the same order as [ChangeCalculator.NOTES]. */
    private lateinit var noteRows: List<ViewNoteRowBinding>

    /** Each row's background and colour tab shapes, in the same order as [noteRows]. */
    private lateinit var rowShapes: List<GradientDrawable>
    private lateinit var accentShapes: List<GradientDrawable>

    /** Digit keys, indexed by their digit. */
    private lateinit var digitKeys: List<MaterialButton>

    /** Digits typed so far, e.g. "688". Empty until the first key press. */
    private var amount = ""

    /** How many of each note in [ChangeCalculator.NOTES] make up [amount]. */
    private var change = LongArray(ChangeCalculator.NOTES.size)

    private val noteColors by lazy { NOTE_COLORS.map { ContextCompat.getColor(this, it) } }
    private val idleRowColor by lazy { ContextCompat.getColor(this, R.color.note_row_idle) }

    /**
     * Lit row backgrounds: each note's colour, faintly, over the card. They are kept opaque so that a
     * fade between idle and lit goes straight from one to the other, never through a brighter colour.
     */
    private val litRowColors by lazy {
        val card = ContextCompat.getColor(this, R.color.card_surface)
        noteColors.map { ColorUtils.compositeColors(ColorUtils.setAlphaComponent(it,
            ACTIVE_ROW_ALPHA), card) }
    }
    private val labelColor by lazy { ContextCompat.getColor(this, R.color.header_content_secondary) }
    private val strongTextColor by lazy {
        MaterialColors.getColor(binding.root, com.google.android.material.R.attr.colorOnSurface)
    }
    private val mutedTextColor by lazy {
        val color = MaterialColors.getColor(binding.root,
            com.google.android.material.R.attr.colorOnSurfaceVariant)
        ColorUtils.setAlphaComponent(color, MUTED_TEXT_ALPHA)
    }

    /** How lit up each row is on screen (0 = faded back, 1 = lit up), and the animations moving it. */
    private val rowLevels = FloatArray(ChangeCalculator.NOTES.size)
    private val rowAnimators = arrayOfNulls<ValueAnimator>(ChangeCalculator.NOTES.size)

    private val restoreCaption = Runnable { binding.captionText.text = caption() }

    /** While the backspace key is held, deletes a digit every [BACKSPACE_REPEAT_INTERVAL] ms. */
    private val repeatBackspace = object : Runnable {
        override fun run() {
            if (amount.isEmpty()) return
            val key = binding.keyBackspace
            key.performHapticFeedback(HapticFeedbackConstants.CLOCK_TICK)
            onBackspace()
            key.postDelayed(this, BACKSPACE_REPEAT_INTERVAL)
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        // Draw behind the system bars. The app is always dark, so both bars keep light icons even
        // when the rest of the phone uses a light theme.
        enableEdgeToEdge(
            statusBarStyle = SystemBarStyle.dark(Color.TRANSPARENT),
            navigationBarStyle = SystemBarStyle.dark(Color.TRANSPARENT),
        )
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
        applySystemBarInsets()

        noteRows = with(binding) { listOf(note500, note100, note50, note20, note10, note5, note2,
            note1) }
        noteRows.forEachIndexed { i, row ->
```

app/src/main/java/com/example/vangtichai/MainActivity.kt (continued)

```
            row.noteLabel.text = getString(R.string.note_label, ChangeCalculator.NOTES[i])
        }
        // Rows are coloured by setting their shapes' colour, not with a background tint: Android 5.0
        // (API 21) can't tint shapes. mutate() gives every row its own copy to colour.
        rowShapes = noteRows.map { it.root.background.mutate() as GradientDrawable }
        accentShapes = noteRows.map { it.noteAccent.background.mutate() as GradientDrawable }

        digitKeys = with(binding) { listOf(key0, key1, key2, key3, key4, key5, key6, key7, key8, key9)
            }
        digitKeys.forEachIndexed { digit, key ->
            key.setOnClickListener {
                it.performHapticFeedback(HapticFeedbackConstants.KEYBOARD_TAP)
                onDigit(digit)
            }
        }
        binding.keyClear.setOnClickListener {
            it.performHapticFeedback(HapticFeedbackConstants.KEYBOARD_TAP)
            onClear()
        }

        // Backspace works like a phone's: a tap deletes one digit, holding it keeps deleting.
        binding.keyBackspace.setOnClickListener {
            it.performHapticFeedback(HapticFeedbackConstants.KEYBOARD_TAP)
            onBackspace()
        }
        binding.keyBackspace.setOnLongClickListener {
            it.removeCallbacks(repeatBackspace)
            repeatBackspace.run()
            true
        }
        binding.keyBackspace.setOnTouchListener { key, event ->
            if (event.actionMasked == MotionEvent.ACTION_UP || event.actionMasked ==
                MotionEvent.ACTION_CANCEL) {
                key.removeCallbacks(repeatBackspace)
            }
            false // The button still handles the touch itself: press state, click and long click
        }

        // After a rotation, put back the amount and change list that were on screen.
        if (savedInstanceState != null) {
            amount = savedInstanceState.getString(KEY_AMOUNT).orEmpty()
            change = savedInstanceState.getLongArray(KEY_CHANGE)
                ?: ChangeCalculator.breakdown(ChangeCalculator.valueOf(amount))
        }
        render(animate = false)
        if (savedInstanceState == null) playEntrance()
    }

    override fun onSaveInstanceState(outState: Bundle) {
        super.onSaveInstanceState(outState)
        outState.putString(KEY_AMOUNT, amount)
        outState.putLongArray(KEY_CHANGE, change)
    }

    /** Hardware keyboards work too: digits type, Backspace deletes, Esc or C clears. */
    override fun onKeyDown(keyCode: Int, event: KeyEvent): Boolean {
        val key = when (keyCode) {
            in KeyEvent.KEYCODE_0..KeyEvent.KEYCODE_9 -> digitKeys[keyCode - KeyEvent.KEYCODE_0]
            in KeyEvent.KEYCODE_NUMPAD_0..KeyEvent.KEYCODE_NUMPAD_9 -> digitKeys[keyCode -
                KeyEvent.KEYCODE_NUMPAD_0]
            KeyEvent.KEYCODE_ESCAPE, KeyEvent.KEYCODE_CLEAR, KeyEvent.KEYCODE_C -> binding.keyClear
            KeyEvent.KEYCODE_DEL -> binding.keyBackspace
            else -> return super.onKeyDown(keyCode, event)
        }
        key.flashPressed()
        key.performClick()
        return true
    }

    private fun onDigit(digit: Int) {
```

app/src/main/java/com/example/vangtichai/MainActivity.kt (continued)

```
        val updated = ChangeCalculator.appendDigit(amount, digit)
        if (updated == null) {
            rejectDigit()
            return
        }
        setAmount(updated)
    }

    private fun onClear() = setAmount("")

    private fun onBackspace() {
        if (amount.isNotEmpty()) setAmount(ChangeCalculator.removeLastDigit(amount))
    }

    private fun setAmount(newAmount: String) {
        amount = newAmount
        change = ChangeCalculator.breakdown(ChangeCalculator.valueOf(amount))
        render(animate = true)
    }

    /** The amount already has [ChangeCalculator.MAX_DIGITS] digits: shake it and say why. */
    private fun rejectDigit() {
        val amountView = binding.amountText
        amountView.performHapticFeedback(
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) HapticFeedbackConstants.REJECT
            else HapticFeedbackConstants.LONG_PRESS
        )
        val d = resources.getDimension(R.dimen.shake_distance)
        ObjectAnimator.ofFloat(amountView, View.TRANSLATION_X, 0f, d, -d, d / 2, -d / 2, 0f)
            .setDuration(SHAKE_DURATION)
            .start()

        val caption = binding.captionText
        caption.text = resources.getQuantityString(
            R.plurals.caption_max_digits, ChangeCalculator.MAX_DIGITS, ChangeCalculator.MAX_DIGITS
        )
        caption.removeCallbacks(restoreCaption)
        caption.postDelayed(restoreCaption, CAPTION_RESTORE_DELAY)
    }

    private fun render(animate: Boolean) {
        binding.amountText.text = buildSpannedString {
            scale(LABEL_SCALE) { color(labelColor) { append(getString(R.string.label_taka)) } }
            if (amount.isNotEmpty()) {
                append(' ')
                bold { append(amount) }
            }
        }
        binding.amountText.contentDescription =
            if (amount.isEmpty()) getString(R.string.cd_amount_empty) else
                getString(R.string.cd_amount, amount)

        binding.captionText.removeCallbacks(restoreCaption)
        binding.captionText.text = caption()

        change.forEachIndexed { i, count -> renderRow(i, count, animate) }
        renderBackspace(animate)
        if (animate) binding.amountText.pop(AMOUNT_POP_SCALE)
    }

    /** Like in a phone dialer, the backspace key only shows while there is a digit to delete. */
    private fun renderBackspace(animate: Boolean) {
        val key = binding.keyBackspace
        val show = amount.isNotEmpty()
        val alpha = if (show) 1f else 0f
        if (key.isVisible == show && key.alpha == alpha) return
        key.animate().cancel()
        if (!animate) {
            key.alpha = alpha
            key.isInvisible = !show
```

app/src/main/java/com/example/vangtichai/MainActivity.kt (continued)

```
            return
        }
        key.isInvisible = false
        key.animate()
            .alpha(alpha)
            .setDuration(BACKSPACE_FADE_DURATION)
            .withEndAction { key.isInvisible = !show }
            .start()
    }

    private fun renderRow(index: Int, count: Long, animate: Boolean) {
        val row = noteRows[index]
        val countText = count.toString()
        val changed = row.noteCount.text.toString() != countText
        row.noteCount.text = countText
        row.root.contentDescription = resources.getQuantityString(
            R.plurals.cd_note_row, count.toInt(), ChangeCalculator.NOTES[index], countText
        )

        // Notes that are part of the change light up in their own colour; the rest fade back.
        lightRow(index, if (count > 0) 1f else 0f, animate)
        if (animate && changed) row.noteCount.pop(COUNT_POP_SCALE, pivotAtEnd = true)
    }

    private fun caption(): String {
        if (amount.isEmpty()) return getString(R.string.hint_enter_amount)
        val notes = change.sum()
        return resources.getQuantityString(R.plurals.caption_note_count, notes.toInt(),
            notes.toString())
    }

    /** Moves a row towards [target] (0 = faded back, 1 = lit up), smoothly when [animate] is set. */
    private fun lightRow(index: Int, target: Float, animate: Boolean) {
        rowAnimators[index]?.cancel()
        rowAnimators[index] = null
        val shown = rowLevels[index]
        if (!animate || shown == target) {
            setRowLevel(index, target)
            return
        }
        rowAnimators[index] = ValueAnimator.ofFloat(shown, target).apply {
            duration = ROW_FADE_DURATION
            addUpdateListener { setRowLevel(index, it.animatedValue as Float) }
            start()
        }
    }

    /** Colours a row for its [level]: the background, colour tab, note and count all blend together.
        */
    private fun setRowLevel(index: Int, level: Float) {
        rowLevels[index] = level
        val row = noteRows[index]
        val noteColor = noteColors[index]
        row.noteLabel.setTextColor(ColorUtils.blendARGB(mutedTextColor, noteColor, level))
        row.noteCount.setTextColor(ColorUtils.blendARGB(mutedTextColor, strongTextColor, level))
        accentShapes[index].setColor(
            ColorUtils.blendARGB(ColorUtils.setAlphaComponent(noteColor, IDLE_ACCENT_ALPHA),
                noteColor, level)
        )
        rowShapes[index].setColor(ColorUtils.blendARGB(idleRowColor, litRowColors[index], level))
    }

    /**
     * The window is drawn edge to edge, so the header grows under the status bar while the change
     * table and keypad keep clear of the navigation bar and any display cutout.
     */
    private fun applySystemBarInsets() {
        val header = binding.header
        val headerLeft = header.paddingLeft
        val headerTop = header.paddingTop
```

app/src/main/java/com/example/vangtichai/MainActivity.kt (continued)

```
        val headerRight = header.paddingRight
        val notesMargins = binding.notesCard.layoutParams as ViewGroup.MarginLayoutParams
        val notesStart = notesMargins.marginStart
        val notesBottom = notesMargins.bottomMargin
        val keypadEnd = (binding.keypad.layoutParams as ViewGroup.MarginLayoutParams).marginEnd

        // The root layout is always left-to-right, so start is left and end is right.
        ViewCompat.setOnApplyWindowInsetsListener(binding.root) { _, insets ->
            val bars = insets.getInsets(
                WindowInsetsCompat.Type.systemBars() or WindowInsetsCompat.Type.displayCutout()
            )
            header.updatePadding(
                left = headerLeft + bars.left,
                top = headerTop + bars.top,
                right = headerRight + bars.right,
            )
            binding.notesCard.updateLayoutParams<ViewGroup.MarginLayoutParams> {
                marginStart = notesStart + bars.left
                bottomMargin = notesBottom + bars.bottom
            }
            // The keypad takes its top and bottom from the change table, so it only needs the side.
            binding.keypad.updateLayoutParams<ViewGroup.MarginLayoutParams> {
                marginEnd = keypadEnd + bars.right
            }
            WindowInsetsCompat.CONSUMED
        }
    }

    /** On a fresh start the change table, then the keypad, rise gently into place. */
    private fun playEntrance() {
        val rise = resources.getDimension(R.dimen.entrance_rise)
        listOf(binding.notesCard, binding.keypad).forEachIndexed { i, section ->
            section.alpha = 0f
            section.translationY = rise
            section.animate()
                .alpha(1f)
                .translationY(0f)
                .setStartDelay(ENTRANCE_DELAY + i * ENTRANCE_STAGGER)
                .setDuration(ENTRANCE_DURATION)
                .setInterpolator(DecelerateInterpolator(ENTRANCE_DECELERATION))
                .start()
        }
    }

    /** Briefly enlarges the view and lets it spring back, drawing the eye to a changed value. */
    private fun View.pop(scale: Float, pivotAtEnd: Boolean = false) {
        animate().cancel()
        pivotX = if (pivotAtEnd) width.toFloat() else width / 2f
        pivotY = height / 2f
        scaleX = scale
        scaleY = scale
        animate()
            .scaleX(1f)
            .scaleY(1f)
            .setDuration(POP_DURATION)
            .setInterpolator(OvershootInterpolator(OVERSHOOT_TENSION))
            .start()
    }

    /** Shows the pressed state for a moment, so hardware key presses are visible on screen. */
    private fun View.flashPressed() {
        isPressed = true
        postDelayed({ isPressed = false }, KEY_FLASH_DURATION)
    }

    private companion object {
        const val KEY_AMOUNT = "amount"
        const val KEY_CHANGE = "change"

        /** Note colours, in the same order as [ChangeCalculator.NOTES]. */
```

app/src/main/java/com/example/vangtichai/MainActivity.kt (continued)

```
387         val NOTE_COLORS = intArrayOf(
388             R.color.note_500, R.color.note_100, R.color.note_50, R.color.note_20,
389             R.color.note_10, R.color.note_5, R.color.note_2, R.color.note_1,
390         )
391
392         /** "Taka:" is drawn at this fraction of the amount's text size. */
393         const val LABEL_SCALE = 0.62f
394
395         const val ACTIVE_ROW_ALPHA = 0x29
396         const val IDLE_ACCENT_ALPHA = 0x40
397         const val MUTED_TEXT_ALPHA = 0x99
398
399         const val AMOUNT_POP_SCALE = 1.06f
400         const val COUNT_POP_SCALE = 1.35f
401         const val POP_DURATION = 280L
402         const val OVERSHOOT_TENSION = 3f
403         const val ROW_FADE_DURATION = 220L
404         const val SHAKE_DURATION = 400L
405         const val CAPTION_RESTORE_DELAY = 1600L
406         const val KEY_FLASH_DURATION = 90L
407         const val BACKSPACE_REPEAT_INTERVAL = 70L
408         const val BACKSPACE_FADE_DURATION = 150L
409         const val ENTRANCE_DELAY = 60L
410         const val ENTRANCE_STAGGER = 70L
411         const val ENTRANCE_DURATION = 380L
412         const val ENTRANCE_DECELERATION = 2f
413     }
414 }
```

A.4 Tests

app/src/test/java/com/example/vangtichai/**ChangeCalculatorTest.kt** 97 lines · Kotlin

ABOUT THIS FILE

The 11 JVM unit tests from Part 3, Step 17.1. Notes for reading the listing:

- **Line 12** is a comment that lines up the note values over the expected arrays on lines 13–16, so each count sits under its note.
- `assertArrayEquals(expected, actual)` takes the expected value *first*. When a test fails, JUnit prints both.
- **Line 33** works out the sum of count × note with `sumOf`. A `Long` times an `Int` is a `Long`, so the total compares directly with the `Long` amount.
- **Lines 41–51** are the dynamic-programming check explained in Step 17.1. Its two loops take about 2,000 × 8 = 16,000 steps.
- The message `"amount $amount"` (lines 34 and 50) names the amount that failed, which makes a failure easy to reproduce.
- **Line 54**: `@Test(expected = ...)` passes only if that exception is thrown.
- **Lines 62–63** use `!!` because `appendDigit` returns `String?`. It cannot return null here: the amounts are far below 9 digits.
- **Lines 71–74** cover all four cases of the leading-zero rule: nothing typed + 0, "0" + 0, "0" + 5 and "5" + 0.

app/src/test/java/com/example/vangtichai/ChangeCalculatorTest.kt

```kotlin
package com.example.vangtichai

import org.junit.Assert.assertArrayEquals
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Test

class ChangeCalculatorTest {

    @Test
    fun breakdownMatchesTheAssignmentScreenshots() {
        //                          500 100  50  20  10   5   2   1
        assertArrayEquals(longArrayOf(0, 0, 0, 0, 0, 1, 0, 1), ChangeCalculator.breakdown(6))
        assertArrayEquals(longArrayOf(0, 0, 1, 0, 1, 1, 1, 1), ChangeCalculator.breakdown(68))
        assertArrayEquals(longArrayOf(1, 1, 1, 1, 1, 1, 1, 1), ChangeCalculator.breakdown(688))
        assertArrayEquals(longArrayOf(13, 3, 1, 1, 1, 0, 0, 0), ChangeCalculator.breakdown(6880))
    }

    @Test
    fun zeroNeedsNoNotes() {
        assertArrayEquals(LongArray(ChangeCalculator.NOTES.size), ChangeCalculator.breakdown(0))
    }

    @Test
    fun largestAcceptedAmount() {
        assertArrayEquals(longArrayOf(1_999_999, 4, 1, 2, 0, 1, 2, 0),
            ChangeCalculator.breakdown(999_999_999))
    }

    @Test
    fun notesAlwaysAddUpToTheAmount() {
        for (amount in 0L..5_000L) {
            val counts = ChangeCalculator.breakdown(amount)
            val total = counts.indices.sumOf { counts[it] * ChangeCalculator.NOTES[it] }
            assertEquals("amount $amount", amount, total)
        }
    }

    @Test
    fun usesTheFewestNotesPossible() {
        // Fewest notes for every amount, worked out the slow way (dynamic programming)
        val limit = 2_000
        val fewest = IntArray(limit + 1) { Int.MAX_VALUE }
        fewest[0] = 0
        for (amount in 1..limit) {
            for (note in ChangeCalculator.NOTES) {
                if (note <= amount) fewest[amount] = minOf(fewest[amount], fewest[amount - note] + 1)
            }
        }
        for (amount in 0..limit) {
            assertEquals("amount $amount", fewest[amount].toLong(),
                ChangeCalculator.breakdown(amount.toLong()).sum())
        }
    }

    @Test(expected = IllegalArgumentException::class)
    fun negativeAmountIsRejected() {
        ChangeCalculator.breakdown(-1)
    }

    @Test
    fun digitsAreAddedFromTheRight() {
        val two = ChangeCalculator.appendDigit("", 2)
        val twentyThree = ChangeCalculator.appendDigit(two!!, 3)
        val twoThirtyFour = ChangeCalculator.appendDigit(twentyThree!!, 4)
        assertEquals("2", two)
        assertEquals("23", twentyThree)
        assertEquals("234", twoThirtyFour)
    }

```

app/src/test/java/com/example/vangtichai/ChangeCalculatorTest.kt (continued)

```
69      @Test
70      fun leadingZeroIsReplaced() {
71          assertEquals("0", ChangeCalculator.appendDigit("", 0))
72          assertEquals("0", ChangeCalculator.appendDigit("0", 0))
73          assertEquals("5", ChangeCalculator.appendDigit("0", 5))
74          assertEquals("50", ChangeCalculator.appendDigit("5", 0))
75      }
76
77      @Test
78      fun inputStopsAtMaxDigits() {
79          val full = "1".repeat(ChangeCalculator.MAX_DIGITS)
80          assertNull(ChangeCalculator.appendDigit(full, 1))
81          assertEquals(full, ChangeCalculator.appendDigit(full.dropLast(1), 1))
82      }
83
84      @Test
85      fun backspaceRemovesTheRightmostDigit() {
86          assertEquals("23", ChangeCalculator.removeLastDigit("234"))
87          assertEquals("2", ChangeCalculator.removeLastDigit("23"))
88          assertEquals("", ChangeCalculator.removeLastDigit("2"))
89          assertEquals("", ChangeCalculator.removeLastDigit(""))
90      }
91
92      @Test
93      fun valueOfTypedDigits() {
94          assertEquals(0L, ChangeCalculator.valueOf(""))
95          assertEquals(688L, ChangeCalculator.valueOf("688"))
96      }
97  }
```

app/src/androidTest/java/com/example/vangtichai/**MainActivityTest.kt** 118 lines · Kotlin

ABOUT THIS FILE

The 7 Espresso tests from Part 3, Step 17.2. They need a device or an emulator, because they drive the real activity. All 7 pass on the Pixel XL and Nexus 10 emulators and on Android 5.0 (Part 5.2).

- **Lines 26–27**: `ActivityScenarioRule` starts a fresh `MainActivity` before every test and closes it afterwards, so no test sees another's typing.
- **Lines 92–117**: four helpers keep the tests short. `tap` clicks a view by id. `type("688")` turns each character into a digit with `digitToInt()` and taps that key. `assertAmount` compares the amount's plain text, so the spans (the small "Taka:", the bold digits) do not matter. `assertCounts` takes eight counts as a `vararg` and checks each row's `note_count` *inside* that row (`isDescendantOfA`), because all eight rows contain a view with the id `note_count`.
- **Line 67**: the backspace key starts `INVISIBLE`, and Espresso's `isDisplayed()` is false for an invisible view, so `not(isDisplayed())` passes.
- **Lines 73–81**: a long press must delete at least two of the seven digits. How many more go depends on timing, so lines 77–79 list every acceptable text, from "Taka:" (all deleted) to "Taka: 12345" (two deleted), and `anyOf` accepts any of them.
- **Line 87**: `scenario.recreate()` destroys and recreates the activity through the same save-and-restore path as a rotation.

app/src/androidTest/java/com/example/vangtichai/MainActivityTest.kt

```
1   package com.example.vangtichai
2
3   import androidx.annotation.IdRes
4   import androidx.test.espresso.Espresso.onView
5   import androidx.test.espresso.action.ViewActions.click
```

app/src/androidTest/java/com/example/vangtichai/MainActivityTest.kt (continued)

```kotlin
import androidx.test.espresso.action.ViewActions.longClick
import androidx.test.espresso.assertion.ViewAssertions.matches
import androidx.test.espresso.matcher.ViewMatchers.isDescendantOfA
import androidx.test.espresso.matcher.ViewMatchers.isDisplayed
import androidx.test.espresso.matcher.ViewMatchers.withId
import androidx.test.espresso.matcher.ViewMatchers.withText
import androidx.test.ext.junit.rules.ActivityScenarioRule
import androidx.test.ext.junit.runners.AndroidJUnit4
import org.hamcrest.Matchers.allOf
import org.hamcrest.Matchers.anyOf
import org.hamcrest.Matchers.equalTo
import org.hamcrest.Matchers.not
import org.junit.Rule
import org.junit.Test
import org.junit.runner.RunWith

/** Runs on a device or emulator: drives the keypad and checks the amount and change table. */
@RunWith(AndroidJUnit4::class)
class MainActivityTest {

    @get:Rule
    val activityRule = ActivityScenarioRule(MainActivity::class.java)

    @Test
    fun digitsAreAddedFromTheRight() {
        tap(R.id.key_2)
        assertAmount("Taka: 2")
        tap(R.id.key_3)
        assertAmount("Taka: 23")
        tap(R.id.key_4)
        assertAmount("Taka: 234")
    }

    @Test
    fun changeTableListsTheNotes() {
        type("6880")
        assertCounts("13", "3", "1", "1", "1", "0", "0", "0")
    }

    @Test
    fun clearResetsAmountAndTable() {
        type("688")
        tap(R.id.key_clear)
        assertAmount("Taka:")
        assertCounts("0", "0", "0", "0", "0", "0", "0", "0")
    }

    @Test
    fun backspaceRemovesTheLastDigit() {
        type("688")
        tap(R.id.key_backspace)
        assertAmount("Taka: 68")
        assertCounts("0", "0", "1", "0", "1", "1", "1", "1")
        tap(R.id.key_backspace)
        tap(R.id.key_backspace)
        assertAmount("Taka:")
        assertCounts("0", "0", "0", "0", "0", "0", "0", "0")
    }

    @Test
    fun backspaceShowsOnceSomethingIsTyped() {
        onView(withId(R.id.key_backspace)).check(matches(not(isDisplayed())))
        type("5")
        onView(withId(R.id.key_backspace)).check(matches(isDisplayed()))
    }

    @Test
    fun holdingBackspaceKeepsDeleting() {
        type("1234567")
        onView(withId(R.id.key_backspace)).perform(longClick())
```

app/src/androidTest/java/com/example/vangtichai/MainActivityTest.kt (continued)

```
        // A tap deletes one digit; holding keeps deleting, so at least two are gone when it is let go
        val atLeastTwoDeleted = (0..5).map { kept ->
            equalTo(if (kept == 0) "Taka:" else "Taka: " + "1234567".take(kept))
        }
        onView(withId(R.id.amount_text)).check(matches(withText(anyOf<String>(atLeastTwoDeleted))))
    }

    @Test
    fun amountAndChangeSurviveRecreation() {
        type("688")
        // The same save/restore path a rotation goes through
        activityRule.scenario.recreate()
        assertAmount("Taka: 688")
        assertCounts("1", "1", "1", "1", "1", "1", "1", "1")
    }

    private fun tap(@IdRes key: Int) {
        onView(withId(key)).perform(click())
    }

    private fun type(digits: String) {
        val keys = intArrayOf(
            R.id.key_0, R.id.key_1, R.id.key_2, R.id.key_3, R.id.key_4,
            R.id.key_5, R.id.key_6, R.id.key_7, R.id.key_8, R.id.key_9,
        )
        digits.forEach { tap(keys[it.digitToInt()]) }
    }

    private fun assertAmount(text: String) {
        onView(withId(R.id.amount_text)).check(matches(withText(text)))
    }

    private fun assertCounts(vararg counts: String) {
        val rows = intArrayOf(
            R.id.note_500, R.id.note_100, R.id.note_50, R.id.note_20,
            R.id.note_10, R.id.note_5, R.id.note_2, R.id.note_1,
        )
        rows.forEachIndexed { i, row ->
            onView(allOf(withId(R.id.note_count), isDescendantOfA(withId(row))))
                .check(matches(withText(counts[i])))
        }
    }
}
```

A.5 Values

app/src/main/res/values/**colors.xml** 67 lines · XML

ABOUT THIS FILE

Every colour in the app (Part 3, Step 4, which also shows the most important ones as swatches). The file has five groups:

Lines	Group	Colours
3–35	Material 3 colour roles (`md_*`), mapped to theme attributes in `themes.xml`	32
37–43	Header gradient, glow, and the text drawn on top of it	6
45–47	The change table card and idle rows	2
49–57	One colour per note	8
59–66	Launcher icon	7

Colours with eight hex digits start with an alpha (opacity) byte, `#AARRGGBB`: `header_glow` (`#1FFFFFFF`) is white at 12%, `header_content_secondary` (`#D9FFFFFF`) white at 85%, `header_ripple` (`#3DFFFFFF`) white at 24%, and `icon_note_back` (`#8CFFFFFF`) white at 55%.

`md_surface` and `md_background` are the same near-black green (`#0E1411`). The change-table card uses neither: its style sets the slightly lighter `card_surface`.

app/src/main/res/values/colors.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Material 3 colour roles. The app always uses this dark theme, whatever the system setting. -->
    <color name="md_primary">#6ADBAD</color>
    <color name="md_on_primary">#003827</color>
    <color name="md_primary_container">#005139</color>
    <color name="md_on_primary_container">#87F8C9</color>
    <color name="md_secondary">#B3CCBF</color>
    <color name="md_on_secondary">#1E352B</color>
    <color name="md_secondary_container">#2A3B34</color>
    <color name="md_on_secondary_container">#D3ECDF</color>
    <color name="md_tertiary">#FFB3B6</color>
    <color name="md_on_tertiary">#680019</color>
    <color name="md_tertiary_container">#5C1A24</color>
    <color name="md_on_tertiary_container">#FFDADB</color>
    <color name="md_error">#FFB4AB</color>
    <color name="md_on_error">#690005</color>
    <color name="md_error_container">#93000A</color>
    <color name="md_on_error_container">#FFDAD6</color>
    <color name="md_background">#0E1411</color>
    <color name="md_on_background">#DEE4DF</color>
    <color name="md_surface">#0E1411</color>
    <color name="md_on_surface">#DEE4DF</color>
    <color name="md_surface_variant">#404944</color>
    <color name="md_on_surface_variant">#BFC9C2</color>
    <color name="md_outline">#8A938D</color>
    <color name="md_outline_variant">#2E3833</color>
    <color name="md_surface_container_lowest">#090E0C</color>
    <color name="md_surface_container_low">#161D19</color>
    <color name="md_surface_container">#1B221E</color>
    <color name="md_surface_container_high">#252C28</color>
    <color name="md_surface_container_highest">#303733</color>
    <color name="md_inverse_surface">#DEE4DF</color>
    <color name="md_inverse_on_surface">#2C322E</color>
    <color name="md_inverse_primary">#006C4E</color>

    <!-- Header gradient and the text drawn on top of it -->
    <color name="header_gradient_start">#0B6E4D</color>
    <color name="header_gradient_end">#06301F</color>
    <color name="header_glow">#1FFFFFFF</color>
    <color name="header_content">#FFFFFF</color>
    <color name="header_content_secondary">#D9FFFFFF</color>
    <color name="header_ripple">#3DFFFFFF</color>

    <!-- Change table card, and the background of rows whose note is not in the change -->
    <color name="card_surface">#151C18</color>
    <color name="note_row_idle">#1C2420</color>

    <!-- One colour per Taka note in the change table, light enough to read on dark surfaces -->
    <color name="note_500">#D4A9F2</color>
    <color name="note_100">#9DC4FF</color>
    <color name="note_50">#FFB77C</color>
    <color name="note_20">#6FD9C7</color>
    <color name="note_10">#FF8DB7</color>
    <color name="note_5">#F2C150</color>
    <color name="note_2">#A6D77A</color>
    <color name="note_1">#B5C3BD</color>

    <!-- Launcher icon -->
    <color name="icon_background_start">#13A874</color>
    <color name="icon_background_end">#014A34</color>
```

app/src/main/res/values/colors.xml (continued)

```
    <color name="icon_note">#FFFFFF</color>
    <color name="icon_note_back">#8CFFFFFF</color>
    <color name="icon_note_detail">#0B7A55</color>
    <color name="icon_coin">#F7C548</color>
    <color name="icon_coin_detail">#C98F12</color>
</resources>
```

app/src/main/res/values/**strings.xml** 44 lines · XML

ABOUT THIS FILE

All user-visible text (Part 3, Step 5). Where each string is used:

String (line)	Text	Used by
`app_name` (2)	VangtiChai	The name under the launcher icon, and the toolbar title
`app_tagline` (3)	Taka change calculator	The toolbar subtitle, in the tablet layouts only
`label_taka` (6)	Taka:	`render()` builds "Taka: 688" from it; also the amount's text before the first render
`hint_enter_amount` (7)	Enter an amount	`caption()` when nothing is typed; also the caption's starting text
`caption_max_digits` (8–11)	Up to 9 digits	`rejectDigit()`
`caption_note_count` (12–15)	8 notes	`caption()`
`notes_title` (18)	Change	The card's title, shown as "CHANGE" because its style sets `textAllCaps`
`note_label` (19)	%1$d:	Each row's label ("500:"), filled in by `onCreate()`
`notes_count_heading` (20)	Count	Tablet portrait only, shown as "COUNT"
`key_0` … `key_9` (23–32)	0 … 9	The digit keys, marked `translatable="false"`
`key_clear` (33)	CLEAR	The CLEAR key
`cd_amount_empty` (36)	No amount entered	What TalkBack says for an empty amount
`cd_amount` (37)	%1$s Taka	What TalkBack says for the amount, e.g. "688 Taka"
`cd_note_row` (38–41)	1 note of 500 Taka	What TalkBack says for each row
`cd_key_clear` (42)	Clear the amount	The CLEAR key's content description
`cd_key_backspace` (43)	Delete the last digit	The backspace key's content description

Format codes. `%1$d` means "argument 1, as a whole number" and `%1$s` "argument 1, as text". In `cd_note_row` the count (`%2$s`) comes before the note (`%1$d`) in the sentence, although the code passes the note first. Numbered arguments allow that reordering, which translations often need.

Plurals. English has two plural forms, `one` and `other`, and Android picks one from the quantity passed to `getQuantityString`. That is why `rejectDigit()` passes `MAX_DIGITS` twice: once as the quantity (choosing "digits") and once as the number printed in the text.

app/src/main/res/values/strings.xml

```
<resources>
    <string name="app_name">VangtiChai</string>
    <string name="app_tagline">Taka change calculator</string>

```

app/src/main/res/values/strings.xml (continued)

```
    <!-- Amount display -->
    <string name="label_taka">Taka:</string>
    <string name="hint_enter_amount">Enter an amount</string>
    <plurals name="caption_max_digits">
        <item quantity="one">Up to %1$d digit</item>
        <item quantity="other">Up to %1$d digits</item>
    </plurals>
    <plurals name="caption_note_count">
        <item quantity="one">%1$s note</item>
        <item quantity="other">%1$s notes</item>
    </plurals>

    <!-- Change table -->
    <string name="notes_title">Change</string>
    <string name="note_label">%1$d:</string>
    <string name="notes_count_heading">Count</string>

    <!-- Keypad. Digits are the same in every language. -->
    <string name="key_0" translatable="false">0</string>
    <string name="key_1" translatable="false">1</string>
    <string name="key_2" translatable="false">2</string>
    <string name="key_3" translatable="false">3</string>
    <string name="key_4" translatable="false">4</string>
    <string name="key_5" translatable="false">5</string>
    <string name="key_6" translatable="false">6</string>
    <string name="key_7" translatable="false">7</string>
    <string name="key_8" translatable="false">8</string>
    <string name="key_9" translatable="false">9</string>
    <string name="key_clear">CLEAR</string>

    <!-- Screen reader descriptions -->
    <string name="cd_amount_empty">No amount entered</string>
    <string name="cd_amount">%1$s Taka</string>
    <plurals name="cd_note_row">
        <item quantity="one">%2$s note of %1$d Taka</item>
        <item quantity="other">%2$s notes of %1$d Taka</item>
    </plurals>
    <string name="cd_key_clear">Clear the amount</string>
    <string name="cd_key_backspace">Delete the last digit</string>
</resources>
```

app/src/main/res/values/**sizes.xml** 87 lines · XML

ABOUT THIS FILE

The default sizes. Phones in portrait use them as they are, and every other configuration falls back to them for anything it does not override (Part 3, Step 6). The 56 entries are grouped by area: match constraints (lines 10–12), spacing (14–17), header (19–44), change table (46–65), the tallest the content may grow (67–68), keypad (70–74), the auto-size step (76–77) and proportions (79–86).

`match_constraint` (line 12) is ConstraintLayout's `0dp`. Every layout uses `@dimen/match_constraint` for a width or height that stretches between constraints, so not even that zero is written in a layout (Part 2.9).

Entries written as `<item ... format="float" type="dimen">` hold plain numbers instead of sizes; layouts read them as `@dimen/...` all the same. Kotlin reads real sizes with `resources.getDimension(...)`, which converts dp to pixels for the current screen, as `rejectDigit()` and `playEntrance()` do.

Every dimension in every configuration. The table after this box is generated from the four `sizes.xml` files. Upright values are set in that configuration's own file; grey italic values are inherited from a more general one.

Dimension	Phone portrait	Phone landscape	Tablet portrait	Tablet landscape
match_constraint	0dp	*0dp*	*0dp*	*0dp*
screen_margin	16dp	16dp	24dp	*24dp*

Dimension	Phone portrait	Phone landscape	Tablet portrait	Tablet landscape
section_gap	16dp	12dp	24dp	24dp
column_gap	12dp	12dp	20dp	20dp
header_corner_radius	28dp	24dp	40dp	40dp
header_padding_horizontal	20dp	20dp	32dp	32dp
header_padding_bottom	20dp	8dp	32dp	24dp
header_glow_size	220dp	180dp	380dp	380dp
header_glow_offset	-90dp	-80dp	-150dp	-150dp
header_glow_radius	110dp	90dp	190dp	190dp
toolbar_height	56dp	56dp	76dp	76dp
toolbar_title_text	22sp	20sp	26sp	26sp
toolbar_subtitle_text	14sp	14sp	15sp	15sp
amount_height	64dp	44dp	112dp	88dp
amount_text_max	48sp	36sp	88sp	68sp
amount_text_min	16sp	16sp	24sp	24sp
amount_side_margin	60dp	60dp	96dp	96dp
caption_text	14sp	13sp	20sp	18sp
backspace_size	48dp	48dp	72dp	72dp
backspace_icon_size	28dp	28dp	44dp	44dp
backspace_margin_end	10dp	10dp	18dp	18dp
caption_margin_top	2dp	2dp	4dp	4dp
shake_distance	10dp	10dp	16dp	16dp
entrance_rise	24dp	24dp	40dp	40dp
card_corner_radius	24dp	20dp	32dp	32dp
card_elevation	0dp	0dp	0dp	0dp
card_stroke_width	1dp	1dp	1dp	1dp
notes_card_padding	8dp	6dp	14dp	14dp
notes_title_text	12sp	12sp	14sp	14sp
notes_title_margin_horizontal	8dp	8dp	14dp	14dp
notes_title_margin_vertical	8dp	4dp	12dp	12dp
note_row_gap	4dp	4dp	8dp	8dp
note_column_half_gap	4dp	4dp	6dp	6dp
note_row_corner_radius	14dp	12dp	20dp	20dp
note_row_padding_start	8dp	8dp	12dp	12dp
note_row_padding_end	10dp	10dp	18dp	18dp
note_accent_width	4dp	4dp	6dp	6dp
note_accent_corner_radius	2dp	2dp	3dp	3dp
note_label_margin_start	6dp	6dp	12dp	12dp
note_label_text	18sp	18sp	30sp	28sp
note_count_margin_start	4dp	4dp	8dp	8dp
note_count_text_max	22sp	20sp	32sp	30sp
note_count_text_min	9sp	9sp	10sp	10sp
content_max_height	500dp	500dp	780dp	780dp
key_gap	10dp	8dp	16dp	16dp
key_corner_radius	20dp	16dp	32dp	32dp
key_text	28sp	24sp	46sp	40sp
key_clear_text	20sp	18sp	30sp	28sp
autosize_step	1sp	1sp	1sp	1sp
half	0.5	0.5	0.5	0.5

Dimension	Phone portrait	Phone landscape	Tablet portrait	Tablet landscape
notes_width_percent	0.4	0.42	0.4	0.47
header_height_percent	0.32	*0.32*	0.3	*0.3*
content_vertical_bias	0.5	*0.5*	*0.5*	*0.5*
note_accent_height_percent	0.5	*0.5*	*0.5*	*0.5*
notes_title_letter_spacing	0.12	*0.12*	*0.12*	*0.12*
key_clear_letter_spacing	0.08	*0.08*	*0.08*	*0.08*

app/src/main/res/values/sizes.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!--
    Sizes for phones in portrait (designed on a Pixel XL, 411 x 731 dp).
    The layouts never hardcode sizes. Other screen configurations override the values that differ:
      values-land/          phones in landscape
      values-sw600dp/       tablets in portrait (Nexus 10)
      values-sw600dp-land/  tablets in landscape
-->
<resources>
    <!-- ConstraintLayout's "match constraints": a width or height of 0 lets a view stretch between
         its constraints. Kept here like every other size, so no size is hardcoded in a layout. -->
    <dimen name="match_constraint">0dp</dimen>

    <!-- Space around and between the main sections -->
    <dimen name="screen_margin">16dp</dimen>
    <dimen name="section_gap">16dp</dimen>
    <dimen name="column_gap">12dp</dimen>

    <!-- Header: app bar and amount display -->
    <dimen name="header_corner_radius">28dp</dimen>
    <dimen name="header_padding_horizontal">20dp</dimen>
    <dimen name="header_padding_bottom">20dp</dimen>
    <dimen name="header_glow_size">220dp</dimen>
    <dimen name="header_glow_offset">-90dp</dimen>
    <dimen name="header_glow_radius">110dp</dimen>
    <dimen name="toolbar_height">56dp</dimen>
    <dimen name="toolbar_title_text">22sp</dimen>
    <dimen name="toolbar_subtitle_text">14sp</dimen>
    <dimen name="amount_height">64dp</dimen>
    <dimen name="amount_text_max">48sp</dimen>
    <dimen name="amount_text_min">16sp</dimen>
    <!-- Room on both sides of the amount: enough for the backspace key on the right, and the
         same on the left so the amount stays centred -->
    <dimen name="amount_side_margin">60dp</dimen>
    <dimen name="caption_text">14sp</dimen>
    <!-- Backspace key at the end of the amount row. The margin lines the icon up with the
         app bar title's padding: header_padding_horizontal - (size - icon size) / 2. -->
    <dimen name="backspace_size">48dp</dimen>
    <dimen name="backspace_icon_size">28dp</dimen>
    <dimen name="backspace_margin_end">10dp</dimen>
    <dimen name="caption_margin_top">2dp</dimen>
    <dimen name="shake_distance">10dp</dimen>
    <!-- How far the change table and keypad rise into place when the app starts -->
    <dimen name="entrance_rise">24dp</dimen>

    <!-- Change table -->
    <dimen name="card_corner_radius">24dp</dimen>
    <dimen name="card_elevation">0dp</dimen>
    <dimen name="card_stroke_width">1dp</dimen>
    <dimen name="notes_card_padding">8dp</dimen>
    <dimen name="notes_title_text">12sp</dimen>
    <dimen name="notes_title_margin_horizontal">8dp</dimen>
    <dimen name="notes_title_margin_vertical">8dp</dimen>
    <dimen name="note_row_gap">4dp</dimen>
    <dimen name="note_column_half_gap">4dp</dimen>
    <dimen name="note_row_corner_radius">14dp</dimen>
    <dimen name="note_row_padding_start">8dp</dimen>
    <dimen name="note_row_padding_end">10dp</dimen>
```

app/src/main/res/values/sizes.xml (continued)

```xml
    <dimen name="note_accent_width">4dp</dimen>
    <dimen name="note_accent_corner_radius">2dp</dimen>
    <dimen name="note_label_margin_start">6dp</dimen>
    <dimen name="note_label_text">18sp</dimen>
    <dimen name="note_count_margin_start">4dp</dimen>
    <dimen name="note_count_text_max">22sp</dimen>
    <dimen name="note_count_text_min">9sp</dimen>

    <!-- Tallest the change table (and the keypad beside it) grows on portrait screens -->
    <dimen name="content_max_height">500dp</dimen>

    <!-- Keypad -->
    <dimen name="key_gap">10dp</dimen>
    <dimen name="key_corner_radius">20dp</dimen>
    <dimen name="key_text">28sp</dimen>
    <dimen name="key_clear_text">20sp</dimen>

    <!-- Smallest step auto-sized text shrinks by -->
    <dimen name="autosize_step">1sp</dimen>

    <!-- Proportions, as fractions of the available space -->
    <item name="half" format="float" type="dimen">0.5</item>
    <item name="notes_width_percent" format="float" type="dimen">0.4</item>
    <item name="header_height_percent" format="float" type="dimen">0.32</item>
    <item name="content_vertical_bias" format="float" type="dimen">0.5</item>
    <item name="note_accent_height_percent" format="float" type="dimen">0.5</item>
    <item name="notes_title_letter_spacing" format="float" type="dimen">0.12</item>
    <item name="key_clear_letter_spacing" format="float" type="dimen">0.08</item>
</resources>
```

app/src/main/res/values-land/**sizes.xml** 36 lines · XML

ABOUT THIS FILE

Overrides for phones in landscape. Height is short here, so most of the 24 entries tighten the header and the spacing: a lower amount row (`amount_height` 44 dp instead of 64), smaller key text, smaller glows, and a slightly wider change column (`notes_width_percent` 0.42) for its two columns of rows. Four entries (`screen_margin`, `column_gap`, `note_row_gap` and `note_label_text`) repeat the default value, so they change nothing.

app/src/main/res/values-land/sizes.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!--
    Sizes for phones in landscape (designed on a Pixel XL, 731 x 411 dp).
    Height is scarce here, so the caption shares the app bar row and spacing is tighter.
    Anything not listed falls back to values/sizes.xml.
-->
<resources>
    <dimen name="screen_margin">16dp</dimen>
    <dimen name="section_gap">12dp</dimen>
    <dimen name="column_gap">12dp</dimen>

    <dimen name="header_corner_radius">24dp</dimen>
    <dimen name="header_padding_bottom">8dp</dimen>
    <dimen name="header_glow_size">180dp</dimen>
    <dimen name="header_glow_offset">-80dp</dimen>
    <dimen name="header_glow_radius">90dp</dimen>
    <dimen name="toolbar_title_text">20sp</dimen>
    <dimen name="amount_height">44dp</dimen>
    <dimen name="amount_text_max">36sp</dimen>
    <dimen name="caption_text">13sp</dimen>

    <dimen name="card_corner_radius">20dp</dimen>
    <dimen name="notes_card_padding">6dp</dimen>
```

app/src/main/res/values-land/sizes.xml (continued)

```
    <dimen name="notes_title_margin_vertical">4dp</dimen>
    <dimen name="note_row_gap">4dp</dimen>
    <dimen name="note_row_corner_radius">12dp</dimen>
    <dimen name="note_label_text">18sp</dimen>
    <dimen name="note_count_text_max">20sp</dimen>

    <dimen name="key_gap">8dp</dimen>
    <dimen name="key_corner_radius">16dp</dimen>
    <dimen name="key_text">24sp</dimen>
    <dimen name="key_clear_text">18sp</dimen>

    <item name="notes_width_percent" format="float" type="dimen">0.42</item>
</resources>
```

app/src/main/res/values-sw600dp/**sizes.xml** 60 lines · XML

ABOUT THIS FILE

Overrides for tablets in portrait, and through inheritance for tablets in landscape too. Nearly everything is scaled up: 47 entries, from bigger margins and corners to `key_text` 46 sp and `amount_text_max` 88 sp. Two entries are worth a closer look:

- `header_height_percent` is **0.3** here, a little less than the 0.32 phones use. Only tablet portrait uses it, because the landscape header wraps its content.
- `notes_width_percent` repeats the default 0.4, so it changes nothing.

Every value in `values-land/sizes.xml` is also defined here. Smallest width outranks orientation when Android chooses resources (Part 2.7), so a tablet in landscape never picks up the phone-landscape sizes.

app/src/main/res/values-sw600dp/sizes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    Sizes for tablets in portrait (designed on a Nexus 10, 800 x 1280 dp).
    Everything is scaled up so the screen is filled without stretching the keys.
    Anything not listed falls back to values/sizes.xml.
-->
<resources>
    <dimen name="screen_margin">24dp</dimen>
    <dimen name="section_gap">24dp</dimen>
    <dimen name="column_gap">20dp</dimen>

    <dimen name="header_corner_radius">40dp</dimen>
    <dimen name="header_padding_horizontal">32dp</dimen>
    <dimen name="header_padding_bottom">32dp</dimen>
    <dimen name="header_glow_size">380dp</dimen>
    <dimen name="header_glow_offset">-150dp</dimen>
    <dimen name="header_glow_radius">190dp</dimen>
    <dimen name="toolbar_height">76dp</dimen>
    <dimen name="toolbar_title_text">26sp</dimen>
    <dimen name="toolbar_subtitle_text">15sp</dimen>
    <dimen name="amount_height">112dp</dimen>
    <dimen name="amount_text_max">88sp</dimen>
    <dimen name="amount_text_min">24sp</dimen>
    <dimen name="amount_side_margin">96dp</dimen>
    <dimen name="backspace_size">72dp</dimen>
    <dimen name="backspace_icon_size">44dp</dimen>
    <dimen name="backspace_margin_end">18dp</dimen>
    <dimen name="caption_text">20sp</dimen>
    <dimen name="caption_margin_top">4dp</dimen>
    <dimen name="shake_distance">16dp</dimen>
    <dimen name="entrance_rise">40dp</dimen>

```

app/src/main/res/values-sw600dp/sizes.xml (continued)

```xml
    <dimen name="card_corner_radius">32dp</dimen>
    <dimen name="notes_card_padding">14dp</dimen>
    <dimen name="notes_title_text">14sp</dimen>
    <dimen name="notes_title_margin_horizontal">14dp</dimen>
    <dimen name="notes_title_margin_vertical">12dp</dimen>
    <dimen name="note_row_gap">8dp</dimen>
    <dimen name="note_column_half_gap">6dp</dimen>
    <dimen name="note_row_corner_radius">20dp</dimen>
    <dimen name="note_row_padding_start">12dp</dimen>
    <dimen name="note_row_padding_end">18dp</dimen>
    <dimen name="note_accent_width">6dp</dimen>
    <dimen name="note_accent_corner_radius">3dp</dimen>
    <dimen name="note_label_margin_start">12dp</dimen>
    <dimen name="note_label_text">30sp</dimen>
    <dimen name="note_count_margin_start">8dp</dimen>
    <dimen name="note_count_text_max">32sp</dimen>
    <dimen name="note_count_text_min">10sp</dimen>

    <dimen name="content_max_height">780dp</dimen>

    <dimen name="key_gap">16dp</dimen>
    <dimen name="key_corner_radius">32dp</dimen>
    <dimen name="key_text">46sp</dimen>
    <dimen name="key_clear_text">30sp</dimen>

    <item name="notes_width_percent" format="float" type="dimen">0.4</item>
    <item name="header_height_percent" format="float" type="dimen">0.3</item>
</resources>
```

app/src/main/res/values-sw600dp-land/**sizes.xml** 19 lines · XML

ABOUT THIS FILE

Only nine overrides for tablets in landscape; everything else comes from `values-sw600dp/sizes.xml`, then `values/sizes.xml`. They make the amount row lower (88 dp, with text up to 68 sp), shrink the caption, the note labels, the counts and the key text a little, because landscape has less height, and give the two-column change table 47% of the width.

app/src/main/res/values-sw600dp-land/sizes.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!--
    Sizes for tablets in landscape (designed on a Nexus 10, 1280 x 800 dp).
    Anything not listed falls back to values-sw600dp/sizes.xml, then values/sizes.xml.
-->
<resources>
    <dimen name="header_padding_bottom">24dp</dimen>
    <dimen name="amount_height">88dp</dimen>
    <dimen name="amount_text_max">68sp</dimen>
    <dimen name="caption_text">18sp</dimen>

    <dimen name="note_label_text">28sp</dimen>
    <dimen name="note_count_text_max">30sp</dimen>

    <dimen name="key_text">40sp</dimen>
    <dimen name="key_clear_text">28sp</dimen>

    <item name="notes_width_percent" format="float" type="dimen">0.47</item>
</resources>
```

app/src/main/res/values/**integers.xml** 6 lines · XML

ABOUT THIS FILE

Two integers used from XML:

- `single_line` (line 3) is used as `android:maxLines="@integer/single_line"` in four styles (amount, caption, card title and count), so not even a "1" is hardcoded.
- `key_press_duration` (line 5) is the 90 ms used by all four animators in `key_press.xml`.

app/src/main/res/values/integers.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <integer name="single_line">1</integer>
4      <!-- How long (ms) a key takes to shrink when pressed and to spring back -->
5      <integer name="key_press_duration">90</integer>
6  </resources>
```

app/src/main/res/values/**themes.xml** 41 lines · XML

ABOUT THIS FILE

The app theme (Part 3, Step 7).

- **Line 5**: `Base.Theme.VangtiChai` extends `Theme.Material3.Dark.NoActionBar`: always dark, with no default action bar.
- **Lines 6–37**: 32 `<item>`s, one per Material 3 colour role, each pointing at an `md_*` colour in `colors.xml`. Components and styles read these roles: the key style uses `colorSecondaryContainer` for the keys and the card style `colorOutlineVariant` for the card's border.
- **Line 22**: `android:colorBackground` has the `android:` prefix because it is a platform attribute (the window background). The other roles are Material Components attributes, written without a prefix.
- **Line 40**: `Theme.VangtiChai`, the name the manifest uses, inherits everything from the base theme.

There is no `values-night/themes.xml`, so the theme is the same whatever the phone's dark-mode setting is.

app/src/main/res/values/themes.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3
4      <!-- Always dark, whatever the system setting, so there is no light theme to switch to -->
5      <style name="Base.Theme.VangtiChai" parent="Theme.Material3.Dark.NoActionBar">
6          <item name="colorPrimary">@color/md_primary</item>
7          <item name="colorOnPrimary">@color/md_on_primary</item>
8          <item name="colorPrimaryContainer">@color/md_primary_container</item>
9          <item name="colorOnPrimaryContainer">@color/md_on_primary_container</item>
10         <item name="colorSecondary">@color/md_secondary</item>
11         <item name="colorOnSecondary">@color/md_on_secondary</item>
12         <item name="colorSecondaryContainer">@color/md_secondary_container</item>
13         <item name="colorOnSecondaryContainer">@color/md_on_secondary_container</item>
14         <item name="colorTertiary">@color/md_tertiary</item>
15         <item name="colorOnTertiary">@color/md_on_tertiary</item>
16         <item name="colorTertiaryContainer">@color/md_tertiary_container</item>
17         <item name="colorOnTertiaryContainer">@color/md_on_tertiary_container</item>
18         <item name="colorError">@color/md_error</item>
19         <item name="colorOnError">@color/md_on_error</item>
20         <item name="colorErrorContainer">@color/md_error_container</item>
21         <item name="colorOnErrorContainer">@color/md_on_error_container</item>
22         <item name="android:colorBackground">@color/md_background</item>
```

app/src/main/res/values/themes.xml (continued)

```
        <item name="colorOnBackground">@color/md_on_background</item>
        <item name="colorSurface">@color/md_surface</item>
        <item name="colorOnSurface">@color/md_on_surface</item>
        <item name="colorSurfaceVariant">@color/md_surface_variant</item>
        <item name="colorOnSurfaceVariant">@color/md_on_surface_variant</item>
        <item name="colorOutline">@color/md_outline</item>
        <item name="colorOutlineVariant">@color/md_outline_variant</item>
        <item name="colorSurfaceContainerLowest">@color/md_surface_container_lowest</item>
        <item name="colorSurfaceContainerLow">@color/md_surface_container_low</item>
        <item name="colorSurfaceContainer">@color/md_surface_container</item>
        <item name="colorSurfaceContainerHigh">@color/md_surface_container_high</item>
        <item name="colorSurfaceContainerHighest">@color/md_surface_container_highest</item>
        <item name="colorSurfaceInverse">@color/md_inverse_surface</item>
        <item name="colorOnSurfaceInverse">@color/md_inverse_on_surface</item>
        <item name="colorPrimaryInverse">@color/md_inverse_primary</item>
    </style>

    <style name="Theme.VangtiChai" parent="Base.Theme.VangtiChai" />
</resources>
```

app/src/main/res/values/**styles.xml** 135 lines · XML

ABOUT THIS FILE

All the widget styles; Part 3, Step 8 has a table of what each one does. Where each style is, and what it inherits from:

Lines	Style	Parent
6–13	`Widget.VangtiChai.Toolbar`	`Widget.Material3.Toolbar`
15–18	`TextAppearance.VangtiChai.ToolbarTitle`	`TextAppearance.Material3.TitleLarge`
20–22	`TextAppearance.VangtiChai.ToolbarSubtitle`	`TextAppearance.Material3.BodyMedium`
25–36	`Widget.VangtiChai.Amount`	none (`parent=""`)
39–53	`Widget.VangtiChai.Backspace`	`Widget.Material3.Button.IconButton`
55–61	`Widget.VangtiChai.Caption`	none
65–71	`Widget.VangtiChai.Card`	`Widget.Material3.CardView.Outlined`
73–81	`Widget.VangtiChai.NotesTitle`	none
83–87	`TextAppearance.VangtiChai.NoteLabel`	none
89–98	`Widget.VangtiChai.NoteCount`	none
102–115	`Widget.VangtiChai.Key`	`Widget.Material3.Button.TonalButton`
117–124	`Widget.VangtiChai.Key.Clear`	`Widget.VangtiChai.Key`, implied by the name
126–129	`ShapeAppearance.VangtiChai.Key`	none
131–134	`ShapeAppearance.VangtiChai.Circle`	none

Three details:

- `parent=""` turns off inheritance by name. Without it, Android would treat a style called `Widget.VangtiChai.Amount` as a child of `Widget.VangtiChai`, a style that does not exist here.
- **Attributes without** `android:`, such as `autoSizeTextType`, `backgroundTint` and `icon`, come from AppCompat and Material Components. AppCompat swaps every `TextView` for an `AppCompatTextView` (Material's `MaterialTextView` is one), and that class supports `autoSizeTextType` on Android 5.0 to 7.1 too. The platform's own `android:autoSizeTextType` only exists from Android 8 (API 26).

- `?attr/...` **values**, such as `?attr/colorSecondaryContainer` on line 111, are looked up in the theme when a view is created, so the styles never name a colour directly.

app/src/main/res/values/styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <!-- Header -->

    <style name="Widget.VangtiChai.Toolbar" parent="Widget.Material3.Toolbar">
        <item name="android:background">@android:color/transparent</item>
        <item name="titleTextAppearance">@style/TextAppearance.VangtiChai.ToolbarTitle</item>
        <item name="titleTextColor">@color/header_content</item>
        <item name="subtitleTextAppearance">@style/TextAppearance.VangtiChai.ToolbarSubtitle</item>
        <item name="subtitleTextColor">@color/header_content_secondary</item>
        <item name="contentInsetStart">@dimen/header_padding_horizontal</item>
    </style>

    <style name="TextAppearance.VangtiChai.ToolbarTitle" parent="TextAppearance.Material3.TitleLarge">
        <item name="android:textSize">@dimen/toolbar_title_text</item>
        <item name="android:fontFamily">sans-serif-medium</item>
    </style>

    <style name="TextAppearance.VangtiChai.ToolbarSubtitle"
        parent="TextAppearance.Material3.BodyMedium">
        <item name="android:textSize">@dimen/toolbar_subtitle_text</item>
    </style>

    <!-- "Taka: 688". Shrinks to fit, so long amounts never get cut off. -->
    <style name="Widget.VangtiChai.Amount" parent="">
        <item name="android:textColor">@color/header_content</item>
        <item name="android:gravity">center</item>
        <item name="android:maxLines">@integer/single_line</item>
        <item name="android:includeFontPadding">false</item>
        <item name="android:fontFeatureSettings">tnum</item>
        <item name="android:accessibilityLiveRegion">polite</item>
        <item name="autoSizeTextType">uniform</item>
        <item name="autoSizeMinTextSize">@dimen/amount_text_min</item>
        <item name="autoSizeMaxTextSize">@dimen/amount_text_max</item>
        <item name="autoSizeStepGranularity">@dimen/autosize_step</item>
    </style>

    <!-- Backspace key beside the amount, like the one beside the number in a phone dialer -->
    <style name="Widget.VangtiChai.Backspace" parent="Widget.Material3.Button.IconButton">
        <item name="android:insetTop">0dp</item>
        <item name="android:insetBottom">0dp</item>
        <item name="android:minWidth">0dp</item>
        <item name="android:minHeight">0dp</item>
        <item name="android:padding">0dp</item>
        <item name="icon">@drawable/ic_backspace</item>
        <item name="iconGravity">textStart</item>
        <item name="iconPadding">0dp</item>
        <item name="iconSize">@dimen/backspace_icon_size</item>
        <item name="iconTint">@color/header_content</item>
        <item name="rippleColor">@color/header_ripple</item>
        <item name="shapeAppearance">@style/ShapeAppearance.VangtiChai.Circle</item>
        <item name="android:stateListAnimator">@animator/key_press</item>
    </style>

    <style name="Widget.VangtiChai.Caption" parent="">
        <item name="android:textColor">@color/header_content_secondary</item>
        <item name="android:textSize">@dimen/caption_text</item>
        <item name="android:gravity">center</item>
        <item name="android:maxLines">@integer/single_line</item>
        <item name="android:ellipsize">end</item>
    </style>

    <!-- Change table -->
```

app/src/main/res/values/styles.xml (continued)

```xml

    <style name="Widget.VangtiChai.Card" parent="Widget.Material3.CardView.Outlined">
        <item name="cardBackgroundColor">@color/card_surface</item>
        <item name="cardCornerRadius">@dimen/card_corner_radius</item>
        <item name="cardElevation">@dimen/card_elevation</item>
        <item name="strokeColor">?attr/colorOutlineVariant</item>
        <item name="strokeWidth">@dimen/card_stroke_width</item>
    </style>

    <style name="Widget.VangtiChai.NotesTitle" parent="">
        <item name="android:textColor">?attr/colorPrimary</item>
        <item name="android:textSize">@dimen/notes_title_text</item>
        <item name="android:fontFamily">sans-serif-medium</item>
        <item name="android:letterSpacing">@dimen/notes_title_letter_spacing</item>
        <item name="android:textAllCaps">true</item>
        <item name="android:maxLines">@integer/single_line</item>
        <item name="android:ellipsize">end</item>
    </style>

    <style name="TextAppearance.VangtiChai.NoteLabel" parent="">
        <item name="android:textSize">@dimen/note_label_text</item>
        <item name="android:fontFamily">sans-serif-medium</item>
        <item name="android:fontFeatureSettings">tnum</item>
    </style>

    <style name="Widget.VangtiChai.NoteCount" parent="">
        <item name="android:gravity">end|center_vertical</item>
        <item name="android:maxLines">@integer/single_line</item>
        <item name="android:fontFamily">sans-serif-medium</item>
        <item name="android:fontFeatureSettings">tnum</item>
        <item name="autoSizeTextType">uniform</item>
        <item name="autoSizeMinTextSize">@dimen/note_count_text_min</item>
        <item name="autoSizeMaxTextSize">@dimen/note_count_text_max</item>
        <item name="autoSizeStepGranularity">@dimen/autosize_step</item>
    </style>

    <!-- Keypad -->

    <style name="Widget.VangtiChai.Key" parent="Widget.Material3.Button.TonalButton">
        <item name="android:insetTop">0dp</item>
        <item name="android:insetBottom">0dp</item>
        <item name="android:minWidth">0dp</item>
        <item name="android:minHeight">0dp</item>
        <item name="android:padding">0dp</item>
        <item name="android:textSize">@dimen/key_text</item>
        <item name="android:fontFamily">sans-serif</item>
        <item name="android:textColor">?attr/colorOnSecondaryContainer</item>
        <item name="backgroundTint">?attr/colorSecondaryContainer</item>
        <item name="rippleColor">?attr/colorPrimary</item>
        <item name="shapeAppearance">@style/ShapeAppearance.VangtiChai.Key</item>
        <item name="android:stateListAnimator">@animator/key_press</item>
    </style>

    <style name="Widget.VangtiChai.Key.Clear">
        <item name="android:textSize">@dimen/key_clear_text</item>
        <item name="android:fontFamily">sans-serif-medium</item>
        <item name="android:letterSpacing">@dimen/key_clear_letter_spacing</item>
        <item name="android:textColor">?attr/colorOnTertiaryContainer</item>
        <item name="backgroundTint">?attr/colorTertiaryContainer</item>
        <item name="rippleColor">?attr/colorTertiary</item>
    </style>

    <style name="ShapeAppearance.VangtiChai.Key" parent="">
        <item name="cornerFamily">rounded</item>
        <item name="cornerSize">@dimen/key_corner_radius</item>
    </style>

    <style name="ShapeAppearance.VangtiChai.Circle" parent="">
        <item name="cornerFamily">rounded</item>
```

app/src/main/res/values/styles.xml (continued)

```
        <item name="cornerSize">50%</item>
    </style>
</resources>
```

A.6 Layouts

app/src/main/res/layout/**view_note_row.xml** 51 lines · XML

ABOUT THIS FILE

One row of the change table (Part 3, Step 12), included eight times by every `activity_main.xml`.

- **Lines 6–14**: the row itself, a `ConstraintLayout` with the rounded background that `setRowLevel()` colours. `importantForAccessibility="yes"` (line 12) makes TalkBack treat the whole row as one item, read from the content description `renderRow()` sets.
- **Lines 16–24,** `note_accent`: the colour tab, 4 dp wide (6 dp on tablets) and half the row's height (`note_accent_height_percent`), centred by its top and bottom constraints.
- **Lines 26–36,** `note_label`: "500:", placed after the tab. `onCreate()` fills in the text.
- **Lines 38–49,** `note_count`: the count. `@dimen/match_constraint` (`0dp`) for both width and height stretches it from the label to the row's end and over the row's full height: the fixed box auto-sizing needs.

Both text views set `importantForAccessibility="no"` (lines 31 and 44) so TalkBack does not read them a second time. `tools:text` (lines 36 and 49) only fills Android Studio's preview.

app/src/main/res/layout/view_note_row.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    One row of the change table: a colour tab, the note ("500:") and how many of that note make
    up the change. Included eight times by every activity_main layout; MainActivity fills it in.
-->
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/
    android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@drawable/bg_note_row"
    android:importantForAccessibility="yes"
    android:paddingStart="@dimen/note_row_padding_start"
    android:paddingEnd="@dimen/note_row_padding_end">

    <View
        android:id="@+id/note_accent"
        android:layout_width="@dimen/note_accent_width"
        android:layout_height="@dimen/match_constraint"
        android:background="@drawable/bg_note_accent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintHeight_percent="@dimen/note_accent_height_percent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/note_label"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="@dimen/note_label_margin_start"
        android:importantForAccessibility="no"
        android:textAppearance="@style/TextAppearance.VangtiChai.NoteLabel"
        app:layout_constraintBottom_toBottomOf="parent"
```

app/src/main/res/layout/view_note_row.xml (continued)

```
        app:layout_constraintStart_toEndOf="@id/note_accent"
        app:layout_constraintTop_toTopOf="parent"
        tools:text="500:" />

    <TextView
        android:id="@+id/note_count"
        style="@style/Widget.VangtiChai.NoteCount"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:layout_marginStart="@dimen/note_count_margin_start"
        android:importantForAccessibility="no"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/note_label"
        app:layout_constraintTop_toTopOf="parent"
        tools:text="1" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

app/src/main/res/layout/**activity_main.xml** 379 lines · XML

ABOUT THIS FILE

The phone portrait layout (Part 3, Step 13; Figure 3 shows the views on screen). A map of the listing:

Lines	View (id)	Role
7–14	`root`	The root `ConstraintLayout`; line 13 forces left-to-right
21–89	`header`	The green header: 32% of the height, but never less than its content
34–42	`toolbar`	`MaterialToolbar` with the app name
44–57	`amount_text`	"Taka: 688"; head of the packed vertical chain (line 56)
61–72	`key_backspace`	Centred on the amount row; starts invisible (line 68)
74–87	`caption_text`	"8 notes"; tail of the chain, tied to the header's bottom
92–97	`split`	Vertical guideline at `notes_width_percent` (40%)
101–224	`notes_card`	The card left of `split`, capped at `content_max_height`
121–132	`notes_title`	"CHANGE"
135–221	`note_500` … `note_1`	Eight `<include>`d rows in a vertical chain
235–377	`keypad`	Its top and bottom are tied to the card's (lines 241 and 244)
246–375	`key_1` … `key_clear`	The 11 keys

Reading one key. `key_5` (lines 293–303) has four constraints. Its start and end attach to `key_4`'s end and `key_6`'s start, which makes it the middle link of row 2's horizontal chain. Its top and bottom attach to `key_4`'s top and bottom, so it is exactly as tall as the first key of its row. `layout_marginStart="@dimen/key_gap"` puts the gap on its left. Part 2.9 works out what all this means in dp on a Pixel XL.

app/src/main/res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    Phone, portrait (designed on a Pixel XL, 411 x 731 dp).
    The amount is shown on top. Below it, the change table fills the left column and a
    3-column keypad sits on the right. Sizes come from values/sizes.xml.
-->
```

app/src/main/res/layout/activity_main.xml (continued)

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/
    android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layoutDirection="ltr"
    tools:context=".MainActivity">

    <!--
        Header: app bar and amount. Drawn behind the status bar; MainActivity pads it.
        It takes header_height_percent of the screen height, but never less than its content,
        and the amount stays centred in whatever room is below the app bar.
    -->
    <androidx.constraintlayout.widget.ConstraintLayout
        android:id="@+id/header"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:background="@drawable/bg_header"
        android:paddingBottom="@dimen/header_padding_bottom"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHeight_default="percent"
        app:layout_constraintHeight_min="wrap"
        app:layout_constraintHeight_percent="@dimen/header_height_percent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            style="@style/Widget.VangtiChai.Toolbar"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="?attr/actionBarSize"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:title="@string/app_name" />

        <TextView
            android:id="@+id/amount_text"
            style="@style/Widget.VangtiChai.Amount"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/amount_height"
            android:layout_marginStart="@dimen/amount_side_margin"
            android:layout_marginEnd="@dimen/amount_side_margin"
            android:text="@string/label_taka"
            app:layout_constraintBottom_toTopOf="@id/caption_text"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/toolbar"
            app:layout_constraintVertical_chainStyle="packed"
            tools:text="Taka: 688" />

        <!-- Backspace: deletes the last digit, and keeps deleting while held. Hidden until
             something is typed. -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_backspace"
            style="@style/Widget.VangtiChai.Backspace"
            android:layout_width="@dimen/backspace_size"
            android:layout_height="@dimen/backspace_size"
            android:layout_marginEnd="@dimen/backspace_margin_end"
            android:contentDescription="@string/cd_key_backspace"
            android:visibility="invisible"
            app:layout_constraintBottom_toBottomOf="@id/amount_text"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintTop_toTopOf="@id/amount_text"
            tools:visibility="visible" />

        <TextView
            android:id="@+id/caption_text"
```

app/src/main/res/layout/activity_main.xml (continued)

```
            style="@style/Widget.VangtiChai.Caption"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="wrap_content"
            android:layout_marginStart="@dimen/header_padding_horizontal"
            android:layout_marginTop="@dimen/caption_margin_top"
            android:layout_marginEnd="@dimen/header_padding_horizontal"
            android:text="@string/hint_enter_amount"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/amount_text"
            tools:text="8 notes" />

    </androidx.constraintlayout.widget.ConstraintLayout>

    <!-- The change table ends here and the keypad begins -->
    <androidx.constraintlayout.widget.Guideline
        android:id="@+id/split"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintGuide_percent="@dimen/notes_width_percent" />

    <!-- Change table: notes from 500 down to 1 in a single column. On very tall screens it stops
         growing at content_max_height and sits at content_vertical_bias in the space left. -->
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/notes_card"
        style="@style/Widget.VangtiChai.Card"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:layout_marginStart="@dimen/screen_margin"
        android:layout_marginTop="@dimen/section_gap"
        android:layout_marginBottom="@dimen/screen_margin"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@id/split"
        app:layout_constraintHeight_max="@dimen/content_max_height"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/header"
        app:layout_constraintVertical_bias="@dimen/content_vertical_bias">

        <androidx.constraintlayout.widget.ConstraintLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="@dimen/notes_card_padding">

            <TextView
                android:id="@+id/notes_title"
                style="@style/Widget.VangtiChai.NotesTitle"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="wrap_content"
                android:layout_marginStart="@dimen/notes_title_margin_horizontal"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                android:layout_marginEnd="@dimen/notes_title_margin_horizontal"
                android:text="@string/notes_title"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toTopOf="parent" />

            <!-- Eight equally tall rows, chained from the title down to the bottom of the card -->
            <include
                android:id="@+id/note_500"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                app:layout_constraintBottom_toTopOf="@id/note_100"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/notes_title" />

```

app/src/main/res/layout/activity_main.xml (continued)

```
            <include
                android:id="@+id/note_100"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_50"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_500" />

            <include
                android:id="@+id/note_50"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_20"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_100" />

            <include
                android:id="@+id/note_20"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_10"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_50" />

            <include
                android:id="@+id/note_10"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_5"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_20" />

            <include
                android:id="@+id/note_5"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_2"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_10" />

            <include
                android:id="@+id/note_2"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_1"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_5" />

            <include
                android:id="@+id/note_1"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
```

app/src/main/res/layout/activity_main.xml (continued)

```
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toBottomOf="parent"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_2" />

        </androidx.constraintlayout.widget.ConstraintLayout>
    </com.google.android.material.card.MaterialCardView>

    <!--
        Keypad, 3 columns:   1 2 3
                             4 5 6
                             7 8 9
                             0 CLEAR
        Rows 1-3 are horizontal chains; the first column is a vertical chain. The bottom row lines
        up with the columns above it, so CLEAR spans exactly the 2nd and 3rd columns.
        The keypad is exactly as tall as the change table next to it.
    -->
    <androidx.constraintlayout.widget.ConstraintLayout
        android:id="@+id/keypad"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:layout_marginStart="@dimen/column_gap"
        android:layout_marginEnd="@dimen/screen_margin"
        app:layout_constraintBottom_toBottomOf="@id/notes_card"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/split"
        app:layout_constraintTop_toTopOf="@id/notes_card">

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_1"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:text="@string/key_1"
            app:layout_constraintBottom_toTopOf="@id/key_4"
            app:layout_constraintEnd_toStartOf="@id/key_2"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_2"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_2"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
            app:layout_constraintEnd_toStartOf="@id/key_3"
            app:layout_constraintStart_toEndOf="@id/key_1"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_3"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_3"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/key_2"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_4"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
```

app/src/main/res/layout/activity_main.xml (continued)

```
            android:layout_marginTop="@dimen/key_gap"
            android:text="@string/key_4"
            app:layout_constraintBottom_toTopOf="@id/key_7"
            app:layout_constraintEnd_toStartOf="@id/key_5"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_5"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_5"
            app:layout_constraintBottom_toBottomOf="@id/key_4"
            app:layout_constraintEnd_toStartOf="@id/key_6"
            app:layout_constraintStart_toEndOf="@id/key_4"
            app:layout_constraintTop_toTopOf="@id/key_4" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_6"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_6"
            app:layout_constraintBottom_toBottomOf="@id/key_4"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/key_5"
            app:layout_constraintTop_toTopOf="@id/key_4" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_7"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginTop="@dimen/key_gap"
            android:text="@string/key_7"
            app:layout_constraintBottom_toTopOf="@id/key_0"
            app:layout_constraintEnd_toStartOf="@id/key_8"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/key_4" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_8"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_8"
            app:layout_constraintBottom_toBottomOf="@id/key_7"
            app:layout_constraintEnd_toStartOf="@id/key_9"
            app:layout_constraintStart_toEndOf="@id/key_7"
            app:layout_constraintTop_toTopOf="@id/key_7" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_9"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_9"
            app:layout_constraintBottom_toBottomOf="@id/key_7"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/key_8"
            app:layout_constraintTop_toTopOf="@id/key_7" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_0"
            style="@style/Widget.VangtiChai.Key"
```

app/src/main/res/layout/activity_main.xml (continued)

```
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginTop="@dimen/key_gap"
            android:text="@string/key_0"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="@id/key_7"
            app:layout_constraintStart_toStartOf="@id/key_7"
            app:layout_constraintTop_toBottomOf="@id/key_7" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_clear"
            style="@style/Widget.VangtiChai.Key.Clear"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:contentDescription="@string/cd_key_clear"
            android:text="@string/key_clear"
            app:layout_constraintBottom_toBottomOf="@id/key_0"
            app:layout_constraintEnd_toEndOf="@id/key_9"
            app:layout_constraintStart_toStartOf="@id/key_8"
            app:layout_constraintTop_toTopOf="@id/key_0" />

    </androidx.constraintlayout.widget.ConstraintLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

app/src/main/res/layout-land/activity_main.xml

381 lines · XML

ABOUT THIS FILE

The phone landscape layout (Part 3, Step 14; Figure 4). What changes from portrait:

- **Header** (lines 18–80): `wrap_content` height instead of 32% of the screen, so it is only as tall as the toolbar and the amount.
- **Caption** (lines 39–50): moves up into the toolbar row, tied to the toolbar's top and bottom and the header's end, with `gravity="end"`.
- **Amount** (lines 52–63): no longer in a chain; it sits under the toolbar.
- **Change card** (lines 91–229): no `content_max_height` cap and no bias, so it always fills the height under the header.
- **Two columns of rows**: the `notes_columns` guideline (lines 123–128) at 50% (`@dimen/half`) splits the card. The left chain runs from `note_500` down to `note_20` and the right chain from `note_10` down to `note_1`, each from the title to the bottom, with `note_column_half_gap` on either side of the guideline.
- **Keypad** (lines 238–379): four columns, `1 2 3 4` / `5 6 7 8` / `9 0 CLEAR`. The first column's vertical chain is `key_1`, `key_5`, `key_9`; `key_0` copies `key_6`'s column, and CLEAR runs from `key_7`'s start to `key_8`'s end.

Every id matches the portrait layout, so the same Kotlin code drives both.

app/src/main/res/layout-land/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    Phone, Landscape (designed on a Pixel XL, 731 x 411 dp).
    Height is scarce, so the caption moves up into the app bar row, the change table is split
    into two columns of four notes, and the keypad becomes 4 columns x 3 rows.
    Sizes come from values-land/sizes.xml.
-->
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/
    android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/root"
```

app/src/main/res/layout-land/activity_main.xml (continued)

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layoutDirection="ltr"
    tools:context=".MainActivity">

    <!-- Header: app bar and amount. Drawn behind the status bar; MainActivity pads it. -->
    <androidx.constraintlayout.widget.ConstraintLayout
        android:id="@+id/header"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="wrap_content"
        android:background="@drawable/bg_header"
        android:paddingBottom="@dimen/header_padding_bottom"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            style="@style/Widget.VangtiChai.Toolbar"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="?attr/actionBarSize"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:title="@string/app_name" />

        <!-- Caption shares the app bar row, opposite the title -->
        <TextView
            android:id="@+id/caption_text"
            style="@style/Widget.VangtiChai.Caption"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="@dimen/header_padding_horizontal"
            android:gravity="end"
            android:text="@string/hint_enter_amount"
            app:layout_constraintBottom_toBottomOf="@id/toolbar"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintTop_toTopOf="@id/toolbar"
            tools:text="8 notes" />

        <TextView
            android:id="@+id/amount_text"
            style="@style/Widget.VangtiChai.Amount"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/amount_height"
            android:layout_marginStart="@dimen/amount_side_margin"
            android:layout_marginEnd="@dimen/amount_side_margin"
            android:text="@string/label_taka"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/toolbar"
            tools:text="Taka: 688" />

        <!-- Backspace: deletes the last digit, and keeps deleting while held. Hidden until
             something is typed. -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_backspace"
            style="@style/Widget.VangtiChai.Backspace"
            android:layout_width="@dimen/backspace_size"
            android:layout_height="@dimen/backspace_size"
            android:layout_marginEnd="@dimen/backspace_margin_end"
            android:contentDescription="@string/cd_key_backspace"
            android:visibility="invisible"
            app:layout_constraintBottom_toBottomOf="@id/amount_text"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintTop_toTopOf="@id/amount_text"
            tools:visibility="visible" />

    </androidx.constraintlayout.widget.ConstraintLayout>

```

app/src/main/res/layout-land/activity_main.xml (continued)

```
    <!-- The change table ends here and the keypad begins -->
    <androidx.constraintlayout.widget.Guideline
        android:id="@+id/split"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintGuide_percent="@dimen/notes_width_percent" />

    <!-- Change table: 500, 100, 50, 20 on the left; 10, 5, 2, 1 on the right -->
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/notes_card"
        style="@style/Widget.VangtiChai.Card"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:layout_marginStart="@dimen/screen_margin"
        android:layout_marginTop="@dimen/section_gap"
        android:layout_marginBottom="@dimen/screen_margin"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@id/split"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/header">

        <androidx.constraintlayout.widget.ConstraintLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="@dimen/notes_card_padding">

            <TextView
                android:id="@+id/notes_title"
                style="@style/Widget.VangtiChai.NotesTitle"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="wrap_content"
                android:layout_marginStart="@dimen/notes_title_margin_horizontal"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                android:layout_marginEnd="@dimen/notes_title_margin_horizontal"
                android:text="@string/notes_title"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toTopOf="parent" />

            <!-- Splits the card into two equal columns -->
            <androidx.constraintlayout.widget.Guideline
                android:id="@+id/notes_columns"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                app:layout_constraintGuide_percent="@dimen/half" />

            <!-- Left column -->
            <include
                android:id="@+id/note_500"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                android:layout_marginEnd="@dimen/note_column_half_gap"
                app:layout_constraintBottom_toTopOf="@id/note_100"
                app:layout_constraintEnd_toStartOf="@id/notes_columns"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/notes_title" />

            <include
                android:id="@+id/note_100"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                android:layout_marginEnd="@dimen/note_column_half_gap"
                app:layout_constraintBottom_toTopOf="@id/note_50"
                app:layout_constraintEnd_toStartOf="@id/notes_columns"
```

app/src/main/res/layout-land/activity_main.xml (continued)

```
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_500" />

            <include
                android:id="@+id/note_50"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                android:layout_marginEnd="@dimen/note_column_half_gap"
                app:layout_constraintBottom_toTopOf="@id/note_20"
                app:layout_constraintEnd_toStartOf="@id/notes_columns"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_100" />

            <include
                android:id="@+id/note_20"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                android:layout_marginEnd="@dimen/note_column_half_gap"
                app:layout_constraintBottom_toBottomOf="parent"
                app:layout_constraintEnd_toStartOf="@id/notes_columns"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_50" />

            <!-- Right column -->
            <include
                android:id="@+id/note_10"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginStart="@dimen/note_column_half_gap"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                app:layout_constraintBottom_toTopOf="@id/note_5"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toEndOf="@id/notes_columns"
                app:layout_constraintTop_toBottomOf="@id/notes_title" />

            <include
                android:id="@+id/note_5"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginStart="@dimen/note_column_half_gap"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_2"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toEndOf="@id/notes_columns"
                app:layout_constraintTop_toBottomOf="@id/note_10" />

            <include
                android:id="@+id/note_2"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginStart="@dimen/note_column_half_gap"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_1"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toEndOf="@id/notes_columns"
                app:layout_constraintTop_toBottomOf="@id/note_5" />

            <include
                android:id="@+id/note_1"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginStart="@dimen/note_column_half_gap"
```

app/src/main/res/layout-land/activity_main.xml (continued)

```
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toBottomOf="parent"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toEndOf="@id/notes_columns"
                app:layout_constraintTop_toBottomOf="@id/note_2" />

        </androidx.constraintlayout.widget.ConstraintLayout>
    </com.google.android.material.card.MaterialCardView>

    <!--
        Keypad, 4 columns:   1 2 3 4
                             5 6 7 8
                             9 0 CLEAR
        Rows 1-2 are horizontal chains; the first column is a vertical chain. 0 lines up with the
        2nd column and CLEAR spans the 3rd and 4th. The keypad is as tall as the change table.
    -->
    <androidx.constraintlayout.widget.ConstraintLayout
        android:id="@+id/keypad"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:layout_marginStart="@dimen/column_gap"
        android:layout_marginEnd="@dimen/screen_margin"
        app:layout_constraintBottom_toBottomOf="@id/notes_card"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/split"
        app:layout_constraintTop_toTopOf="@id/notes_card">

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_1"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:text="@string/key_1"
            app:layout_constraintBottom_toTopOf="@id/key_5"
            app:layout_constraintEnd_toStartOf="@id/key_2"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_2"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_2"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
            app:layout_constraintEnd_toStartOf="@id/key_3"
            app:layout_constraintStart_toEndOf="@id/key_1"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_3"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_3"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
            app:layout_constraintEnd_toStartOf="@id/key_4"
            app:layout_constraintStart_toEndOf="@id/key_2"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_4"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_4"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
```

app/src/main/res/layout-land/activity_main.xml (continued)

```
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/key_3"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_5"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginTop="@dimen/key_gap"
            android:text="@string/key_5"
            app:layout_constraintBottom_toTopOf="@id/key_9"
            app:layout_constraintEnd_toStartOf="@id/key_6"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_6"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_6"
            app:layout_constraintBottom_toBottomOf="@id/key_5"
            app:layout_constraintEnd_toStartOf="@id/key_7"
            app:layout_constraintStart_toEndOf="@id/key_5"
            app:layout_constraintTop_toTopOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_7"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_7"
            app:layout_constraintBottom_toBottomOf="@id/key_5"
            app:layout_constraintEnd_toStartOf="@id/key_8"
            app:layout_constraintStart_toEndOf="@id/key_6"
            app:layout_constraintTop_toTopOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_8"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_8"
            app:layout_constraintBottom_toBottomOf="@id/key_5"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/key_7"
            app:layout_constraintTop_toTopOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_9"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginTop="@dimen/key_gap"
            android:text="@string/key_9"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="@id/key_5"
            app:layout_constraintStart_toStartOf="@id/key_5"
            app:layout_constraintTop_toBottomOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_0"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:text="@string/key_0"
```

app/src/main/res/layout-land/activity_main.xml (continued)

```
            app:layout_constraintBottom_toBottomOf="@id/key_9"
            app:layout_constraintEnd_toEndOf="@id/key_6"
            app:layout_constraintStart_toStartOf="@id/key_6"
            app:layout_constraintTop_toTopOf="@id/key_9" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_clear"
            style="@style/Widget.VangtiChai.Key.Clear"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:contentDescription="@string/cd_key_clear"
            android:text="@string/key_clear"
            app:layout_constraintBottom_toBottomOf="@id/key_9"
            app:layout_constraintEnd_toEndOf="@id/key_8"
            app:layout_constraintStart_toStartOf="@id/key_7"
            app:layout_constraintTop_toTopOf="@id/key_9" />

    </androidx.constraintlayout.widget.ConstraintLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

app/src/main/res/layout-sw600dp/**activity_main.xml** 392 lines · XML

ABOUT THIS FILE

Tablet portrait (Part 3, Step 15; Figure 5, left). It is the phone portrait layout with three changes:

- **Toolbar** (lines 35–44): its height comes from `@dimen/toolbar_height` (76 dp) instead of `?attr/actionBarSize`, and it shows `app:subtitle="@string/app_tagline"`.
- **"CHANGE" title** (lines 123–134): its end now attaches to the start of the new heading, so the two can never overlap.
- `notes_count_heading` (lines 137–145): "COUNT", in the title's style, lined up with the title's baseline and with the right edge of the counts (`note_row_padding_end`).

Every size comes from `values-sw600dp/sizes.xml`, falling back to `values/sizes.xml`.

app/src/main/res/layout-sw600dp/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    Tablet, portrait (designed on a Nexus 10, 800 x 1280 dp).
    Same arrangement as the phone in portrait, scaled up: the amount on top, the change table on
    the left and a 3-column keypad on the right. The extra room adds a subtitle to the app bar
    and a "Count" heading to the change table. Sizes come from values-sw600dp/sizes.xml.
-->
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/
    android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layoutDirection="ltr"
    tools:context=".MainActivity">

    <!--
        Header: app bar and amount. Drawn behind the status bar; MainActivity pads it.
        It takes header_height_percent of the screen height, but never less than its content,
        and the amount stays centred in whatever room is below the app bar.
    -->
    <androidx.constraintlayout.widget.ConstraintLayout
        android:id="@+id/header"
        android:layout_width="@dimen/match_constraint"
```

app/src/main/res/layout-sw600dp/activity_main.xml (continued)

```
        android:layout_height="@dimen/match_constraint"
        android:background="@drawable/bg_header"
        android:paddingBottom="@dimen/header_padding_bottom"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHeight_default="percent"
        app:layout_constraintHeight_min="wrap"
        app:layout_constraintHeight_percent="@dimen/header_height_percent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            style="@style/Widget.VangtiChai.Toolbar"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/toolbar_height"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:subtitle="@string/app_tagline"
            app:title="@string/app_name" />

        <TextView
            android:id="@+id/amount_text"
            style="@style/Widget.VangtiChai.Amount"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/amount_height"
            android:layout_marginStart="@dimen/amount_side_margin"
            android:layout_marginEnd="@dimen/amount_side_margin"
            android:text="@string/label_taka"
            app:layout_constraintBottom_toTopOf="@id/caption_text"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/toolbar"
            app:layout_constraintVertical_chainStyle="packed"
            tools:text="Taka: 688" />

        <!-- Backspace: deletes the last digit, and keeps deleting while held. Hidden until
             something is typed. -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_backspace"
            style="@style/Widget.VangtiChai.Backspace"
            android:layout_width="@dimen/backspace_size"
            android:layout_height="@dimen/backspace_size"
            android:layout_marginEnd="@dimen/backspace_margin_end"
            android:contentDescription="@string/cd_key_backspace"
            android:visibility="invisible"
            app:layout_constraintBottom_toBottomOf="@id/amount_text"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintTop_toTopOf="@id/amount_text"
            tools:visibility="visible" />

        <TextView
            android:id="@+id/caption_text"
            style="@style/Widget.VangtiChai.Caption"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="wrap_content"
            android:layout_marginStart="@dimen/header_padding_horizontal"
            android:layout_marginTop="@dimen/caption_margin_top"
            android:layout_marginEnd="@dimen/header_padding_horizontal"
            android:text="@string/hint_enter_amount"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/amount_text"
            tools:text="8 notes" />

    </androidx.constraintlayout.widget.ConstraintLayout>

    <!-- The change table ends here and the keypad begins -->
    <androidx.constraintlayout.widget.Guideline
```

app/src/main/res/layout-sw600dp/activity_main.xml (continued)

```
        android:id="@+id/split"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintGuide_percent="@dimen/notes_width_percent" />

    <!-- Change table: notes from 500 down to 1 in a single column. On very tall screens it stops
         growing at content_max_height and sits at content_vertical_bias in the space left. -->
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/notes_card"
        style="@style/Widget.VangtiChai.Card"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:layout_marginStart="@dimen/screen_margin"
        android:layout_marginTop="@dimen/section_gap"
        android:layout_marginBottom="@dimen/screen_margin"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@id/split"
        app:layout_constraintHeight_max="@dimen/content_max_height"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/header"
        app:layout_constraintVertical_bias="@dimen/content_vertical_bias">

        <androidx.constraintlayout.widget.ConstraintLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="@dimen/notes_card_padding">

            <TextView
                android:id="@+id/notes_title"
                style="@style/Widget.VangtiChai.NotesTitle"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="wrap_content"
                android:layout_marginStart="@dimen/notes_title_margin_horizontal"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                android:layout_marginEnd="@dimen/notes_title_margin_horizontal"
                android:text="@string/notes_title"
                app:layout_constraintEnd_toStartOf="@id/notes_count_heading"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toTopOf="parent" />

            <!-- Heading over the counts, lined up with them on the right -->
            <TextView
                android:id="@+id/notes_count_heading"
                style="@style/Widget.VangtiChai.NotesTitle"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginEnd="@dimen/note_row_padding_end"
                android:text="@string/notes_count_heading"
                app:layout_constraintBaseline_toBaselineOf="@id/notes_title"
                app:layout_constraintEnd_toEndOf="parent" />

            <!-- Eight equally tall rows, chained from the title down to the bottom of the card -->
            <include
                android:id="@+id/note_500"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                app:layout_constraintBottom_toTopOf="@id/note_100"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/notes_title" />

            <include
                android:id="@+id/note_100"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
```

app/src/main/res/layout-sw600dp/activity_main.xml (continued)

```
                app:layout_constraintBottom_toTopOf="@id/note_50"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_500" />

            <include
                android:id="@+id/note_50"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_20"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_100" />

            <include
                android:id="@+id/note_20"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_10"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_50" />

            <include
                android:id="@+id/note_10"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_5"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_20" />

            <include
                android:id="@+id/note_5"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_2"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_10" />

            <include
                android:id="@+id/note_2"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toTopOf="@id/note_1"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_5" />

            <include
                android:id="@+id/note_1"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                app:layout_constraintBottom_toBottomOf="parent"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_2" />
```

app/src/main/res/layout-sw600dp/activity_main.xml (continued)

```
235
236         </androidx.constraintlayout.widget.ConstraintLayout>
237     </com.google.android.material.card.MaterialCardView>
238
239     <!--
240         Keypad, 3 columns:   1 2 3
241                              4 5 6
242                              7 8 9
243                              0 CLEAR
244         Rows 1-3 are horizontal chains; the first column is a vertical chain. The bottom row lines
245         up with the columns above it, so CLEAR spans exactly the 2nd and 3rd columns.
246         The keypad is exactly as tall as the change table next to it.
247     -->
248     <androidx.constraintlayout.widget.ConstraintLayout
249         android:id="@+id/keypad"
250         android:layout_width="@dimen/match_constraint"
251         android:layout_height="@dimen/match_constraint"
252         android:layout_marginStart="@dimen/column_gap"
253         android:layout_marginEnd="@dimen/screen_margin"
254         app:layout_constraintBottom_toBottomOf="@id/notes_card"
255         app:layout_constraintEnd_toEndOf="parent"
256         app:layout_constraintStart_toEndOf="@id/split"
257         app:layout_constraintTop_toTopOf="@id/notes_card">
258
259         <com.google.android.material.button.MaterialButton
260             android:id="@+id/key_1"
261             style="@style/Widget.VangtiChai.Key"
262             android:layout_width="@dimen/match_constraint"
263             android:layout_height="@dimen/match_constraint"
264             android:text="@string/key_1"
265             app:layout_constraintBottom_toTopOf="@id/key_4"
266             app:layout_constraintEnd_toStartOf="@id/key_2"
267             app:layout_constraintStart_toStartOf="parent"
268             app:layout_constraintTop_toTopOf="parent" />
269
270         <com.google.android.material.button.MaterialButton
271             android:id="@+id/key_2"
272             style="@style/Widget.VangtiChai.Key"
273             android:layout_width="@dimen/match_constraint"
274             android:layout_height="@dimen/match_constraint"
275             android:layout_marginStart="@dimen/key_gap"
276             android:text="@string/key_2"
277             app:layout_constraintBottom_toBottomOf="@id/key_1"
278             app:layout_constraintEnd_toStartOf="@id/key_3"
279             app:layout_constraintStart_toEndOf="@id/key_1"
280             app:layout_constraintTop_toTopOf="@id/key_1" />
281
282         <com.google.android.material.button.MaterialButton
283             android:id="@+id/key_3"
284             style="@style/Widget.VangtiChai.Key"
285             android:layout_width="@dimen/match_constraint"
286             android:layout_height="@dimen/match_constraint"
287             android:layout_marginStart="@dimen/key_gap"
288             android:text="@string/key_3"
289             app:layout_constraintBottom_toBottomOf="@id/key_1"
290             app:layout_constraintEnd_toEndOf="parent"
291             app:layout_constraintStart_toEndOf="@id/key_2"
292             app:layout_constraintTop_toTopOf="@id/key_1" />
293
294         <com.google.android.material.button.MaterialButton
295             android:id="@+id/key_4"
296             style="@style/Widget.VangtiChai.Key"
297             android:layout_width="@dimen/match_constraint"
298             android:layout_height="@dimen/match_constraint"
299             android:layout_marginTop="@dimen/key_gap"
300             android:text="@string/key_4"
301             app:layout_constraintBottom_toTopOf="@id/key_7"
302             app:layout_constraintEnd_toStartOf="@id/key_5"
303             app:layout_constraintStart_toStartOf="parent"
304             app:layout_constraintTop_toBottomOf="@id/key_1" />
```

app/src/main/res/layout-sw600dp/activity_main.xml (continued)

```
305
306         <com.google.android.material.button.MaterialButton
307             android:id="@+id/key_5"
308             style="@style/Widget.VangtiChai.Key"
309             android:layout_width="@dimen/match_constraint"
310             android:layout_height="@dimen/match_constraint"
311             android:layout_marginStart="@dimen/key_gap"
312             android:text="@string/key_5"
313             app:layout_constraintBottom_toBottomOf="@id/key_4"
314             app:layout_constraintEnd_toStartOf="@id/key_6"
315             app:layout_constraintStart_toEndOf="@id/key_4"
316             app:layout_constraintTop_toTopOf="@id/key_4" />
317
318         <com.google.android.material.button.MaterialButton
319             android:id="@+id/key_6"
320             style="@style/Widget.VangtiChai.Key"
321             android:layout_width="@dimen/match_constraint"
322             android:layout_height="@dimen/match_constraint"
323             android:layout_marginStart="@dimen/key_gap"
324             android:text="@string/key_6"
325             app:layout_constraintBottom_toBottomOf="@id/key_4"
326             app:layout_constraintEnd_toEndOf="parent"
327             app:layout_constraintStart_toEndOf="@id/key_5"
328             app:layout_constraintTop_toTopOf="@id/key_4" />
329
330         <com.google.android.material.button.MaterialButton
331             android:id="@+id/key_7"
332             style="@style/Widget.VangtiChai.Key"
333             android:layout_width="@dimen/match_constraint"
334             android:layout_height="@dimen/match_constraint"
335             android:layout_marginTop="@dimen/key_gap"
336             android:text="@string/key_7"
337             app:layout_constraintBottom_toTopOf="@id/key_0"
338             app:layout_constraintEnd_toStartOf="@id/key_8"
339             app:layout_constraintStart_toStartOf="parent"
340             app:layout_constraintTop_toBottomOf="@id/key_4" />
341
342         <com.google.android.material.button.MaterialButton
343             android:id="@+id/key_8"
344             style="@style/Widget.VangtiChai.Key"
345             android:layout_width="@dimen/match_constraint"
346             android:layout_height="@dimen/match_constraint"
347             android:layout_marginStart="@dimen/key_gap"
348             android:text="@string/key_8"
349             app:layout_constraintBottom_toBottomOf="@id/key_7"
350             app:layout_constraintEnd_toStartOf="@id/key_9"
351             app:layout_constraintStart_toEndOf="@id/key_7"
352             app:layout_constraintTop_toTopOf="@id/key_7" />
353
354         <com.google.android.material.button.MaterialButton
355             android:id="@+id/key_9"
356             style="@style/Widget.VangtiChai.Key"
357             android:layout_width="@dimen/match_constraint"
358             android:layout_height="@dimen/match_constraint"
359             android:layout_marginStart="@dimen/key_gap"
360             android:text="@string/key_9"
361             app:layout_constraintBottom_toBottomOf="@id/key_7"
362             app:layout_constraintEnd_toEndOf="parent"
363             app:layout_constraintStart_toEndOf="@id/key_8"
364             app:layout_constraintTop_toTopOf="@id/key_7" />
365
366         <com.google.android.material.button.MaterialButton
367             android:id="@+id/key_0"
368             style="@style/Widget.VangtiChai.Key"
369             android:layout_width="@dimen/match_constraint"
370             android:layout_height="@dimen/match_constraint"
371             android:layout_marginTop="@dimen/key_gap"
372             android:text="@string/key_0"
373             app:layout_constraintBottom_toBottomOf="parent"
374             app:layout_constraintEnd_toEndOf="@id/key_7"
```

app/src/main/res/layout-sw600dp/activity_main.xml (continued)

```
            app:layout_constraintStart_toStartOf="@id/key_7"
            app:layout_constraintTop_toBottomOf="@id/key_7" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_clear"
            style="@style/Widget.VangtiChai.Key.Clear"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:contentDescription="@string/cd_key_clear"
            android:text="@string/key_clear"
            app:layout_constraintBottom_toBottomOf="@id/key_0"
            app:layout_constraintEnd_toEndOf="@id/key_9"
            app:layout_constraintStart_toStartOf="@id/key_8"
            app:layout_constraintTop_toTopOf="@id/key_0" />

    </androidx.constraintlayout.widget.ConstraintLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

app/src/main/res/layout-sw600dp-land/**activity_main.xml** 383 lines · XML

ABOUT THIS FILE

Tablet landscape (Part 3, Step 15; Figure 5, right). It starts from the phone landscape layout and changes only the header:

- **Toolbar** (lines 29–38): `toolbar_height` and the subtitle, as in tablet portrait.
- **Caption** (lines 68–80): back under the amount, as in portrait. Unlike portrait it is not tied to the header's bottom: the header wraps its content, so the caption simply sits below the amount.

Below the header (two-column table, four-column keypad) it is the same as phone landscape. Sizes come from `values-sw600dp-land/sizes.xml`, then `values-sw600dp/sizes.xml`, then `values/sizes.xml`.

app/src/main/res/layout-sw600dp-land/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    Tablet, Landscape (designed on a Nexus 10, 1280 x 800 dp).
    Like the phone in landscape, the change table has two columns of four notes and the keypad is
    4 columns x 3 rows. There is enough height for the caption to sit under the amount, as in
    portrait, and the app bar has a subtitle.
    Sizes come from values-sw600dp-land/sizes.xml.
-->
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/
    android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layoutDirection="ltr"
    tools:context=".MainActivity">

    <!-- Header: app bar and amount. Drawn behind the status bar; MainActivity pads it. -->
    <androidx.constraintlayout.widget.ConstraintLayout
        android:id="@+id/header"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="wrap_content"
        android:background="@drawable/bg_header"
        android:paddingBottom="@dimen/header_padding_bottom"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">
```

app/src/main/res/layout-sw600dp-land/activity_main.xml (continued)

```
28
29          <com.google.android.material.appbar.MaterialToolbar
30              android:id="@+id/toolbar"
31              style="@style/Widget.VangtiChai.Toolbar"
32              android:layout_width="@dimen/match_constraint"
33              android:layout_height="@dimen/toolbar_height"
34              app:layout_constraintEnd_toEndOf="parent"
35              app:layout_constraintStart_toStartOf="parent"
36              app:layout_constraintTop_toTopOf="parent"
37              app:subtitle="@string/app_tagline"
38              app:title="@string/app_name" />
39
40          <TextView
41              android:id="@+id/amount_text"
42              style="@style/Widget.VangtiChai.Amount"
43              android:layout_width="@dimen/match_constraint"
44              android:layout_height="@dimen/amount_height"
45              android:layout_marginStart="@dimen/amount_side_margin"
46              android:layout_marginEnd="@dimen/amount_side_margin"
47              android:text="@string/label_taka"
48              app:layout_constraintEnd_toEndOf="parent"
49              app:layout_constraintStart_toStartOf="parent"
50              app:layout_constraintTop_toBottomOf="@id/toolbar"
51              tools:text="Taka: 688" />
52
53          <!-- Backspace: deletes the last digit, and keeps deleting while held. Hidden until
54               something is typed. -->
55          <com.google.android.material.button.MaterialButton
56              android:id="@+id/key_backspace"
57              style="@style/Widget.VangtiChai.Backspace"
58              android:layout_width="@dimen/backspace_size"
59              android:layout_height="@dimen/backspace_size"
60              android:layout_marginEnd="@dimen/backspace_margin_end"
61              android:contentDescription="@string/cd_key_backspace"
62              android:visibility="invisible"
63              app:layout_constraintBottom_toBottomOf="@id/amount_text"
64              app:layout_constraintEnd_toEndOf="parent"
65              app:layout_constraintTop_toTopOf="@id/amount_text"
66              tools:visibility="visible" />
67
68          <TextView
69              android:id="@+id/caption_text"
70              style="@style/Widget.VangtiChai.Caption"
71              android:layout_width="@dimen/match_constraint"
72              android:layout_height="wrap_content"
73              android:layout_marginStart="@dimen/header_padding_horizontal"
74              android:layout_marginTop="@dimen/caption_margin_top"
75              android:layout_marginEnd="@dimen/header_padding_horizontal"
76              android:text="@string/hint_enter_amount"
77              app:layout_constraintEnd_toEndOf="parent"
78              app:layout_constraintStart_toStartOf="parent"
79              app:layout_constraintTop_toBottomOf="@id/amount_text"
80              tools:text="19 notes" />
81
82      </androidx.constraintlayout.widget.ConstraintLayout>
83
84      <!-- The change table ends here and the keypad begins -->
85      <androidx.constraintlayout.widget.Guideline
86          android:id="@+id/split"
87          android:layout_width="wrap_content"
88          android:layout_height="wrap_content"
89          android:orientation="vertical"
90          app:layout_constraintGuide_percent="@dimen/notes_width_percent" />
91
92      <!-- Change table: 500, 100, 50, 20 on the left; 10, 5, 2, 1 on the right -->
93      <com.google.android.material.card.MaterialCardView
94          android:id="@+id/notes_card"
95          style="@style/Widget.VangtiChai.Card"
96          android:layout_width="@dimen/match_constraint"
97          android:layout_height="@dimen/match_constraint"
```

app/src/main/res/layout-sw600dp-land/activity_main.xml (continued)

```
        android:layout_marginStart="@dimen/screen_margin"
        android:layout_marginTop="@dimen/section_gap"
        android:layout_marginBottom="@dimen/screen_margin"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@id/split"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/header">

        <androidx.constraintlayout.widget.ConstraintLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="@dimen/notes_card_padding">

            <TextView
                android:id="@+id/notes_title"
                style="@style/Widget.VangtiChai.NotesTitle"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="wrap_content"
                android:layout_marginStart="@dimen/notes_title_margin_horizontal"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                android:layout_marginEnd="@dimen/notes_title_margin_horizontal"
                android:text="@string/notes_title"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toTopOf="parent" />

            <!-- Splits the card into two equal columns -->
            <androidx.constraintlayout.widget.Guideline
                android:id="@+id/notes_columns"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                app:layout_constraintGuide_percent="@dimen/half" />

            <!-- Left column -->
            <include
                android:id="@+id/note_500"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/notes_title_margin_vertical"
                android:layout_marginEnd="@dimen/note_column_half_gap"
                app:layout_constraintBottom_toTopOf="@id/note_100"
                app:layout_constraintEnd_toStartOf="@id/notes_columns"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/notes_title" />

            <include
                android:id="@+id/note_100"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                android:layout_marginEnd="@dimen/note_column_half_gap"
                app:layout_constraintBottom_toTopOf="@id/note_50"
                app:layout_constraintEnd_toStartOf="@id/notes_columns"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_500" />

            <include
                android:id="@+id/note_50"
                layout="@layout/view_note_row"
                android:layout_width="@dimen/match_constraint"
                android:layout_height="@dimen/match_constraint"
                android:layout_marginTop="@dimen/note_row_gap"
                android:layout_marginEnd="@dimen/note_column_half_gap"
                app:layout_constraintBottom_toTopOf="@id/note_20"
                app:layout_constraintEnd_toStartOf="@id/notes_columns"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toBottomOf="@id/note_100" />
```

app/src/main/res/layout-sw600dp-land/activity_main.xml (continued)

```
168
169             <include
170                 android:id="@+id/note_20"
171                 layout="@layout/view_note_row"
172                 android:layout_width="@dimen/match_constraint"
173                 android:layout_height="@dimen/match_constraint"
174                 android:layout_marginTop="@dimen/note_row_gap"
175                 android:layout_marginEnd="@dimen/note_column_half_gap"
176                 app:layout_constraintBottom_toBottomOf="parent"
177                 app:layout_constraintEnd_toStartOf="@id/notes_columns"
178                 app:layout_constraintStart_toStartOf="parent"
179                 app:layout_constraintTop_toBottomOf="@id/note_50" />
180
181             <!-- Right column -->
182             <include
183                 android:id="@+id/note_10"
184                 layout="@layout/view_note_row"
185                 android:layout_width="@dimen/match_constraint"
186                 android:layout_height="@dimen/match_constraint"
187                 android:layout_marginStart="@dimen/note_column_half_gap"
188                 android:layout_marginTop="@dimen/notes_title_margin_vertical"
189                 app:layout_constraintBottom_toTopOf="@id/note_5"
190                 app:layout_constraintEnd_toEndOf="parent"
191                 app:layout_constraintStart_toEndOf="@id/notes_columns"
192                 app:layout_constraintTop_toBottomOf="@id/notes_title" />
193
194             <include
195                 android:id="@+id/note_5"
196                 layout="@layout/view_note_row"
197                 android:layout_width="@dimen/match_constraint"
198                 android:layout_height="@dimen/match_constraint"
199                 android:layout_marginStart="@dimen/note_column_half_gap"
200                 android:layout_marginTop="@dimen/note_row_gap"
201                 app:layout_constraintBottom_toTopOf="@id/note_2"
202                 app:layout_constraintEnd_toEndOf="parent"
203                 app:layout_constraintStart_toEndOf="@id/notes_columns"
204                 app:layout_constraintTop_toBottomOf="@id/note_10" />
205
206             <include
207                 android:id="@+id/note_2"
208                 layout="@layout/view_note_row"
209                 android:layout_width="@dimen/match_constraint"
210                 android:layout_height="@dimen/match_constraint"
211                 android:layout_marginStart="@dimen/note_column_half_gap"
212                 android:layout_marginTop="@dimen/note_row_gap"
213                 app:layout_constraintBottom_toTopOf="@id/note_1"
214                 app:layout_constraintEnd_toEndOf="parent"
215                 app:layout_constraintStart_toEndOf="@id/notes_columns"
216                 app:layout_constraintTop_toBottomOf="@id/note_5" />
217
218             <include
219                 android:id="@+id/note_1"
220                 layout="@layout/view_note_row"
221                 android:layout_width="@dimen/match_constraint"
222                 android:layout_height="@dimen/match_constraint"
223                 android:layout_marginStart="@dimen/note_column_half_gap"
224                 android:layout_marginTop="@dimen/note_row_gap"
225                 app:layout_constraintBottom_toBottomOf="parent"
226                 app:layout_constraintEnd_toEndOf="parent"
227                 app:layout_constraintStart_toEndOf="@id/notes_columns"
228                 app:layout_constraintTop_toBottomOf="@id/note_2" />
229
230         </androidx.constraintlayout.widget.ConstraintLayout>
231     </com.google.android.material.card.MaterialCardView>
232
233     <!--
234         Keypad, 4 columns:   1 2 3 4
235                              5 6 7 8
236                              9 0 CLEAR
237         Rows 1-2 are horizontal chains; the first column is a vertical chain. 0 lines up with the
```

app/src/main/res/layout-sw600dp-land/activity_main.xml (continued)

```
        2nd column and CLEAR spans the 3rd and 4th. The keypad is as tall as the change table.
    -->
    <androidx.constraintlayout.widget.ConstraintLayout
        android:id="@+id/keypad"
        android:layout_width="@dimen/match_constraint"
        android:layout_height="@dimen/match_constraint"
        android:layout_marginStart="@dimen/column_gap"
        android:layout_marginEnd="@dimen/screen_margin"
        app:layout_constraintBottom_toBottomOf="@id/notes_card"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/split"
        app:layout_constraintTop_toTopOf="@id/notes_card">

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_1"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:text="@string/key_1"
            app:layout_constraintBottom_toTopOf="@id/key_5"
            app:layout_constraintEnd_toStartOf="@id/key_2"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_2"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_2"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
            app:layout_constraintEnd_toStartOf="@id/key_3"
            app:layout_constraintStart_toEndOf="@id/key_1"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_3"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_3"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
            app:layout_constraintEnd_toStartOf="@id/key_4"
            app:layout_constraintStart_toEndOf="@id/key_2"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_4"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_4"
            app:layout_constraintBottom_toBottomOf="@id/key_1"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/key_3"
            app:layout_constraintTop_toTopOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_5"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginTop="@dimen/key_gap"
            android:text="@string/key_5"
            app:layout_constraintBottom_toTopOf="@id/key_9"
            app:layout_constraintEnd_toStartOf="@id/key_6"
            app:layout_constraintStart_toStartOf="parent"
```

app/src/main/res/layout-sw600dp-land/activity_main.xml (continued)

```
            app:layout_constraintTop_toBottomOf="@id/key_1" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_6"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_6"
            app:layout_constraintBottom_toBottomOf="@id/key_5"
            app:layout_constraintEnd_toStartOf="@id/key_7"
            app:layout_constraintStart_toEndOf="@id/key_5"
            app:layout_constraintTop_toTopOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_7"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_7"
            app:layout_constraintBottom_toBottomOf="@id/key_5"
            app:layout_constraintEnd_toStartOf="@id/key_8"
            app:layout_constraintStart_toEndOf="@id/key_6"
            app:layout_constraintTop_toTopOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_8"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginStart="@dimen/key_gap"
            android:text="@string/key_8"
            app:layout_constraintBottom_toBottomOf="@id/key_5"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/key_7"
            app:layout_constraintTop_toTopOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_9"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:layout_marginTop="@dimen/key_gap"
            android:text="@string/key_9"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="@id/key_5"
            app:layout_constraintStart_toStartOf="@id/key_5"
            app:layout_constraintTop_toBottomOf="@id/key_5" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_0"
            style="@style/Widget.VangtiChai.Key"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:text="@string/key_0"
            app:layout_constraintBottom_toBottomOf="@id/key_9"
            app:layout_constraintEnd_toEndOf="@id/key_6"
            app:layout_constraintStart_toStartOf="@id/key_6"
            app:layout_constraintTop_toTopOf="@id/key_9" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/key_clear"
            style="@style/Widget.VangtiChai.Key.Clear"
            android:layout_width="@dimen/match_constraint"
            android:layout_height="@dimen/match_constraint"
            android:contentDescription="@string/cd_key_clear"
            android:text="@string/key_clear"
            app:layout_constraintBottom_toBottomOf="@id/key_9"
            app:layout_constraintEnd_toEndOf="@id/key_8"
```

app/src/main/res/layout-sw600dp-land/activity_main.xml (continued)

```
            app:layout_constraintStart_toStartOf="@id/key_7"
            app:layout_constraintTop_toTopOf="@id/key_9" />

    </androidx.constraintlayout.widget.ConstraintLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

A.7 Drawables, icons and animators

app/src/main/res/drawable/**bg_header.xml** 16 lines · XML

ABOUT THIS FILE

The header's background on Android 5.0 and 5.1 (API 21 and 22): just the green gradient, without the two glows of `drawable-v23/bg_header.xml`, which is printed next.

- **Lines 7–8**: a single `<shape>`, a rectangle, rather than a `layer-list`.
- **Lines 9–12**: the linear gradient. `angle="315"` runs it from the top-left corner (`header_gradient_start`, `#0B6E4D`) to the bottom-right (`header_gradient_end`, `#06301F`). In Android's gradients, 0 means left to right and larger angles turn anticlockwise, so 270 is top to bottom and 315 is the diagonal between.
- **Lines 13–15**: only the bottom corners are rounded.

Android picks between the two files by version. A `drawable-v23` folder is only used from API 23 on, so newer phones get the glows and older ones get this file. The glows need layer-list features that arrived in API 23; on Android 5.0 they spread into a pale blob across the header (Part 5.2).

app/src/main/res/drawable/bg_header.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    Header background on Android 5.0 and 5.1 (API 21-22): the green gradient with rounded bottom
    corners. These versions can't size or place layer-list items, so the two soft glows are left
    out here; drawable-v23/bg_header.xml adds them on API 23 and newer.
-->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <gradient
        android:angle="315"
        android:endColor="@color/header_gradient_end"
        android:startColor="@color/header_gradient_start" />
    <corners
        android:bottomLeftRadius="@dimen/header_corner_radius"
        android:bottomRightRadius="@dimen/header_corner_radius" />
</shape>
```

app/src/main/res/drawable-v23/**bg_header.xml** 47 lines · XML

ABOUT THIS FILE

The header's background from Android 6.0 (API 23) on (Part 3, Step 9): three layers in a `layer-list`, drawn bottom to top.

- **Lines 8–18**: the base, the same gradient rectangle as in `drawable/bg_header.xml`, with only the bottom corners rounded (lines 14–16).

- **Lines 19–32**: a glow in the top-right corner: an oval `header_glow_size` (220 dp) across, filled with a radial gradient from `header_glow` (white at 12%) to transparent. `gravity="top|end"` puts it in the corner, and the negative `right` and `top` offsets (`header_glow_offset`, −90 dp) push it partly outside, so only a soft edge shows.
- **Lines 33–46**: the same glow in the top-left corner.

An item's `width`, `height` and `gravity` (lines 20–22 and 34–36) only work from API 23, which is why this file lives in `drawable-v23/`. The glow sizes come from `sizes.xml`, so tablets get bigger ones (380 dp).

app/src/main/res/drawable-v23/bg_header.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <!--
3      Header background: a green gradient with rounded bottom corners and two soft glows.
4      The glows are sized and placed with the items' width, height and gravity, which Android only
5      supports from API 23 on. Older versions use drawable/bg_header.xml, which has no glows.
6  -->
7  <layer-list xmlns:android="http://schemas.android.com/apk/res/android">
8      <item>
9          <shape android:shape="rectangle">
10             <gradient
11                 android:angle="315"
12                 android:endColor="@color/header_gradient_end"
13                 android:startColor="@color/header_gradient_start" />
14             <corners
15                 android:bottomLeftRadius="@dimen/header_corner_radius"
16                 android:bottomRightRadius="@dimen/header_corner_radius" />
17         </shape>
18     </item>
19     <item
20         android:width="@dimen/header_glow_size"
21         android:height="@dimen/header_glow_size"
22         android:gravity="top|end"
23         android:right="@dimen/header_glow_offset"
24         android:top="@dimen/header_glow_offset">
25         <shape android:shape="oval">
26             <gradient
27                 android:endColor="@android:color/transparent"
28                 android:gradientRadius="@dimen/header_glow_radius"
29                 android:startColor="@color/header_glow"
30                 android:type="radial" />
31         </shape>
32     </item>
33     <item
34         android:width="@dimen/header_glow_size"
35         android:height="@dimen/header_glow_size"
36         android:gravity="top|start"
37         android:left="@dimen/header_glow_offset"
38         android:top="@dimen/header_glow_offset">
39         <shape android:shape="oval">
40             <gradient
41                 android:endColor="@android:color/transparent"
42                 android:gradientRadius="@dimen/header_glow_radius"
43                 android:startColor="@color/header_glow"
44                 android:type="radial" />
45         </shape>
46     </item>
47 </layer-list>
```

app/src/main/res/drawable/**bg_note_row.xml** 7 lines · XML

ABOUT THIS FILE

The background of each change-table row: a rectangle with `note_row_corner_radius` corners (14 dp on phones). Its white fill is only a placeholder. `setRowLevel()` replaces it with `GradientDrawable.setColor()`, using the row's colour for its current level, so one drawable serves all eight notes and every step of the fade. A background tint would look the same on newer Android, but Android 5.0 can't tint shapes and left the rows white.

app/src/main/res/drawable/bg_note_row.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Row of the change table. MainActivity replaces the white with the row's colour. -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@android:color/white" />
    <corners android:radius="@dimen/note_row_corner_radius" />
</shape>
```

app/src/main/res/drawable/**bg_note_accent.xml** 7 lines · XML

ABOUT THIS FILE

The small colour tab at the start of each row: a rectangle with 2 dp corners (`note_accent_corner_radius`, 3 dp on tablets). Like the row background, `setRowLevel()` sets its colour: the note's colour at 25% when the row is faded, and the full colour when it is lit.

app/src/main/res/drawable/bg_note_accent.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Small colour tab at the start of each change table row; MainActivity colours it per note. -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@android:color/white" />
    <corners android:radius="@dimen/note_accent_corner_radius" />
</shape>
```

app/src/main/res/drawable/**ic_backspace.xml** 12 lines · XML

ABOUT THIS FILE

The backspace icon as a vector drawable (Part 3, Step 9).

- **Lines 4–8**: a 24 × 24 dp image drawn on a 24 × 24 unit grid (the viewport). The button's `iconSize` scales it to 28 dp on phones and 44 dp on tablets, and a vector stays sharp at any size.
- **Line 6**: `autoMirrored="true"` would flip the icon in right-to-left languages. VangtiChai's root layout is always left-to-right, so in practice it never flips.
- **Lines 9–11**: a single path in three parts: the outline of the key shape (`M22,3H7...`), an inner shape cut out of it to leave an outline (`M22,19H7.07...`), and the ×, a 12-point polygon (`M10.41,17L14,13.41...`). Upper-case commands (`M`, `L`, `H`, `V`, `C`) use absolute coordinates; lower-case ones (`l`, `h`, `c`) move relative to the current point.
- The path is filled white, and the button's `iconTint` (`header_content`) colours it in any case.

app/src/main/res/drawable/ic_backspace.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Backspace key icon (Material "backspace", outlined). The key tints it. -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:autoMirrored="true"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="@android:color/white"
        android:pathData="M22,3H7C6.31,3 5.77,3.35 5.41,3.88L0,12l5.41,8.11C5.77,20.64 6.31,21
            7,21h15c1.1,0 2,-0.9 2,-2V5C24,3.9 23.1,3 22,3zM22,19H7.07L2.4,12l4.66,-7H22V19zM10.41,
            17L14,13.41L17.59,17L19,15.59L15.41,12L19,8.41L17.59,7L14,10.59L10.41,7L9,8.41L12.59,12L9,
            15.59z" />
</vector>
```

app/src/main/res/drawable/**ic_launcher_background.xml** 20 lines · XML

ABOUT THIS FILE

The bottom layer of the adaptive icon: a 108 × 108 dp square (`M0,0h108v108h-108z`) filled with a linear gradient from `icon_background_start` (`#13A874`) at the top-left corner (0, 0) to `icon_background_end` (`#014A34`) at the bottom-right (108, 108).

The gradient is written inline with `<aapt:attr name="android:fillColor">` (lines 9–18). The `aapt` namespace (line 3) lets a complex value such as a gradient live inside the vector, instead of in a separate resource file.

app/src/main/res/drawable/ic_launcher_background.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:aapt="http://schemas.android.com/aapt"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <path android:pathData="M0,0h108v108h-108z">
        <aapt:attr name="android:fillColor">
            <gradient
                android:endColor="@color/icon_background_end"
                android:endX="108"
                android:endY="108"
                android:startColor="@color/icon_background_start"
                android:startX="0"
                android:startY="0"
                android:type="linear" />
        </aapt:attr>
    </path>
</vector>
```

app/src/main/res/drawable/**ic_launcher_foreground.xml** 31 lines · XML

ABOUT THIS FILE

The middle layer: two banknotes and a coin, on the same 108 × 108 grid.

Lines	Shape	Where, in grid units
9–11	Back note, white at 55% (`icon_note_back`)	x 37–77, y 35–59
13–15	Front note, white (`icon_note`)	x 30–70, y 43–67
16–18	The note's centre circle and two dots	on the line y = 55
20–22	A ring in `icon_note_detail` that separates the coin from the note	circle, radius 11, centre (68, 64)
23–25	The gold coin (`icon_coin`)	circle, radius 9, same centre
26–30	The coin's inner ring: a 1.3-unit stroke with a transparent fill	circle, radius 6

Launchers crop the icon to many shapes, but always keep a central circle 66 dp across (radius 33 around the centre point, 54, 54). Every shape here stays within about 29 units of the centre, so nothing is ever cut off. Figure 2 shows the result.

app/src/main/res/drawable/ic_launcher_foreground.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Two Taka notes and a coin, kept inside the 66 dp safe zone of the adaptive icon. -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <!-- Note at the back -->
    <path
        android:fillColor="@color/icon_note_back"
        android:pathData="M40.5,35h33a3.5,3.5 0,0 1,3.5 3.5v17a3.5,3.5 0,0 1,-3.5 3.5h-33a3.5,3.5 0,0 1,-3.5 -3.5v-17a3.5,3.5 0,0 1,3.5 -3.5z" />
    <!-- Note at the front -->
    <path
        android:fillColor="@color/icon_note"
        android:pathData="M33.5,43h33a3.5,3.5 0,0 1,3.5 3.5v17a3.5,3.5 0,0 1,-3.5 3.5h-33a3.5,3.5 0,0 1,-3.5 -3.5v-17a3.5,3.5 0,0 1,3.5 -3.5z" />
    <path
        android:fillColor="@color/icon_note_detail"
        android:pathData="M44,55a6,6 0,1 0,12 0a6,6 0,1 0,-12 0zM34.2,55a1.8,1.8 0,1 0,3.6 0a1.8,1.8 0,1 0,-3.6 0zM62.2,55a1.8,1.8 0,1 0,3.6 0a1.8,1.8 0,1 0,-3.6 0z" />
    <!-- Coin, with a ring of background colour separating it from the note -->
    <path
        android:fillColor="@color/icon_note_detail"
        android:pathData="M57,64a11,11 0,1 0,22 0a11,11 0,1 0,-22 0z" />
    <path
        android:fillColor="@color/icon_coin"
        android:pathData="M59,64a9,9 0,1 0,18 0a9,9 0,1 0,-18 0z" />
    <path
        android:fillColor="#00000000"
        android:pathData="M62,64a6,6 0,1 0,12 0a6,6 0,1 0,-12 0z"
        android:strokeWidth="1.3"
        android:strokeColor="@color/icon_coin_detail" />
</vector>
```

app/src/main/res/drawable/**ic_launcher_monochrome.xml**

12 lines · XML

ABOUT THIS FILE

The themed-icon layer for Android 13+: a single note with its centre circle and two dots cut out of it. `fillType="evenOdd"` (line 10) makes the holes: with the even-odd rule, a point inside both the note and the circle is enclosed twice, an even number, so it is left empty. The launcher ignores the black and recolours the shape to suit the wallpaper (Figure 2, right).

app/src/main/res/drawable/ic_launcher_monochrome.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Single-colour version of the launcher icon for themed icons (Android 13+): one note. -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <path
        android:fillColor="#FF000000"
        android:fillType="evenOdd"
        android:pathData="M37.5,41h33a3.5,3.5 0,0 1,3.5 3.5v19a3.5,3.5 0,0 1,-3.5 3.5h-33a3.5,3.5 0,0
            1,-3.5 -3.5v-19a3.5,3.5 0,0 1,3.5 -3.5zM48,54a6,6 0,1 0,12 0a6,6 0,1 0,-12
            0zM38.2,54a1.8,1.8 0,1 0,3.6 0a1.8,1.8 0,1 0,-3.6 0zM66.2,54a1.8,1.8 0,1 0,3.6 0a1.8,1.8
            0,1 0,-3.6 0z" />
</vector>
```

app/src/main/res/mipmap-anydpi-v26/**ic_launcher.xml** 6 lines · XML

ABOUT THIS FILE

The adaptive icon: background, foreground and monochrome layers (lines 3–5). `anydpi` means one vector file serves every screen density, and `-v26` limits the file to Android 8.0 (API 26) and newer, the first version with adaptive icons. Android 5.0 to 7.1 fall back to the bitmaps in `mipmap-*dpi/ic_launcher.webp`.

app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@drawable/ic_launcher_background" />
    <foreground android:drawable="@drawable/ic_launcher_foreground" />
    <monochrome android:drawable="@drawable/ic_launcher_monochrome" />
</adaptive-icon>
```

app/src/main/res/mipmap-anydpi-v26/**ic_launcher_round.xml** 6 lines · XML

ABOUT THIS FILE

Identical to `ic_launcher.xml`. The manifest's `android:roundIcon` points here, for launchers that ask for a round icon. An adaptive icon is masked by the launcher anyway, so both files can share the same three layers.

app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@drawable/ic_launcher_background" />
    <foreground android:drawable="@drawable/ic_launcher_foreground" />
    <monochrome android:drawable="@drawable/ic_launcher_monochrome" />
</adaptive-icon>
```

app/src/main/res/animator/**key_press.xml** 34 lines · XML

ABOUT THIS FILE

The key press animation (Part 3, Step 10), used by every key and by the backspace key through `android:stateListAnimator` in their styles.

- **Lines 4–17**: while `state_pressed` is true, `scaleX` and `scaleY` animate to 0.93 over `key_press_duration` (90 ms).
- **Lines 18–33**: in any other state they animate back to 1 with the overshoot interpolator, which goes a little past 100% before settling.

Only `valueTo` is given, never `valueFrom`, so each animation starts from the current scale: a key released half-way through shrinking springs back from wherever it is, with no jump. Android uses the first `<item>` whose states match, which is why the item with no state comes last.

app/src/main/res/animator/key_press.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Keys shrink slightly while pressed and spring back when released. -->
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:state_pressed="true">
        <set>
            <objectAnimator
                android:duration="@integer/key_press_duration"
                android:propertyName="scaleX"
                android:valueTo="0.93"
                android:valueType="floatType" />
            <objectAnimator
                android:duration="@integer/key_press_duration"
                android:propertyName="scaleY"
                android:valueTo="0.93"
                android:valueType="floatType" />
        </set>
    </item>
    <item>
        <set>
            <objectAnimator
                android:duration="@integer/key_press_duration"
                android:interpolator="@android:anim/overshoot_interpolator"
                android:propertyName="scaleX"
                android:valueTo="1"
                android:valueType="floatType" />
            <objectAnimator
                android:duration="@integer/key_press_duration"
                android:interpolator="@android:anim/overshoot_interpolator"
                android:propertyName="scaleY"
                android:valueTo="1"
                android:valueType="floatType" />
        </set>
    </item>
</selector>
```

app/src/main/res/xml/**backup_rules.xml** 13 lines · XML

ABOUT THIS FILE

A template file for Android's Auto Backup, referenced by `android:fullBackupContent` in the manifest. Android uses it on Android 11 (API 30) and older. Every rule is commented out, so the default behaviour applies. VangtiChai keeps nothing in files or preferences, so there is nothing of its own to back up either way.

app/src/main/res/xml/backup_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--
   Sample backup rules file; uncomment and customize as necessary.
   See https://developer.android.com/guide/topics/data/autobackup
   for details.
   Note: This file is ignored for devices older than API 31
   See https://developer.android.com/about/versions/12/backup-restore
-->
<full-backup-content>
    <!--
   <include domain="sharedpref" path="."/>
   <exclude domain="sharedpref" path="device.xml"/>
-->
</full-backup-content>
```

app/src/main/res/xml/**data_extraction_rules.xml** 19 lines · XML

ABOUT THIS FILE

The Android 12+ (API 31) replacement for `backup_rules.xml`, referenced by `android:dataExtractionRules`. It can set separate rules for cloud backup and for device-to-device transfer (`<device-transfer>`). Everything here is commented out, so the defaults apply.

app/src/main/res/xml/data_extraction_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--
   Sample data extraction rules file; uncomment and customize as necessary.
   See https://developer.android.com/about/versions/12/backup-restore#xml-changes
   for details.
-->
<data-extraction-rules>
    <cloud-backup>
        <!-- TODO: Use <include> and <exclude> to control what is backed up.
        <include .../>
        <exclude .../>
        -->
    </cloud-backup>
    <!--
    <device-transfer>
        <include .../>
        <exclude .../>
    </device-transfer>
    -->
</data-extraction-rules>
```

A.8 README.txt (assignment deliverable)

README.txt

116 lines · Plain text

ABOUT THIS FILE

The required assignment deliverable (Part 3, Step 19). It is plain text so it reads the same in any editor, with sections underlined in the old typewriter style:

Lines	Section
1–8	Title, summary and SDK levels
11–26	What the app does
28–40	Extras
43–70	Layouts, and where the sizes live
73–98	Devices and screen types tested (required by the brief)
97–110	Tests and how to run them
113–116	Project structure

The device list is the part the brief asks for by name. It has the two required devices, the Pixel XL and the Nexus 10, tested with the emulator as the brief says. Then come the four other screen sizes emulated on them, and Android 5.0, the oldest version the app supports (Part 5.2).

README.txt

```
VangtiChai - CSE 489 Mobile Application Development, Assignment 1
=================================================================

VangtiChai is an Android app (Kotlin, XML layouts) that calculates change for an amount in
Taka notes: 500, 100, 50, 20, 10, 5, 2 and 1. The amount is typed on a numeric keypad built
from scratch with ConstraintLayout.

Minimum SDK: API 21 (Android 5.0 Lollipop), as the assignment suggests. Target/compile SDK: API 37.


What the app does
-----------------
- Each digit you tap is added to the right of the amount: 2, 3, 4 shows 2, then 23, then 234.
- The change table lists how many of each note make up the amount, using as few notes as
  possible (500: 1, 100: 1, 50: 1, ... for 688).
- CLEAR resets the amount and the table.
- A backspace key at the end of the amount row deletes the last digit, like the one beside the
  number in a phone dialer: tap it to remove one digit, or hold it to keep removing digits until
  it is let go. It only shows once something is typed. It sits in the header, so the keypad keeps
  exactly the configuration from the assignment (10 digits and CLEAR).
- Rotating the device keeps the amount and the change list. Both are saved in
  onSaveInstanceState() and restored in onCreate(); the activity is recreated so that the
  landscape (or portrait) layout is loaded.
- Amounts of up to 9 digits (999,999,999 Taka) are accepted. Typing more shakes the amount and
  shows "Up to 9 digits". Long amounts and counts shrink to fit, so everything stays on one
  screen in full.

Extras
- Material 3 design with a dark green "Taka" theme. The app is always dark, whatever the
  phone's own theme setting, so there is no theme switching.
- Smooth motion: the change table and keypad rise into place when the app opens; notes that
  are part of the change fade up in their own colour and the rest fade back; counts pop when
  they change; keys shrink slightly when pressed; haptic feedback on key presses.
- Edge-to-edge drawing that keeps content clear of the status bar, navigation bar and
  display cutouts.
- Hardware keyboard support (handy on an emulator): digits type, Backspace deletes the last
```

README.txt (continued)

```
  digit, Esc or C clears.
- Screen reader support: the amount is announced when it changes and each table row has a
  spoken description (e.g. "1 note of 500 Taka").
- Custom adaptive launcher icon, including a themed (monochrome) version.


Layouts
-------
Four layouts, all built on ConstraintLayout (chains, guidelines and percentage sizing):

  res/layout/activity_main.xml               Phone, portrait     Pixel XL, 411 x 731 dp
  res/layout-land/activity_main.xml          Phone, landscape    Pixel XL, 731 x 411 dp
  res/layout-sw600dp/activity_main.xml       Tablet, portrait    Nexus 10, 800 x 1280 dp
  res/layout-sw600dp-land/activity_main.xml  Tablet, landscape   Nexus 10, 1280 x 800 dp
  res/layout/view_note_row.xml               One row of the change table (included 8 times)

Portrait: the amount ("Taka: 688") is on top, the change table on the left and a 3-column
keypad (1-9, then 0 and a wide CLEAR) on the right.
Landscape: the change table is split into two columns (500/100/50/20 and 10/5/2/1) and the
keypad becomes 4 columns (1-4, 5-8, then 9, 0 and a wide CLEAR).
Tablet layouts use the same arrangements, scaled up, with a subtitle in the app bar.

No values are hardcoded in the layout files. Text sizes, margins, paddings, corner radii and
layout proportions are defined in sizes.xml for each configuration:

  res/values/sizes.xml               phones, portrait (the defaults)
  res/values-land/sizes.xml          phones, landscape
  res/values-sw600dp/sizes.xml       tablets, portrait
  res/values-sw600dp-land/sizes.xml  tablets, landscape

Even ConstraintLayout's "match constraints" size, 0dp, is not written in the layouts: they use
@dimen/match_constraint, defined once in res/values/sizes.xml.

All text is in res/values/strings.xml and all colours in res/values/colors.xml.


Devices and screen types tested
-------------------------------
Everything below was tested on the Android Emulator. On each device the on-screen keys were
tapped to enter 2, 3 and 4 (showing 2, then 23, then 234), CLEAR, 688, 6880 and the 9-digit
maximum 999999999, and the device was rotated portrait -> landscape -> portrait. The amount
and the change list were kept on every rotation, and the counts match the assignment's
screenshots (688: one of each note; 6880: 13, 3, 1, 1, 1, 0, 0, 0).

Required devices, on Android 17 (API 37) emulator images:

  Pixel XL    1440 x 2560 px, 560 dpi = 411 x 731 dp    layout/ and layout-land/
  Nexus 10    1600 x 2560 px, 320 dpi = 800 x 1280 dp   layout-sw600dp/ and layout-sw600dp-land/

Other screen types, emulated on those two devices by changing the screen size and density
(adb shell wm size / wm density), each in portrait and landscape:

  Nexus 5, small phone          1080 x 1920 px, 480 dpi = 360 x 640 dp
  Medium Phone (Pixel 7 size)   1080 x 2400 px, 420 dpi = 411 x 914 dp
  Nexus 7, 7-inch tablet        1200 x 1920 px, 320 dpi = 600 x 960 dp
  Pixel C, large tablet         1800 x 2560 px, 320 dpi = 900 x 1280 dp

Oldest supported version: a Pixel XL emulator running Android 5.0 (API 21), with the phone
layouts and the tablet layouts (at 800 x 1280 dp), in portrait and landscape.

The 7 Espresso UI tests (see Tests below) pass on all three emulators: Pixel XL, Nexus 10 and
Android 5.0.


Tests
-----
  app/src/test/.../ChangeCalculatorTest.kt  JVM unit tests for the change logic, including
                                            the four examples from the assignment
                                            screenshots and a check that the fewest notes
                                            are always used. Run: gradlew test
```

README.txt (continued)

```
  app/src/androidTest/.../MainActivityTest.kt  Espresso tests for typing, CLEAR, backspace
                                               (tap and hold) and keeping state across
                                               recreation. Run on a device or emulator:
                                               gradlew connectedAndroidTest


Project structure
-----------------
  app/src/main/java/com/example/vangtichai/MainActivity.kt      UI, keypad, state saving
  app/src/main/java/com/example/vangtichai/ChangeCalculator.kt  Change calculation logic
```

A.9 The rest of the repository

ADDED IN THIS EDITION

The guide's appendix prints 44 files. Git tracks 75 (not counting this PDF), so this edition adds the rest:

- `README.md`, `gradlew` and `gradlew.bat`, printed in full like the files above.
- The 27 binary files (launcher icons, screenshots and the wrapper JAR), each listed with what it is. Every image among them is shown somewhere in this PDF.

The one file left is `PROJECT_GUIDE.md`, which this whole PDF reproduces. Together, the appendix and this section cover every file in the repository.

README.md

186 lines · Markdown

ABOUT THIS FILE

The repository's front page on GitHub (Part 3, Step 19). The guide's appendix leaves it out, so it is printed here. It is Markdown with some HTML mixed in: `<div align="center">` and `<p align="center">` centre the header and the screenshots, `width` on `<img>` sets their sizes, `<kbd>` draws keyboard keys, and shields.io badges show the language and SDK levels. Line 7 holds the app's Bangla name, ভাংতি চাই.

Lines	Section	Also covered in this PDF
1–27	Header: logo, name, Bangla name, tagline, badges and three screenshots	Figure 1 and the cover
31–46	Features	Part 0 (Behaviour summary) and Part 1 (Extras beyond the brief)
48–71	How it works	Part 3, Step 11
73–101	Screenshots (Landscape; Tablet; Input limit and launcher icon)	Figures 2, 4, 5 and 6
103–129	Layouts (Tested screen sizes)	Part 3, Steps 13 to 15, and Part 5.4
131–137	Tech stack	Part 0 (Facts)
139–154	Getting started	Part 0 (How to build and test)
156–161	Tests	Part 3, Step 17
163–180	Project structure	Part 2.3
182–186	About	The cover

The screenshots the README shows are the same image files this PDF uses for its figures.

README.md

```
1  <div align="center">
2
3  <img src="docs/images/logo.png" width="112" alt="VangtiChai app icon">
4
5  # VangtiChai
6
7  **ভাংতি চাই**  ·  *"I need change"*
8
9  A Taka change calculator for Android. Type an amount and see exactly which notes make it up, using as
↪      few notes as possible.
10
11 <br>
12
```

README.md (continued)

````
13 ![Kotlin](https://img.shields.io/badge/Kotlin-7F52FF?style=for-the-badge&logo=kotlin&logoColor=white)
14 ![Android](https://img.shields.io/badge/Android-3DDC84?style=for-the-badge&logo=android&logoColor=
   white)
15 ![Material 3](https://img.shields.io/badge/Material%203-006C4E?style=for-the-badge&logo=
   materialdesign&logoColor=white)
16 ![Min SDK](https://img.shields.io/badge/Min%20SDK-21-0F9D6C?style=for-the-badge)
17 ![Target SDK](https://img.shields.io/badge/Target%20SDK-37-01543B?style=for-the-badge)
18
19 <br>
20
21 <img src="docs/images/phone-688.png" width="250" alt="VangtiChai showing 688 Taka, made of one of each
   note">
22   
23 <img src="docs/images/phone-6880.png" width="250" alt="VangtiChai showing 6880 Taka, made of 19 notes">
24   
25 <img src="docs/images/phone-68.png" width="250" alt="VangtiChai showing 68 Taka, with unused notes
   faded out">
26
27 </div>
28
29 <br>
30
31 ## ✨ Features
32
33 | Feature | What it does |
34 |---|---|
35 | 🔢 **Custom keypad** | A numeric keypad built from scratch with `ConstraintLayout`. Digits are added
   from the right: tapping 2, 3, 4 shows 2, then 23, then 234. |
36 | ⌫ **Backspace** | Works like a phone's: tap it to delete the last digit, or hold it to keep deleting
   until you let go. It sits beside the amount, as in a phone dialer, and only appears once something
   is typed. |
37 | 💶 **Instant change table** | Shows how many 500, 100, 50, 20, 10, 5, 2 and 1 Taka notes make up the
   amount, always using the fewest notes. |
38 | 🔄 **Survives rotation** | The amount and the change list are saved in `onSaveInstanceState()` and
   restored when the layout switches between portrait and landscape. |
39 | 📱 **Phone and tablet layouts** | Four hand-built layouts for phone and tablet, in portrait and
   landscape. |
40 | 🌓 **Dark Material 3 design** | A dark green "Taka" theme that is always on, whatever the phone's
   own theme setting, with edge-to-edge drawing that keeps clear of the status bar, navigation bar
   and display cutouts. |
41 | 🌈 **Colour-coded notes** | Each note has its own colour. Notes that are part of the change light up
   and the rest fade back. |
42 | ✨ **Smooth motion** | The change table and keypad rise into place when the app opens. Rows fade
   smoothly between lit and faded, counts pop when they change, keys shrink slightly when pressed,
   and every key press gives haptic feedback. |
43 | 🔘 **Input limit** | Accepts up to 9 digits (999,999,999 Taka). Typing more shakes the amount and
   shows *"Up to 9 digits"*. Long amounts and counts shrink to fit, so everything stays on one
   screen. |
44 | ⌨️ **Hardware keyboard** | Digits type, <kbd>Backspace</kbd> deletes the last digit, and
   <kbd>Esc</kbd> or <kbd>C</kbd> clears. Handy on an emulator. |
45 | ♿ **Accessible** | Screen readers announce the amount as it changes, and each row has a spoken
   description such as *"1 note of 500 Taka"*. |
46 | 🖼️ **Adaptive launcher icon** | A custom adaptive icon, including a themed (monochrome) version for
   Android 13+. |
47
48 ## 🧾 How it works
49
50 The change logic lives in [`ChangeCalculator.kt`](app/src/main/java/com/example/vangtichai/
   ChangeCalculator.kt), separate from the UI so it can be unit tested on its own:
51
52 ```kotlin
53 val NOTES = intArrayOf(500, 100, 50, 20, 10, 5, 2, 1)
54
55 fun breakdown(amount: Long): LongArray {
56     var remaining = amount
57     return LongArray(NOTES.size) { i ->
58         val count = remaining / NOTES[i]
59         remaining %= NOTES[i]
60         count
61     }
````

README.md (continued)

````
}
```

It takes the largest note that fits, then repeats with what is left. The Taka note system is
    *canonical*, which means this greedy approach always gives the smallest number of notes. The unit
    tests check that against a dynamic-programming solution for every amount from 0 to 2,000.

**Example: 6880 Taka → 19 notes**

| Note | 500 | 100 | 50 | 20 | 10 | 5 | 2 | 1 |
|:-----|:---:|:---:|:---:|:---:|:---:|:---:|:-:|:-:|:-:|
| **Count** | 13 | 3 | 1 | 1 | 1 | 0 | 0 | 0 |

## 🖼 Screenshots

### Landscape

The change table splits into two columns and the keypad grows to four columns.

<p align="center">
  <img src="docs/images/phone-land-688.png" width="400" alt="Phone in landscape, 688 Taka">
   
  <img src="docs/images/phone-land-6880.png" width="400" alt="Phone in landscape, 6880 Taka">
</p>

### Tablet

Tablet layouts use the same arrangement, scaled up, with a subtitle in the app bar.

<p align="center">
  <img src="docs/images/tablet-688.png" width="280" alt="Tablet in portrait, 688 Taka">
   
  <img src="docs/images/tablet-land-688.png" width="460" alt="Tablet in landscape, 688 Taka">
</p>

### Input limit and launcher icon

<p align="center">
  <img src="docs/images/phone-limit.png" width="230" alt="The amount at the 9-digit limit, with the
      caption 'Up to 9 digits'">
    
  <img src="docs/images/launcher-icons.png" width="500" alt="Launcher icon: round, squircle and themed
      monochrome versions">
</p>

## 📐 Layouts

All four layouts are built on `ConstraintLayout` using chains, guidelines and percentage sizing.

| Layout file | Screen | Designed for |
|:--|:--|:--|
| `layout/activity_main.xml` | Phone, portrait | Pixel XL · 411 × 731 dp |
| `layout-land/activity_main.xml` | Phone, landscape | Pixel XL · 731 × 411 dp |
| `layout-sw600dp/activity_main.xml` | Tablet, portrait | Nexus 10 · 800 × 1280 dp |
| `layout-sw600dp-land/activity_main.xml` | Tablet, landscape | Nexus 10 · 1280 × 800 dp |

The layout files contain no hardcoded values. Text sizes, margins, paddings, corner radii and
    proportions come from a `sizes.xml` for each configuration, all text is in `strings.xml`, and all
    colours are in `colors.xml`. Even ConstraintLayout's "match constraints" size, `0dp`, is written
    as `@dimen/match_constraint`.

### Tested screen sizes

Every size was tested on the Android Emulator in portrait and landscape: tapping in 2, 23, 234, 688,
    6880 and 999999999, clearing, and rotating back and forth, which kept the amount and the change
    list every time. The Pixel XL and Nexus 10 are emulator devices running Android 17. The other
    sizes were emulated on them with `adb shell wm size` and `wm density`.

| Device | Size | |
|:--|:--|:--|
````
````

README.md (continued)

```
| Pixel XL | 411 × 731 dp | required |
| Nexus 10 | 800 × 1280 dp | required |
| Nexus 5 | 360 × 640 dp | small phone |
| Medium Phone | 411 × 914 dp | Pixel 7 size |
| Nexus 7 | 600 × 960 dp | 7-inch tablet |
| Pixel C | 900 × 1280 dp | large tablet |

The app also runs on the oldest version it supports: a Pixel XL emulator with Android 5.0 (API 21),
    using both the phone and the tablet layouts.

## 🛠 Tech stack

- **Language:** Kotlin
- **UI:** XML layouts, `ConstraintLayout`, View Binding
- **Design:** Material Components (Material 3)
- **Build:** Gradle 9.6 with Kotlin DSL, Android Gradle Plugin 9.4, version catalog
- **Testing:** JUnit 4 and Espresso

## 🚀 Getting started

```bash
git clone https://github.com/ahsanauddry027/VhangtiChai_AndroidApp_CSE489.git
cd VhangtiChai_AndroidApp_CSE489
```

Open the folder in **Android Studio**, wait for the Gradle sync to finish, then run the `app`
    configuration on an emulator or a device with Android 5.0 (API 21) or newer.

To build from the command line instead:

```bash
./gradlew assembleDebug        # APK in app/build/outputs/apk/debug/
```

On Windows, use `gradlew.bat` in place of `./gradlew`.

🧪 Tests

Command	What it runs
`./gradlew test`	JVM unit tests for the change logic: the examples from the assignment, the
9-digit limit, backspace, and a check that the fewest notes are always used
`./gradlew connectedAndroidTest`	Espresso UI tests for typing, CLEAR, backspace (tap and hold),
and keeping state across activity recreation. They need a device or emulator, and pass on the
Pixel XL, Nexus 10 and Android 5.0 emulators

📂 Project structure

```
app/src/
├── main/
│   ├── java/com/example/vangtichai/
│   │   ├── MainActivity.kt           UI, keypad, backspace, animations, state saving
│   │   └── ChangeCalculator.kt       Change calculation logic
│   └── res/
│       ├── layout/                   Phone portrait + one change-table row
│       ├── layout-land/              Phone landscape
│       ├── layout-sw600dp/           Tablet portrait
│       ├── layout-sw600dp-land/      Tablet landscape
│       ├── values*/                  sizes, strings, colours and themes
│       └── drawable/, mipmap-*/      Backgrounds and the launcher icon
├── test/                             ChangeCalculatorTest.kt
└── androidTest/                      MainActivityTest.kt
```

🎓 About

Built for **CSE 489: Mobile Application Development** (Assignment 1) at BRAC University.

Made by [**@ahsanauddry027**](https://github.com/ahsanauddry027).
```

gradlew

251 lines · Shell script

ABOUT THIS FILE

The wrapper script for macOS and Linux: `./gradlew assembleDebug` runs it (Part 2.4). Gradle generates it, which is why the guide's appendix leaves it out; it is printed here so the PDF covers every file. It must keep LF line endings (`.gitattributes`, line 2). In order, it:

Lines	What it does
1	`#!/bin/sh`: runs with the system's POSIX shell
3–19	Licence header: the script comes from the Gradle project (Apache 2.0)
21–65	Notes from Gradle's template on running and changing the script
67–89	Works out `APP_HOME`, the folder the script is in, following symbolic links
94–103	`warn` and `die`: print a message (and `die` stops the script)
106–115	Detects Cygwin, MSYS/MinGW, macOS (Darwin) and NonStop
121–143	Finds Java: `$JAVA_HOME/bin/java`, or `java` on the PATH, and stops with an error if there is neither
146–162	Raises the open-file limit (`ulimit -n`) where the system allows it
173–201	On Cygwin and MSYS, converts paths to Windows form
205	Default memory for this small launcher process: 64 MB. The build's own memory is set in `gradle.properties`.
213–217	Builds the command line: `-jar gradle/wrapper/gradle-wrapper.jar`, then your arguments
220–249	Stops if `xargs` is missing, then uses it to split `JAVA_OPTS` and `GRADLE_OPTS` safely
251	Replaces itself with Java (`exec`), which runs the wrapper JAR

The wrapper JAR then downloads Gradle 9.6.0 if it is not there yet (`gradle-wrapper.properties`) and starts the build.

gradlew

```
#!/bin/sh

#
# Copyright © 2015 the original authors.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#      https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
# SPDX-License-Identifier: Apache-2.0
#

##############################################################################
#
#   Gradle start up script for POSIX generated by Gradle.
#
#   Important for running:
#
#   (1) You need a POSIX-compliant shell to run this script. If your /bin/sh is
#       noncompliant, but you have some other compliant shell such as ksh or
```

gradlew (continued)

```
#       bash, then to run this script, type that shell name before the whole
#       command line, like:
#
#           ksh Gradle
#
#       Busybox and similar reduced shells will NOT work, because this script
#       requires all of these POSIX shell features:
#         * functions;
#         * expansions «$var», «${var}», «${var:-default}», «${var+SET}»,
#           «${var#prefix}», «${var%suffix}», and «$( cmd )»;
#         * compound commands having a testable exit status, especially «case»;
#         * various built-in commands including «command», «set», and «ulimit».
#
#   Important for patching:
#
#   (2) This script targets any POSIX shell, so it avoids extensions provided
#       by Bash, Ksh, etc; in particular arrays are avoided.
#
#       The "traditional" practice of packing multiple parameters into a
#       space-separated string is a well documented source of bugs and security
#       problems, so this is (mostly) avoided, by progressively accumulating
#       options in "$@", and eventually passing that to Java.
#
#       Where the inherited environment variables (DEFAULT_JVM_OPTS, JAVA_OPTS,
#       and GRADLE_OPTS) rely on word-splitting, this is performed explicitly;
#       see the in-line comments for details.
#
#       There are tweaks for specific operating systems such as AIX, CygWin,
#       Darwin, MinGW, and NonStop.
#
#   (3) This script is generated from the Groovy template
#       https://github.com/gradle/gradle/blob/HEAD/platforms/jvm/plugins-application/src/main/
    resources/org/gradle/api/internal/plugins/unixStartScript.txt
#       within the Gradle project.
#
#       You can find Gradle at https://github.com/gradle/gradle/.
#
##############################################################################

# Attempt to set APP_HOME

# Resolve links: $0 may be a link
app_path=$0

# Need this for daisy-chained symlinks.
while
    APP_HOME=${app_path%"${app_path##*/}"}  # leaves a trailing /; empty if no leading path
    [ -h "$app_path" ]
do
    ls=$( ls -ld "$app_path" )
    link=${ls#*' -> '}
    case $link in             #(
      /*)   app_path=$link ;; #(
      *)    app_path=$APP_HOME$link ;;
    esac
done

# This is normally unused
# shellcheck disable=SC2034
APP_BASE_NAME=${0##*/}
# Discard cd standard output in case $CDPATH is set (https://github.com/gradle/gradle/issues/25036)
APP_HOME=$( cd -P "${APP_HOME:-./}" > /dev/null && printf '%s\n' "$PWD" ) || exit

# Use the maximum available, or set MAX_FD != -1 to use that value.
MAX_FD=maximum

warn () {
    echo "$*"
} >&2

```

gradlew (continued)

```
die () {
    echo
    echo "$*"
    echo
    exit 1
} >&2

# OS specific support (must be 'true' or 'false').
cygwin=false
msys=false
darwin=false
nonstop=false
case "$( uname )" in                #(
  CYGWIN* )         cygwin=true  ;; #(
  Darwin* )         darwin=true  ;; #(
  MSYS* | MINGW* )  msys=true    ;; #(
  NONSTOP* )        nonstop=true ;;
esac

CLASSPATH="\\\"\\\""


# Determine the Java command to use to start the JVM.
if [ -n "$JAVA_HOME" ] ; then
    if [ -x "$JAVA_HOME/jre/sh/java" ] ; then
        # IBM's JDK on AIX uses strange locations for the executables
        JAVACMD=$JAVA_HOME/jre/sh/java
    else
        JAVACMD=$JAVA_HOME/bin/java
    fi
    if [ ! -x "$JAVACMD" ] ; then
        die "ERROR: JAVA_HOME is set to an invalid directory: $JAVA_HOME

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
else
    JAVACMD=java
    if ! command -v java >/dev/null 2>&1
    then
        die "ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH.

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
fi

# Increase the maximum file descriptors if we can.
if ! "$cygwin" && ! "$darwin" && ! "$nonstop" ; then
    case $MAX_FD in #(
      max*)
        # In POSIX sh, ulimit -H is undefined. That's why the result is checked to see if it worked.
        # shellcheck disable=SC2039,SC3045
        MAX_FD=$( ulimit -H -n ) ||
            warn "Could not query maximum file descriptor limit"
    esac
    case $MAX_FD in  #(
      '' | soft) :;; #(
      *)
        # In POSIX sh, ulimit -n is undefined. That's why the result is checked to see if it worked.
        # shellcheck disable=SC2039,SC3045
        ulimit -n "$MAX_FD" ||
            warn "Could not set maximum file descriptor limit to $MAX_FD"
    esac
fi

# Collect all arguments for the java command, stacking in reverse order:
#   * args from the command line
#   * the main class name
#   * -classpath
```

gradlew (continued)

```
#   * -D...appname settings
#   * --module-path (only if needed)
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, and GRADLE_OPTS environment variables.

# For Cygwin or MSYS, switch paths to Windows format before running java
if "$cygwin" || "$msys" ; then
    APP_HOME=$( cygpath --path --mixed "$APP_HOME" )
    CLASSPATH=$( cygpath --path --mixed "$CLASSPATH" )

    JAVACMD=$( cygpath --unix "$JAVACMD" )

    # Now convert the arguments - kludge to limit ourselves to /bin/sh
    for arg do
        if
            case $arg in                                #(
              -*)   false ;;                            # don't mess with options #(
              /?*)  t=${arg#/} t=/${t%%/*}              # looks like a POSIX filepath
                    [ -e "$t" ] ;;                      #(
              *)    false ;;
            esac
        then
            arg=$( cygpath --path --ignore --mixed "$arg" )
        fi
        # Roll the args list around exactly as many times as the number of
        # args, so each arg winds up back in the position where it started, but
        # possibly modified.
        #
        # NB: a `for` loop captures its iteration list before it begins, so
        # changing the positional parameters here affects neither the number of
        # iterations, nor the values presented in `arg`.
        shift                   # remove old arg
        set -- "$@" "$arg"      # push replacement arg
    done
fi


# Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to this
    script.
DEFAULT_JVM_OPTS='"-Xmx64m" "-Xms64m"'

# Collect all arguments for the java command:
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, and optsEnvironmentVar are not allowed to contain shell fragments,
#     and any embedded shellness will be escaped.
#   * For example: A user cannot expect ${Hostname} to be expanded, as it is an environment variable
    and will be
#     treated as '${Hostname}' itself on the command line.

set -- \
        "-Dorg.gradle.appname=$APP_BASE_NAME" \
        -classpath "$CLASSPATH" \
        -jar "$APP_HOME/gradle/wrapper/gradle-wrapper.jar" \
        "$@"

# Stop when "xargs" is not available.
if ! command -v xargs >/dev/null 2>&1
then
    die "xargs is not available"
fi

# Use "xargs" to parse quoted args.
#
# With -n1 it outputs one arg per line, with the quotes and backslashes removed.
#
# In Bash we could simply go:
#
#   readarray ARGS < <( xargs -n1 <<<"$var" ) &&
#   set -- "${ARGS[@]}" "$@"
#
# but POSIX shell has neither arrays nor command substitution, so instead we
# post-process each arg (as a line of input to sed) to backslash-escape any
```

gradlew (continued)

```
# character that might be a shell metacharacter, then use eval to reverse
# that process (while maintaining the separation between arguments), and wrap
# the whole thing up as a single "set" statement.
#
# This will of course break if any of these variables contains a newline or
# an unmatched quote.
#

eval "set -- $(
        printf '%s\n' "$DEFAULT_JVM_OPTS $JAVA_OPTS $GRADLE_OPTS" |
        xargs -n1 |
        sed ' s~[^-[:alnum:]+,./:=@_]~\\&~g; ' |
        tr '\n' ' '
    )" '"$@"'

exec "$JAVACMD" "$@"
```

gradlew.bat

94 lines · Windows batch file

ABOUT THIS FILE

The same wrapper for Windows: `gradlew.bat assembleDebug` runs it. Like `gradlew`, Gradle generates it, and it must keep CRLF line endings (`.gitattributes`, line 3).

Lines	What it does
1–17	Licence header (Apache 2.0)
19	`@echo off`, unless the `DEBUG` variable is set
27	`setlocal`: the variables set below stay inside this script
29–36	Works out `APP_HOME`, the script's own folder (`%~dp0`), and removes `.` and `..` from it
39	Default memory for the launcher process: 64 MB
41–68	Finds `java.exe` on the PATH, or under `JAVA_HOME` (from line 56), with an error message if neither works
70–77	Runs the wrapper JAR with your arguments (`%*`)
79–94	Hands Gradle's exit code back to whoever ran the script

gradlew.bat

```
@rem
@rem Copyright 2015 the original author or authors.
@rem
@rem Licensed under the Apache License, Version 2.0 (the "License");
@rem you may not use this file except in compliance with the License.
@rem You may obtain a copy of the License at
@rem
@rem      https://www.apache.org/licenses/LICENSE-2.0
@rem
@rem Unless required by applicable law or agreed to in writing, software
@rem distributed under the License is distributed on an "AS IS" BASIS,
@rem WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
@rem See the License for the specific language governing permissions and
@rem limitations under the License.
@rem
@rem SPDX-License-Identifier: Apache-2.0
@rem

@if "%DEBUG%"=="" @echo off
@rem ##########################################################################
@rem
@rem  Gradle startup script for Windows
```

gradlew.bat (continued)

```
@rem
@rem ##########################################################################

@rem Set local scope for the variables with windows NT shell
if "%OS%"=="Windows_NT" setlocal

set DIRNAME=%~dp0
if "%DIRNAME%"=="" set DIRNAME=.
@rem This is normally unused
set APP_BASE_NAME=%~n0
set APP_HOME=%DIRNAME%

@rem Resolve any "." and ".." in APP_HOME to make it shorter.
for %%i in ("%APP_HOME%") do set APP_HOME=%%~fi

@rem Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to
    this script.
set DEFAULT_JVM_OPTS="-Xmx64m" "-Xms64m"

@rem Find java.exe
if defined JAVA_HOME goto findJavaFromJavaHome

set JAVA_EXE=java.exe
%JAVA_EXE% -version >NUL 2>&1
if %ERRORLEVEL% equ 0 goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH. 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:findJavaFromJavaHome
set JAVA_HOME=%JAVA_HOME:"=%
set JAVA_EXE=%JAVA_HOME%/bin/java.exe

if exist "%JAVA_EXE%" goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is set to an invalid directory: %JAVA_HOME% 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:execute
@rem Setup the command line

set CLASSPATH=


@rem Execute Gradle
"%JAVA_EXE%" %DEFAULT_JVM_OPTS% %JAVA_OPTS% %GRADLE_OPTS% "-Dorg.gradle.appname=%APP_BASE_NAME%"
    -classpath "%CLASSPATH%" -jar "%APP_HOME%\gradle\wrapper\gradle-wrapper.jar" %*

:end
@rem End local scope for the variables with windows NT shell
if %ERRORLEVEL% equ 0 goto mainEnd

:fail
rem Set variable GRADLE_EXIT_CONSOLE if you need the _script_ return code instead of
rem the _cmd.exe /c_ return code!
set EXIT_CODE=%ERRORLEVEL%
if %EXIT_CODE% equ 0 set EXIT_CODE=1
if not ""=="%GRADLE_EXIT_CONSOLE%" exit %EXIT_CODE%
exit /b %EXIT_CODE%

```

gradlew.bat (continued)

```
:mainEnd
if "%OS%"=="Windows_NT" endlocal

:omega
```

Binary files

ABOUT THESE FILES

Git also tracks 27 binary files. They cannot be printed as text, so the table lists what each one is and where this PDF shows it.

The ten `mipmap-*dpi` bitmaps deserve a closer look. They are still the project template's default icon, the green Android robot. Android 5.0 to 7.1 (API 21 to 25), which VangtiChai supports, cannot show adaptive icons and fall back to these bitmaps, so on those phones the app's icon is the robot rather than the VangtiChai design. Part 6 lists regenerating them as an improvement.

File	Size	What it is	Shown in
app/src/main/res/mipmap-hdpi/ic_launcher.webp	1.4 KB	Square launcher icon for Android 5.0 to 7.1, hdpi, 72 × 72 px: the template's robot	Figure 10
app/src/main/res/mipmap-hdpi/ic_launcher_round.webp	2.8 KB	Round launcher icon for Android 5.0 to 7.1, hdpi, 72 × 72 px: the template's robot	Figure 10
app/src/main/res/mipmap-mdpi/ic_launcher.webp	1.0 KB	Square launcher icon for Android 5.0 to 7.1, mdpi, 48 × 48 px: the template's robot	Figure 10
app/src/main/res/mipmap-mdpi/ic_launcher_round.webp	1.7 KB	Round launcher icon for Android 5.0 to 7.1, mdpi, 48 × 48 px: the template's robot	Figure 10
app/src/main/res/mipmap-xhdpi/ic_launcher.webp	1.9 KB	Square launcher icon for Android 5.0 to 7.1, xhdpi, 96 × 96 px: the template's robot	Figure 10
app/src/main/res/mipmap-xhdpi/ic_launcher_round.webp	3.8 KB	Round launcher icon for Android 5.0 to 7.1, xhdpi, 96 × 96 px: the template's robot	Figure 10
app/src/main/res/mipmap-xxhdpi/ic_launcher.webp	2.8 KB	Square launcher icon for Android 5.0 to 7.1, xxhdpi, 144 × 144 px: the template's robot	Figure 10
app/src/main/res/mipmap-xxhdpi/ic_launcher_round.webp	5.8 KB	Round launcher icon for Android 5.0 to 7.1, xxhdpi, 144 × 144 px: the template's robot	Figure 10
app/src/main/res/mipmap-xxxhdpi/ic_launcher.webp	3.8 KB	Square launcher icon for Android 5.0 to 7.1, xxxhdpi, 192 × 192 px: the template's robot	Figure 10
app/src/main/res/mipmap-xxxhdpi/ic_launcher_round.webp	7.6 KB	Round launcher icon for Android 5.0 to 7.1, xxxhdpi, 192 × 192 px: the template's robot	Figure 10
docs/images/emulator-android-5-after.png	187.9 KB	Android 5.0 emulator after the fix (half size), 720 × 1280 px	Figure 9
docs/images/emulator-android-5-before.png	263.8 KB	Android 5.0 emulator before the row-colour fix (half size), 720 × 1280 px	Figure 9

File	Size	What it is	Shown in
docs/images/emulator-nexus-10-688.png	149.2 KB	Nexus 10 emulator, portrait, 688 Taka (half size), 800 × 1280 px	Figure 8
docs/images/emulator-nexus-10-land-6880.png	135.5 KB	Nexus 10 emulator, landscape, 6880 Taka (half size), 1280 × 800 px	Figure 8
docs/images/emulator-pixel-xl-688.png	154.6 KB	Pixel XL emulator, portrait, 688 Taka (half size), 720 × 1280 px	Figure 7
docs/images/emulator-pixel-xl-land-6880.png	131.9 KB	Pixel XL emulator, landscape, 6880 Taka (half size), 1280 × 720 px	Figure 7
docs/images/launcher-icons.png	35.2 KB	README image: the launcher icon as a circle, a rounded square and the themed version, 1376 × 472 px	Figure 2
docs/images/logo.png	15.0 KB	README logo: the round launcher icon, 288 × 288 px	Cover
docs/images/phone-68.png	153.8 KB	Screenshot: phone, portrait, 68 Taka, 822 × 1366 px	Figure 1
docs/images/phone-688.png	154.6 KB	Screenshot: phone, portrait, 688 Taka, 822 × 1366 px	Figures 1 and 3; cover
docs/images/phone-6880.png	158.2 KB	Screenshot: phone, portrait, 6880 Taka, 822 × 1366 px	Figure 1
docs/images/phone-land-688.png	123.5 KB	Screenshot: phone, landscape, 688 Taka, 1366 × 822 px	Figure 4
docs/images/phone-land-6880.png	127.0 KB	Screenshot: phone, landscape, 6880 Taka, 1366 × 822 px	Figure 4
docs/images/phone-limit.png	162.2 KB	Screenshot: the 9-digit limit, 822 × 1366 px	Figure 6
docs/images/tablet-688.png	126.1 KB	Screenshot: tablet, portrait, 688 Taka, 800 × 1232 px	Figure 5
docs/images/tablet-land-688.png	115.6 KB	Screenshot: tablet, landscape, 688 Taka, 1280 × 752 px	Figure 5
gradle/wrapper/gradle-wrapper.jar	44.4 KB	The Gradle wrapper's code. `gradlew` and `gradlew.bat` run it, and it downloads and starts Gradle 9.6.0	Not an image

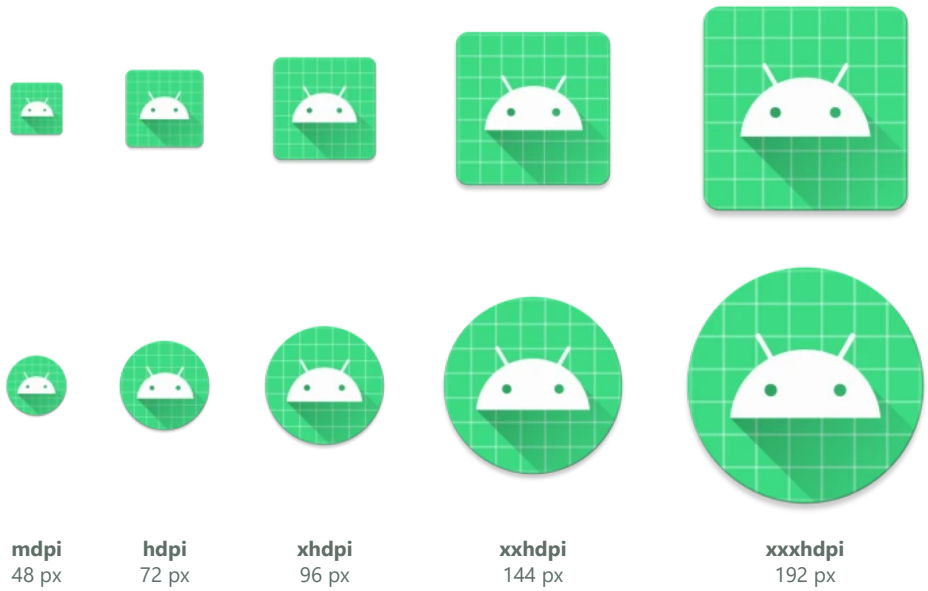


Figure 10. *The bitmap launcher icons in* `mipmap-mdpi` *to* `mipmap-xxxhdpi`*, square (top) and round (middle), at half their pixel size. Android 5.0 to 7.1 use them because they cannot show adaptive icons. They are still the project template's robot, not the VangtiChai design in Figure 2.*